Identifying Basins from Sparse Koopman Supports

Anonymous Authors

Abstract

Koopman autoencoders (KAEs) seek a higher-dimensional latent representation in which nonlinear dynamics evolve linearly. However, many interesting systems have multiple basins of attraction, and both theoretical and empirical work has shown these multi-basin systems cannot generally admit a single finite-dimensional Koopman embedding under standard assumptions. A global KAE is therefore forced to collapse basin-specific dynamics when trained over multiple basins. We posit that encoders inducing latent sparsity with a sparse coding objective will provide latent supports as an inspectable basin-modeling principle for Koopman autoencoders. We use these encoders in training Sparse Koopman Autoencoders (SKAEs) without basin labels or other regime annotations, and treat the learned latent supports as model-produced regime variables only after training. Across a range of procedurally generated multi-basin systems and chaotic flows, SKAEs using sparse encoders have superior forecasting performance compared to dense-latent KAEs. SKAEs also produce latent supports that are both essential for the quality of the representation and useful for identifying basins on held-out basin interior states, whereas dense-latent KAEs collapse to mixed supports. Finally, we show that learned support families can route a second-stage bank of local affine latent models, improving controlled multibasin forecasting over a strong global-K LISTA baseline. These results identify sparse latents and their corresponding supports as label-free, interpretable regime variables for Koopman learning in nonlinear systems with multiple local dynamical laws.

1 Introduction

Learning representations for nonlinear dynamical systems remains a central problem in machine learning, scientific computing, and control. A particularly useful goal is to embed nonlinear dynamics into coordinates in which the evolution is linear, or at least locally linear, because linear models can be combined with mature tools for prediction, estimation, and control, including model predictive control and linear quadratic regulation (Mayne et al., 2000; Grüne and Pannek, 2017; Korda and Mezić, 2018; Kalman, 1960; Mamakoukas et al., 2019). This goal is also well motivated theoretically. Classical dynamical-systems results provide local topological linearizations near hyperbolic fixed points (Grobman, 1959; Hartman, 1960), while Koopman eigenfunction constructions can yield linearizing coordinates on basins of attraction for stable equilibria and periodic orbits (Lan and Mezić, 2013; Kvalheim and Revzen, 2021). Thus, nonlinear systems may admit linear dynamics with a specific change of coordinates.

The difficulty is that these results are primarily existential. They establish when useful coordinates can exist, but they do not by themselves provide a finite-data procedure for constructing such coordinates. Koopman operator theory formalizes the linearization idea by lifting nonlinear state evolution to a linear operator acting on observables (Koopman, 1931; Koopman and Neumann, 1932; Mezić, 2005; Brunton et al., 2022). This viewpoint has inspired data-driven approximations of Koopman dynamics, such as DMD and eDMD (Schmid, 2010; Williams et al., 2015, 2016), and more recently to Koopman autoencoders, which learn an encoder from state space to a latent representation, a linear latent transition matrix, and a decoder back to state space (Takeishi et al., 2017;

Lusch et al., 2018; Otto and Rowley, 2019; Azencot et al., 2020; Shi and Meng, 2022; Fathi et al., 2024). These models provide an appealing route from existence results to practical learning: rather than hand-designing observables, they attempt to learn the embedding directly from trajectory data.

A central challenge for Koopman approximations is that many practically interesting nonlinear systems typically contain multiple equilibria or basins of attraction; for example, a damped mechanical system can settle into different wells, a biological or chemical model can approach different stable equilibria, and a controlled system can move between regions with distinct local dynamics. However, theory shows that these systems cannot generally admit a single finite-dimensional global linearization with a continuous encoder-decoder pair (Lan and Mezić, 2013; Brunton et al., 2016; Pan and Duraisamy, 2024; Liu et al., 2025), and empirical evidence supports this in practice (Fathi et al., 2024). In such multistable systems, it is possible to linearize the dynamics within basins, but the linearizations needed in different basins of attraction are generally incompatible with each other. Therefore, for multi-attractor systems, the right objective is not to force all basins through one continuous global linearization, but to discover and use distinct local linearizations.

This paper proposes inducing sparse latent structure as a practical and interpretable method for Koopman learning of multi-attractor systems. By inducing the state to activate only a small subset of coordinates in lifted latent space, we can incentivize different regions of state to occupy different active coordinates, since nearby states should have similar latent embeddings, and thus enable local linearizations. In this sparse representation, nonzero coefficient values in a latent vector can provide continuous coordinates for prediction within a selected region, while the active indices provide a discrete support object that can be inspected across states and used to select local linearizations.

Sparse coding and LISTA-style encoders provide the right inductive bias for this purpose. They naturally induce piecewise-linear, union-of-subspaces structure (Olshausen and Field, 1996; Chen et al., 1998; Donoho and Elad, 2003; Gregor and LeCun, 2010), which is well-matched to this setting where different attractors or basins require different local linearizations in theory. We hypothesize that sparse-latent Koopman autoencoders will assign local dynamical regimes to distinct latent support families, yielding a learned regime variable without basin supervision. Then, periodically decoding and re-encoding the state can refresh the quality of the latent embedding over time by re-selecting the correct local dynamics when trajectories move across the state space (Fathi et al., 2024).

Contributions. This paper makes four contributions. First, we formulate multibasin Koopman learning as a label-free representation problem in which a useful finite-dimensional latent should encode both within-regime coordinates and a discrete indicator of the active local dynamics. Second, we show how sparse Koopman autoencoders can expose this indicator through learned latent supports and support families, without using basin labels, basin counts, or trajectory-to-basin assignments during training. Third, across controlled multibasin systems and an external chaotic-flow forecasting stress test, we show that sparse latents improve rollout accuracy, that active coordinate identities are functionally required for prediction, and that LISTA support families identify basin interiors after training. Fourth, we show that those same learned support families can route a second-stage bank of local affine latent predictors, yielding a strong controlled-multibasin forecasting improvement over the matched global-K LISTA model.

2 Problem formulation

The introduction motivates a multibasin version of Koopman learning. Given sampled trajectories of a nonlinear system, the aim is to learn a state-reconstructing latent representation whose evolution is linear, without being told which basin or local dynamical regime generated each observation. We formulate the problem at the cadence of the stored benchmark observations. For each system, let $\mathcal{X} \subseteq \mathbb{R}^{d_x}$ be the observed state space and let

$$x_{k+1} = F_{\Delta t}(x_k), \qquad x_k \in \mathcal{X}, \tag{1}$$

where $x_k \in \mathbb{R}^{d_x}$ is the state at the kth stored sample, $F_{\Delta t}$ denotes the transformation to the next discrete observation step based on underlying dynamics, and Δt is the system-specific observation interval. Forecasting is therefore defined by repeatedly applying the map $F_{\Delta t}$; a horizon of h means h applications of $F_{\Delta t}$; we denote k compositions as $F_{\Delta t}^{(k)}$. Because Δt may differ across systems, equal values of h may yield different physical time.

Basins of attraction. Many systems of interest are multistable: different initial conditions can approach different long-run behaviors. A fixed point is a state x^* satisfying $F_{\Delta t}(x^*) = x^*$. More generally, a trajectory may approach an attractor A, such as a stable equilibrium, limit cycle, countable finite set, or chaotic invariant set. The basin of attraction of A is the set of initial conditions whose trajectories converge to A:

$$\mathcal{B}(A) = \left\{ x_0 \in \mathbb{R}^{d_x} : \inf_{a \in A} \left\| F_{\Delta t}^{(k)}(x_0) - a \right\|_2 \to 0 \text{ as } k \to \infty \right\}. \tag{2}$$

When multiple attractors coexist, their basins partition the state space up to basin boundaries. This defines the setup for this paper. The task is to learn a representation that can predict while preserving inspectable information about where the state lies in the multibasin geometry.

Koopman theory. The stored-step map $F_{\Delta t}$ induces a Koopman operator $\mathcal{K}$ on scalar observables $g : \mathcal{X} \to \mathbb{R}$ by composition, $(\mathcal{K}g)(x) = g(F_{\Delta t}(x))$. Although $F_{\Delta t}$ may be nonlinear, $\mathcal{K}$ is linear on the space of observables. A finite-dimensional Koopman approximation searches for a vector observable $\Phi : \mathcal{X} \to \mathbb{R}^{d_z}$ and a matrix K such that $\Phi(F_{\Delta t}(x)) \approx K\Phi(x)$, while retaining enough information to reconstruct the state. Additional background is provided in section R.

Koopman autoencoders. We study this setting with Koopman autoencoders (KAEs). The model learns an encoder $E_\theta : \mathbb{R}^{d_x} \to \mathbb{R}^{d_z}$, a decoder $D_\phi : \mathbb{R}^{d_z} \to \mathbb{R}^{d_x}$, and a latent linear transition matrix $K \in \mathbb{R}^{d_z \times d_z}$. For observation x_k at arbitrary timestep k, the corresponding open-loop h-step forecast is

$$\hat{x}_{k+h} = D_\phi\Big(K^h E_\theta(x_k)\Big). \tag{3}$$

The same matrix K is applied at every step of the rollout, so any state-dependent structure required for prediction must be encoded in the learned representation rather than in a supplied regime label.

Training windows. Training uses windows of adjacent stored observations. For a batch of B windows of rollout length L, $\{x_{b,k}, x_{b,k+1}, \ldots, x_{b,k+L}\}_{b=1}^{B}$, where b indexes windows and ℓ indexes offsets within a window, the model encodes each observed state, rolls out from the initial encoded

state, and decodes both reconstructed observations and latent forecasts. The encoded/reconstructed quantities are defined for $\ell = 0, \ldots, L$, while rollout/forecast quantities are defined for $\ell = 1, \ldots, L$:

$$z^{\text{enc}}_{b,k+\ell} = E_\theta(x_{b,k+\ell}), \quad z^{\text{roll}}_{b,k+\ell} = K^\ell z^{\text{enc}}_{b,k}, \quad x^{\text{rec}}_{b,k+\ell} = D_\phi(z^{\text{enc}}_{b,k+\ell}), \quad \hat{x}_{b,k+\ell} = D_\phi(z^{\text{roll}}_{b,k+\ell}). \tag{4}$$

The training losses described later are built from these reconstructed and rolled-out quantities. Each application of K is interpreted as one stored benchmark step, matching the observation map in equation (1).

Model discovers basins unsupervised. In the intended training and deployment setting, the sparse-support mechanism is not given basin labels, the number of basins, or trajectory-to-basin assignments. When benchmark basin labels are available, they are reserved for post-hoc evaluation; they are not used to choose support families, train the encoder or transition, or select supports. The block-diagonal and soft-block transition rows are architecture diagnostics: for those ablations only, the fixed number of latent transition groups is set from benchmark generator metadata, as detailed in section A.

3 Method

3.1 Sparse supports as local regime variables

A finite-dimensional Koopman representation for the multi-basin systems in section 2 should encode two complementary quantities: (i) continuous coordinates that linearize the dynamics within a basin, and (ii) indicate which local linearization is active. We use sparsity to couple these two roles. If the state x is represented by a sparse latent code z, then the support of z can act as a regime variable, while the nonzero coefficient values provide coordinates for the corresponding local linear representation.

Given an overcomplete dictionary D, sparse coding estimates z by solving the Lasso objective (Beck and Teboulle, 2009)

$$z^\star(x) = \arg\min_{z \in \mathbb{R}^{d_z}} \frac{1}{2} \|x - Dz\|_2^2 + \lambda_{\text{sc}} \|z\|_1, \qquad \lambda_{\text{sc}} > 0. \tag{5}$$

which finds the code z with the fewest non-zero coefficients to reconstruct the input x. In other words, it finds the sparsest solution to the under-specified problem.

The classical solver to this problem (ISTA (Beck and Teboulle, 2009)) can be unrolled into a depth-L network of ISTA iterations, yielding learned ISTA (LISTA) (Gregor and LeCun, 2010); in our setting, LISTA is an encoder yielding the sparse code $z = E_\theta(x)$, where the *support* is the set of active z coordinates. With a linear decoder, the support determines which decoder columns participate in reconstructing x, and the associated nonzero coefficients specify coordinates within that selected reconstruction subspace. When D and z are learned jointly, as in sparse coding (Olshausen and Field, 1996), this induces a data-adaptive partition of the input state space where each region can be locally reconstructed with a code having the same support.

We therefore posit that combining Koopman representations and sparse coding objectives in a sparse Koopman autoencoder (SKAE) implicitly captures multi-basin attractor dynamics without explicit basin labels in training. The Koopman objective encourages the continuous coefficients on each support to evolve linearly, while the sparsity objective encourages different local regimes to use different active coordinates. Thus, the model can represent a multi-basin system using a discrete support pattern to identify the active basin and continuous coefficients for its localized Koopman

(linear) representation. When ground-truth basin labels are available in benchmarks, we use them only after training to assess support–basin alignment or to construct diagnostic interventions.

3.2 Koopman autoencoder and sparse encoder

The predictor follows the KAE formulation in section 2: an encoder, linear decoder, and latent transition, all of which are optimized jointly, produce the rollout in equation (3). Like in sparse coding, (Gregor and LeCun, 2010) we constrain the columns of the decoder $D_\phi = D$ to be unit norm to avoid degenerate solutions. We experiment with K that is dense, block diagonal, or softly block-regularized.

Sparse encoding. Our main sparse encoder is LISTA (Gregor and LeCun, 2010). It first computes a dense pre-code $c_t = f_\theta(x_t)$, then applies learned shrinkage refinements for $q = 0, \dots, Q-1$:

$$u_t^{(0)} = \mathrm{shrink}(c_t, \tau), \qquad u_t^{(q+1)} = \mathrm{shrink}(S u_t^{(q)} + c_t, \tau), \qquad z_t = u_t^{(Q)}. \tag{6}$$

Here, $\mathrm{shrink}(a, \tau) = \mathrm{sign}(a)\max(|a| - \tau, 0)$ is applied element-wise, S is learned, and τ controls the threshold scale for shrinkage. The thresholding step creates exact zeros, so the encoder explicitly selects active latent coordinates while learning the selection rule from the prediction objective.

Transition families. We vary the structure of K separately from the encoder. The dense transition leaves K unrestricted. The block-diagonal transition partitions latent coordinates into M fixed groups and constrains

$$K_{\mathrm{block}} = \mathrm{blkdiag}(K_1, \dots, K_M). \tag{7}$$

We also use a soft-block LISTA ablation that keeps K dense but penalizes cross-group entries,

$$\mathcal{L}_{\mathrm{offblock}} = \sum_{i,j:\, g(i) \neq g(j)} |K_{ij}|, \tag{8}$$

where $g(i)$ is the fixed architectural group containing coordinate i.

3.3 Training objective

Training uses adjacent observation windows. For a batch of B windows with prediction horizon L, the model produces reconstructed future states $x^{\mathrm{rec}}_{b,k+\ell}$, latent rollouts $z^{\mathrm{roll}}_{b,k+\ell}$, encoded future latents $z^{\mathrm{enc}}_{b,k+\ell}$, and decoded rollout predictions $\hat{x}_{b,k+\ell}$ as defined in equation (4). The rollout is seeded by the initial state, and the following sums are over offsets $\ell = 1, \dots, L$:

$$\bar{\mathcal{L}}_{\mathrm{pred}} = \frac{1}{BL\sqrt{d_x}} \sum_{b,\ell} \|\hat{x}_{b,k+\ell} - x_{b,k+\ell}\|_2, \qquad \bar{\mathcal{L}}_{\mathrm{rec}} = \frac{1}{BL\sqrt{d_x}} \sum_{b,\ell} \left\|x^{\mathrm{rec}}_{b,k+\ell} - x_{b,k+\ell}\right\|_2, \tag{9}$$

$$\bar{\mathcal{L}}_{\mathrm{lin}} = \frac{1}{BL} \sum_{b,\ell} \left\|z^{\mathrm{roll}}_{b,k+\ell} - z^{\mathrm{enc}}_{b,k+\ell}\right\|_2, \qquad \bar{\mathcal{L}}_{\mathrm{sp}} = \frac{1}{BL} \sum_{b,\ell} \left\|z^{\mathrm{roll}}_{b,k+\ell}\right\|_1. \tag{10}$$

The observation-space terms are normalized by $\sqrt{d_x}$ so that their scale is comparable across systems of different state dimension. The latent terms are left in their native scale because the latent dimension is fixed within each comparison.

The total objective is

$$\mathcal{L} = \frac{1}{L}\left(\lambda_{\mathrm{pred}}\bar{\mathcal{L}}_{\mathrm{pred}} + \lambda_{\mathrm{rec}}\bar{\mathcal{L}}_{\mathrm{rec}} + \lambda_{\mathrm{lin}}\bar{\mathcal{L}}_{\mathrm{lin}} + \lambda_{\mathrm{sp}}\bar{\mathcal{L}}_{\mathrm{sp}}\right) + \mathcal{L}_{\mathrm{struct}}. \tag{11}$$

Here $\mathcal{L}_{\text{struct}} = 0$ for the dense and block-diagonal transition variants. For the soft-block transition, $\mathcal{L}_{\text{struct}} = \lambda_{\text{off}} \mathcal{L}_{\text{offblock}}$, which probes whether a soft transition grouping is sufficient to induce useful support structure. The prediction and latent-linearity terms make the representation useful for Koopman rollout, the reconstruction term keeps the code tied to the observed state, and the sparsity term encourages selective activation.

3.4 Support objects

Let $z = E_\theta(x) \in \mathbb{R}^{d_z}$. A *support rule* first converts the latent code z into a Boolean active-coordinate vector $m(x) = \{|z_i| > 0\} \in \{0,1\}^{d_z}$, where $m_i(x) = 1$ means that latent coordinate i is active for state x. Equivalently, the exact support is the index set $S(x) = \{i : m_i(x) = 1\}$. Exact supports defined by an arbitrary threshold can be unstable: a small perturbation in x-space can change the support, so one basin may be covered by several nearby or partially overlapping active-coordinate vectors. We therefore distinguish exact supports from *support families*, which group nearby Boolean support vectors and test whether these groups form basin-scale objects. Support families in general are more robust and lead to less basin fragmentation.

The main construction is the absolute-threshold support family F_{abs}. First, $m_{\text{abs},i}(x) = \mathbf{1}\{|z_i| > 10^{-3}\}$, and $S_{\text{abs}}(x) = \{i : m_{\text{abs},i}(x) = 1\}$. The family label $F_{\text{abs}}(x)$ is then assigned by grouping exact m_{abs} masks whose active-index sets substantially overlap. On an evaluation collection $\mathcal{X}$, we count distinct Boolean masks and visit them from most to least frequent. Each family f stores one representative mask r_f; a new mask m joins the family with the largest Jaccard score

$$J(m, r_f) = \frac{\sum_i \mathbf{1}\{m_i = 1 \text{ and } r_{f,i} = 1\}}{\sum_i \mathbf{1}\{m_i = 1 \text{ or } r_{f,i} = 1\}},$$

with $J(0,0) = 1$, if that score is at least $\tau = 0.5$; otherwise it starts a new family. Pseudocode and implementation details are provided in section F.

F_{abs} is the main basin-alignment and local-linearization object in the paper. The appendix also defines a stable support component C_{stab}, which groups support families by empirical support-flow fate. We keep C_{stab} as a diagnostic extension rather than the main forecasting route because the matched local-linearization controls show that F_{abs} routing is already at least as strong for the controlled multibasin forecasting task.

3.5 Periodic re-encoding

If a support indicates the currently relevant local dynamics, a long rollout should be able to update that support. We therefore evaluate periodic re-encoding, following Fathi et al. (2024). Starting from $z_k = E_\theta(x_k)$ and a re-encoding period m, the model advances for m Koopman steps, decodes the predicted state, and encodes that prediction again:

$$\tilde{z}_{k+m} = K^m z_k, \qquad \tilde{x}_{k+m} = D_\phi(\tilde{z}_{k+m}), \qquad z_{k+m} = E_\theta(\tilde{x}_{k+m}). \tag{12}$$

The same procedure is repeated over the rollout horizon. Each rollout uses only the model's predicted state; it does not use the true future state, a basin label, or an oracle for transitions between basins. For each dynamical system we tune a separate re-encoding period m on the training set.

4 Experiments

The experiments aim to evaluate if finite-dimensional Koopman representations can be learned for multi-attractor systems implicitly without resorting to side information such as basin labels. We first

discuss the quality of learned Koopman representations from sparse KAEs via a forecasting-based evaluation. Next, in the context of sparse inference, we evaluate the importance of the specific active set of coordinates for forecasting within a basin of attraction. We then evaluate alignment of support families F_{abs} with withheld basin labels. Finally, we ask whether a support-stable basin object can be used as a route for downstream local affine latent predictors.

Benchmark systems. The main benchmark for our experiments contains 15 procedurally generated two-dimensional multi-basin systems. These systems are constructed by specifying potential wells around known attractor coordinates, and superimposing dynamics that encourage transitions to different far-away regimes; see Appendix C. Because we specify the locations of the potential wells, the basin labels are well-defined and used only for evaluation. We also use a subset of 10 chaotic systems from the Dysts repository (Gilpin, 2021, 2023) with the step size dt multiplied by a factor of 30 as an additional forecasting test. Analytic expressions and more details are in Appendix C, D.

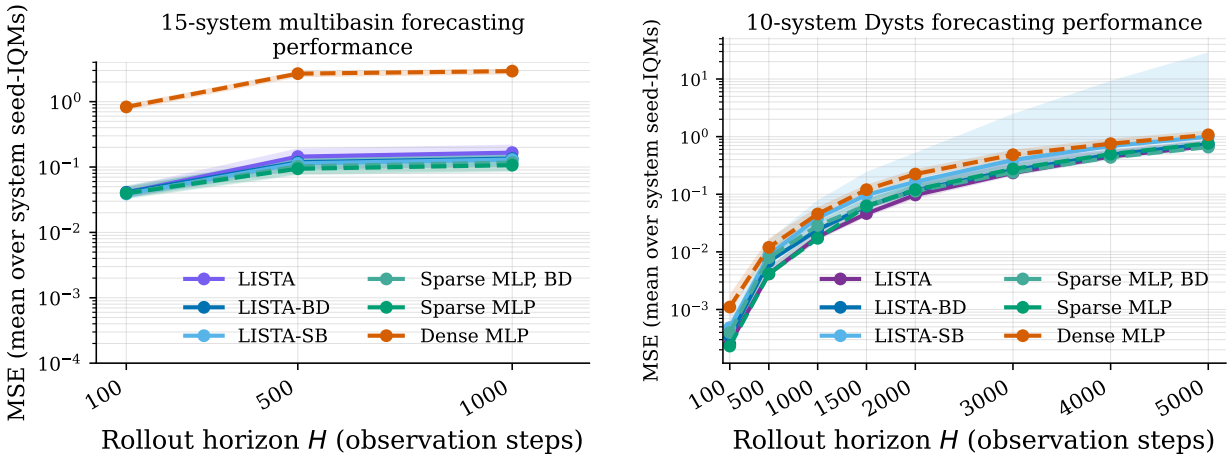
Training models. We compare the six KAE models shown in the upper block of table 1 by varying the encoder and latent transition structure. We test three LISTA sparse encoders (where LISTA-BD denotes block diagonal latent transitions and LISTA-SB denotes soft-block regularization), two sparse-latent MLP controls applying the sparsity penalty $\bar{\mathcal{L}}_{\text{sp}}$ (10) with dense (Fathi et al., 2024) or block-diagonal (BD) transitions, and a dense-latent MLP control baseline with no sparsity incentive. Each model is trained for 15 random seeds on each system before evaluation. Exact training details are reported in Appendix A.

Statistical details. Point estimates are interquartile means (IQM) over seeds within each system to reduce seed uncertainty (Agarwal et al., 2021), then arithmetic means across systems. Horizon-trend lines use the same point estimate as the tables, with seed-bootstrap bands that summarize uncertainty over seeds for an average system. Statistical significance is rigorously tested with paired Wilcoxon signed-rank tests and Holm corrections; details are provided in Appendix I.

4.1 Forecasting experiments

Sparse encoders learn high-quality Koopman representations. The quality of the representation learned by a KAE is primarily measured by its ability to make accurate, multi-step forecasts. Thus, we evaluate the forecasting performance of trained sparse and dense-latent KAEs on the two benchmarks spanning 25 systems. Many of the model architectures are biased toward sparse inference by varying degrees. In particular, LISTA networks are specifically designed for performing sparse inference. Since our method combines the sparse inference and Koopman objectives, we do not necessarily expect these architectures to be the most performant in our setting. Forecasting is evaluated on held-out trajectories at the stored observation cadence and summarized by mean squared error (MSE).

The results (figure 1 and table 1) show that sparse models yield a significant reduction of MSE forecasting error at all horizons on both sets of systems. In particular, at longer horizons, the dense MLP has 17-27x larger MSE at $H1000$ on the multibasin systems and about 1.5x larger MSE at $H4000$ on the Dysts systems compared to most of the sparse models (the LISTA-SB model outperforms the baseline but is noticeably worse than the rest). On the other hand, there are marginal differences between MSE predictions among the sparse-latent encoders. The strongest standalone control, RBF-dictionary EDMD, beats the dense-latent MLP on the controlled multibasin benchmark but remains far behind every sparse KAE row; on the Dysts $dt\times30$ systems, all standalone



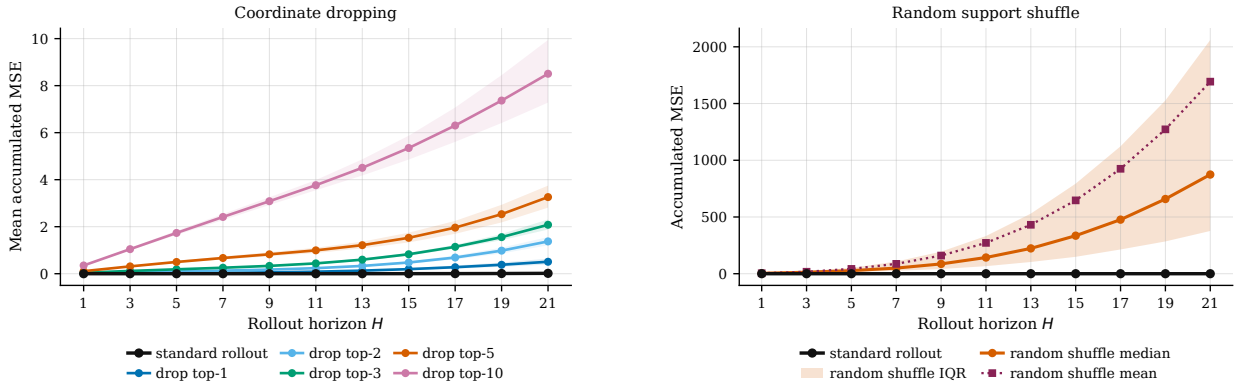
(a) 15-system multibasin benchmark.

(b) 10-system Dysts $dt\times30$ benchmark.

Figure 1: Forecasting horizon trends. Lines show arithmetic means across per-system seed-IQM MSEs on a log scale. Bands show fixed-system seed-bootstrap 95% intervals after system-wise log-relative normalization, anchored to the displayed mean, so they summarize seed uncertainty within an average system.

Table 1: Forecasting performance on the controlled multibasin and Dysts benchmarks. Values are MSEs; lower is better. Each cell is an arithmetic mean across systems after summarizing seeds within each system by IQM. Superscripts on KAE rows mark Holm-corrected tests against the dense-latent MLP baseline: multibasin forecasting cells use within-system Wilcoxon/Holm tests and Dysts cells use exact sign tests across systems ($*$, $p < 0.05$). Standalone state-space and local-linear controls are descriptive reviewer-facing baselines. **Bold** marks the best KAE entry in each column. The Dysts k-means H4000 cell has finite entries for 9/10 systems because one rollout diverged numerically.

	Multibasin, 15 systems			Dysts $dt\times30$, 10 systems		
Model	H100	H500	H1000	H100	H2000	H4000
LISTA	0.0407*	0.144*	0.166*	2.53×10^{-4}*	**0.0972***	0.452*
LISTA-BD	0.0411*	0.118*	0.135*	3.38×10^{-4}	0.117*	0.485*
LISTA-SB	**0.0387***	0.114*	0.130*	4.85×10^{-4}	0.165	0.686
Sparse MLP, BD	0.0473*	0.136*	0.159*	3.96×10^{-4}	0.113*	**0.436***
Sparse MLP (Fathi et al., 2024)	0.0397*	**0.0940***	**0.107***	**2.32×10^{-4}***	0.120*	0.497
Dense MLP (Lusch et al., 2018) *[baseline]*	0.830	2.68	2.93	0.00110	0.224	0.754
DMD	2.58	2.14	1.70	2.85	4.19	3.83
Polynomial EDMD	0.864	0.968	0.887	2.32	3.93	3.61
RBF-dictionary EDMD	0.660	0.808	0.751	0.890	2.84	2.91
k-means local linear	1.40	3.04	138	2.29	1.5×10^{150}	3.2×10^{140}
GMM local linear	1.65	3.57	3.93	10.6	7.8×10^{18}	4.5×10^{44}
Soft-gated GMM local linear	1.66	3.71	4.07	5.77	3.0×10^{13}	4.1×10^{34}



(a) Dropping the largest active coordinates.

(b) Moving active values to inactive coordinates.

Figure 2: Support-coordinate interventions on a multibasin system; interventions severely increase forecasting errors compared to the standard rollout. **(a)** The standard rollout is compared with cumulative dropping of the largest active coordinates of the initial sparse code; shaded bands are bootstrap 95% intervals over 100 initial conditions. **(b)** Active coefficient values are preserved but reassigned to randomly chosen inactive coordinates; the band is the interquartile range over shuffles.

controls are worse than the dense-latent MLP at the displayed horizons, and the local-linear mixture baselines often become numerically unstable at long horizons. These results suggest that having some bias toward code sparsity is crucial for good performance over all horizon prediction lengths, and that the improvement is not explained by fitting a standard state-space Koopman or clustering-local-linear model. The scale-normalized Dysts summary in table 30 gives the same average-case view: the primary sparse rows remain below the dense-latent baseline throughout, while no single transition structure dominates every chaotic-flow horizon.

Sparse supports are essential for good representations. If a support formed by sparse encoders is meant to carry basin-relevant dynamical information, then changing only the initial active set should damage forecasts, even when the coefficient values are otherwise kept fixed. We do an ablation study to show whether accurate forecasts within a basin of attraction require the correct coordinates to be active by isolating the selection of sparse coordinates from their values. We do this by forcing a sparse KAE to use an incorrect active set with specific interventions, and evaluating 20-step forecasting performance under these settings. The interventions are the following:

1. **Coordinate dropping:** Force the coordinates of z with the $k \in \{1, \ldots, 10\}$ largest magnitude coefficients to zero before forecasting.
2. **Random support:** the active coefficient values are preserved, but reassigned to randomly chosen inactive latent coordinates.

The results show that these interventions have a significant detrimental effect on forecast quality, even on very short horizons. At $H = 21$, the standard rollout has mean accumulated MSE 0.0158, compared to 0.508/1.37/2.08/3.26/8.51 for the rollouts after dropping the top 1/2/3/5/10 active coordinates, and random support shuffling is much more destructive as seen in figure 2 and section G. The trajectory panel (figure 3) shows the qualitative effects of the intervention. So, supports formed by sparse encoders do not merely correlate with basin labels; the active coordinates are key to the quality of the Koopman representation.

Table 2: Basin-support diagnostics on the controlled multibasin benchmark. Values are computed on the evaluation-only per-basin deep slice. Superscripts mark per-basin deep-slice Wilcoxon/Holm tests against the dense-latent MLP baseline ($*$, $p < 0.05$). **Bold** marks the best entry in each directional column.

	Per-basin deep slice	
Model	$H(B \mid F_{\text{abs}}) \downarrow$	$\overline{\lvert F_{\text{abs}} \rvert}$
LISTA	**0.130***	**4.0**
LISTA-BD	0.156*	3.8
LISTA-SB	0.153*	3.8
Sparse MLP, BD	0.358*	3.0
Sparse MLP	0.223*	3.3
Dense MLP	1.28 *[baseline]*	1.0 *[baseline]*

4.2 Support-basin alignment experiments

This mechanistic experiment examines the extent to which sparse support families fit after training can behave like basin-interior regime variables. States near separatrices are challenging and may confound the inferences: near these states, support changes can be induced by small perturbations in x, making supports unreliable for basin identification at separatrices. We therefore evaluate this diagnostic on held-out states deep in basins far from basin boundaries.

For a state x, let $d_1(x)$ and $d_2(x)$ be its distances to the nearest and second-nearest benchmark attractor centers. The *basin-depth margin* is $d_2(x) - d_1(x)$. This support-basin alignment experiment takes the top quartile of held-out states ranked by basin-depth margin within each benchmark basin, using ground-truth benchmark geometry to identify each basin. If a support family F_{abs} identifies a basin-specific regime used by the model, then on states deep inside a basin, it should leave little uncertainty about the withheld benchmark basin label B, measured by conditional entropy $H(B \mid F_{\text{abs}})$ as the main evaluation metric. $H(B \mid F_{\text{abs}})$ should be small when support families are basin-aligned. Also, different basins should be adequately represented by distinct support families, so we count the number of distinct support families formed in a system, denoted as $|F_{\text{abs}}|$, and compare it with the number of basins represented.

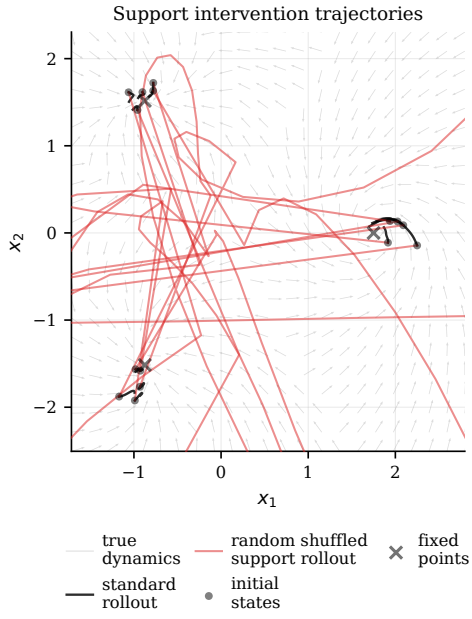


Figure 3: Randomly shuffled support trajectories (red) compared to standard rollouts (black). Randomly shuffling the active coordinates leads to divergences.

Support families F_{abs} identify basin membership. As seen in table 2 and figure 4, LISTA F_{abs} support families provide more information to identify a ground truth basin label than any other model architecture, as seen by the lowest conditional entropy of 0.130. Other LISTA models with structured latent transitions are close competitors, followed by the sparse-latent MLPs, and then finally the dense-latent MLP baseline. LISTA-based models also expose the most support families, which better matches the benchmark's basin-count mean/median 4.20/4; ideally, if a basin can truly be recovered by a single support family cluster, then the basin count should be one-to-one with the number of support families, $\overline{|F_{\text{abs}}|}$. In stark contrast, the dense-latent MLP collapses the representation to one support family, so distinct basins have correlated learned latent dynamics. This suggests that the support families induced by sparse coding may serve as a good proxy for ground-truth basin labels; the sparse LISTA models may have the emergent ability to discover basins unsupervised, despite this not being explicit in the training objectives.

Appendix section K studies a more elaborate support-flow object, C_{stab}, that groups support families by recurrent fate. The matched route controls below show that this extra machinery is not needed for the main forecasting claim: the simpler F_{abs} route already selects strong local affine predictors.

4.3 Support-family local linearizations

The preceding diagnostics show that F_{abs} support families are learned without basin labels and nevertheless identify basin-interior states. We next ask whether this support object is only descriptive, or whether it can be used operationally to specialize prediction. The local-linearization experiment uses the same sparse LISTA architecture and the same total training budget as the matched global-K baseline. The first half of training learns the standard Koopman autoencoder with one shared latent matrix K. We then freeze E_θ, D_ϕ, and the learned global K, build F_{abs} support families from training-split trajectories, and train one affine latent predictor for each retained family.

For support family f, let $\bar{z}_f$ be the mean source latent assigned to that family. The route-local update is

$$T_f(z) = d_f + K_f(z - \bar{z}_f), \tag{13}$$

where K_f is initialized from the frozen global matrix and d_f is initialized to $K\bar{z}_f$ and then learned. At rollout time, the model periodically decodes and re-encodes its own predicted state, recomputes m_{abs}, assigns the current support to an F_{abs} family by the frozen Jaccard representatives, and applies the corresponding T_f. If the support cannot be assigned to a trained family, the step falls back to the frozen global K. This procedure uses no basin labels, basin centers, attractor identities, or trajectory-to-basin assignments. The only route variable is the support family produced by the globally trained sparse KAE itself.

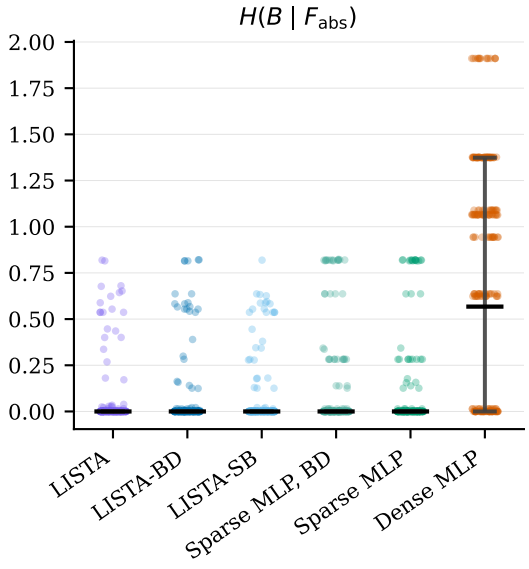


Figure 4: Distribution of conditional entropy results. Dense MLP cannot identify basins based on support families. Dots show system-seed pairs; grey bars mark first to third quartile ranges, and black bars mark IQM is the mean of $H(B \mid F_{\text{abs}})$.

This is the operational version of the paper's support hypothesis. The learned support family is not merely correlated with basin labels after training; it can condition a bank of distinct local latent linearizations that improve downstream forecasting while preserving the same encoder and decoder. In the retained 15-system controlled multibasin benchmark, with 15 seeds per system

Table 3: F_{abs}-routed staged local affine forecasting on the controlled multibasin benchmark. The baseline is the matched same-budget global-K LISTA model. Both models use the same LISTA p256 architecture and are evaluated using the same periodic re-encoding grid; lower MSEs and ratios are better.

Metric	H100	H500	H1000	All horizons
Paired local wins	199/225	192/225	189/225	180/225
Mean MSE, F_{abs}-local/global	0.0128/0.0229	0.0347/0.0623	0.0403/0.0706	–
Median local/global MSE ratio	0.355	0.312	0.328	–
Geometric local/global MSE ratio	0.252	0.180	0.174	–

and the same periodic re-encoding grid for both models, F_{abs}-routed local maps beat the matched global-K LISTA baseline on 199/225, 192/225, and 189/225 system–seed pairs at $H100$, $H500$, and $H1000$, respectively. The median local/global MSE ratios are 0.355, 0.312, and 0.328. This supports the central claim that sparse support families can act as label-free regime variables for robust basin-local prediction.

5 Discussion

The experiments support a single interpretation from four complementary angles. Sparse-latent Koopman autoencoders forecast better than dense-latent controls on controlled multibasin systems and retain an advantage on the Dysts chaotic-flow stress test. The coordinate-intervention experiment then shows that this improvement is not merely a consequence of shrinking latent amplitudes: changing which coordinates are active, while preserving or only partially removing coefficient values, sharply degrades rollouts. The basin-alignment diagnostic shows that the support families exposed by LISTA encoders carry basin-interior information that was never supplied during training. Finally, the support-family local-linearization experiment shows that those same learned F_{abs} families can route affine latent predictors and improve controlled multibasin forecasting over the matched global-K LISTA baseline. Taken together, these results suggest that sparsity can turn a Koopman latent from a purely continuous coordinate system into a hybrid representation: coefficient values describe state within a local regime, while active-coordinate identities provide an inspectable, model-produced regime variable.

This places the work near two broader uses of sparsity, but with a different object of interpretation. In language-model mechanistic interpretability, sparse dictionary learning and sparse autoencoders are used to decompose superposed activations into sparse features, with recent work emphasizing the need to evaluate reconstruction, sparsity, interpretability, and causal control separately (Elhage et al., 2022; Cunningham et al., 2023; Gao et al., 2024; Makelov et al., 2024; Karvonen et al., 2025). Continual-learning methods likewise use sparse subnetworks, masks, or non-overlapping activations to reduce interference between tasks (Mallya and Lazebnik, 2018; Serrà et al., 2018; Wortsman et al., 2020; Abbasi et al., 2022; Lin et al., 2025). Our setting differs from both: the model is not a post-hoc probe of a frozen language model, and it is not trained sequentially with task identities or task boundaries. The shared principle is that sparse support identities can separate otherwise interfering computations; for SKAEs, those computations are local Koopman-compatible dynamics, and the support is evaluated as a regime variable rather than as a semantic feature or a task mask.

This interpretation also clarifies what is and is not being claimed about Koopman learning in multibasin systems. We do not claim that a single sparse model discovers a globally smooth finite-dimensional linearization of the full state space; the theory motivating this work suggests

that such a representation should often fail. Instead, the sparse support acts as a data-driven way to separate locally compatible pieces of the dynamics while preserving a shared encoder and decoder. The strongest version of this idea is the staged local-linearization result: once F_{abs} support families are available, the latent dynamics can be specialized by local affine maps rather than forced through a single matrix. The result is a practical compromise between dense global KAEs, which can collapse incompatible basins into a mixed latent, and supervised mixture or switching models, which typically require the regime labels, basin count, or routing rule that we assume are unavailable at training and deployment time.

The main limitations are the same places where the evidence is most deliberately controlled. The strongest support-identification and local-linearization results are measured on two-dimensional procedural systems with known attractor geometry, and the cleanest entropy diagnostic is restricted to held-out states deep inside basins. This is the right slice for testing whether supports can become basin-scale objects, but it does not settle behavior near separatrices, during rare transitions, under partial observation, or in high-dimensional physical systems where the relevant regime geometry may not be known. The Dysts experiments broaden the forecasting evidence beyond the procedural generator, but they are not basin-identification tests because those systems do not provide the same evaluated basin structure; moreover, the current staged local affine recipe is negative on the retained Dysts stress test. The support-family construction also introduces fixed choices, such as the activation threshold and Jaccard merge threshold; our appendix sensitivities show why family count must be interpreted together with conditional entropy rather than optimized in isolation.

These limitations also define two concrete benchmark extensions for this line of work. First, a spatialized reaction–diffusion version of the procedural multibasin suite can lift each two-dimensional vector field f_s to 64×64 or 128×128 two-channel fields with diffusion coupling, producing a genuinely high-dimensional autoencoding problem while retaining evaluation-only attractor metadata. Second, contact-rich robotic insertion in ManiSkill3 (Tao et al., 2024) can test whether sparse supports become contact and outcome regime variables under control. The two benchmarks are deliberately asymmetric: the spatialized benchmark is the low-risk scaling test, while robotic insertion is the higher-risk application-style test. Both keep the core protocol unchanged: basin or outcome labels, counts, and contact phases are withheld during training and are used only after training for support-family evaluation. We spell out the proposed protocols in section E, using PDEBench (Takamoto et al., 2022) and The Well (Ohana et al., 2024) as high-dimensional PDE benchmark references.

These qualifications point to a concrete research program rather than a retreat from the claim. Future work should learn support-transition graphs, quantify uncertainty near basin boundaries, and study objectives that encourage support persistence within regimes while allowing support changes when the state crosses into a new dynamical region. A second direction is to use support families as explicit interfaces for downstream world models and control: the same learned support key could index cached local statistics, select candidate linear controllers, or provide an interpretable state abstraction for planning. The broader contribution of this work is therefore not a new benchmark-specific clustering heuristic, but evidence that sparse Koopman representations can expose regime structure from trajectories alone. For multistable scientific systems, robotics settings with contact or mode changes, and learned world models with recurring local dynamics, that inspectable support structure is precisely the kind of representation that can make a learned predictor more useful than a black-box latent rollout.

A Additional experimental details

This appendix records the paper-facing experimental protocol. The main experiments ask whether sparse Koopman autoencoders learn useful latent supports without basin supervision. The model is never given basin labels, basin counts, or trajectory-to-basin assignments when training the encoder, decoder, latent transition, support rules, or periodic rollouts. Benchmark basin metadata is used only after training to define evaluation slices, construct controlled diagnostic interventions, and score support–basin agreement. The only training-time exception is architectural: the block-diagonal and soft-block transition diagnostics use benchmark metadata to set the fixed number of transition groups, but they still do not receive trajectory labels or support labels.

Benchmarks. The controlled benchmark contains 15 two-dimensional multibasin systems, listed with their equations and attractor metadata in section C. These systems are integrated to produce stored observation sequences; the learner observes only stored states, not vector fields, integration substeps, basin labels, or basin assignments. The external forecasting stress test uses the 10 three-dimensional Dysts $dt{\times}30$ systems listed in section D. For Dysts, stored observations are generated at 30 times each system's native integration interval and coordinates are standardized after trajectory generation.

Model rows. The six model rows in tables 1 and 2 are LISTA, LISTA–BD, LISTA–SB, Sparse MLP, Sparse MLP–BD, and Dense MLP. All use latent dimension $d_z = 256$. LISTA rows use shrinkage to produce exact zeros; MLP sparse rows use the sparsity penalty in equation (10); Dense MLP removes the intended sparsity mechanism and sets the sparsity coefficient to zero. The BD rows use a block-diagonal latent transition. The SB row keeps a dense transition but adds an off-block penalty. All rows use a linear decoder dictionary as the state decoder. The controlled multibasin LISTA rows use threshold scale $\alpha = 0.15$, two shrinkage-refinement loops, and a sign-split final operation. The Dysts $dt{\times}30$ LISTA rows use the same threshold scale with one refinement loop and a ReLU final operation. MLP encoders use two hidden layers of width 64; Dense MLP uses a tanh encoder without the sparse output gate.

Controlled multibasin training. For every controlled system, each model row is trained for seeds $0, \ldots, 14$. Training uses rollout length $L = 8$ windows, meaning 9 adjacent stored states per window, for 200,000 optimization steps with minibatches of 256 windows. The optimizer is AdamW. Encoder and decoder parameters use learning rate $5{\times}10^{-5}$, the latent transition uses learning rate $5{\times}10^{-6}$, and non-transition parameters use weight decay 10^{-4}. In equation (11), the loss weights are $\lambda_{\text{pred}} = 1$, $\lambda_{\text{rec}} = 0.03$, $\lambda_{\text{lin}} = 1$, and $\lambda_{\text{sp}} = 3{\times}10^{-3}$ for sparse rows; Dense MLP uses $\lambda_{\text{sp}} = 0$. Soft-block rows add the off-block L_1 transition penalty with weight 10^{-4}.

All controlled rows use the same label-free boundary-emphasized reset distribution. Before training, 4096 candidate initial states are probed for 32 steps. Candidates are scored by short-horizon perturbation sensitivity and residual late-rollout motion, and the top 1024 states are retained. The scoring probe uses four perturbations, perturbation scale 0.04, a late-transient window of 8 steps, and transient weight 0.5. Half of training windows start from this retained pool with jitter scale 0.25. The retained pool is intended to expose boundary-adjacent and transient regions that would otherwise be rare under ordinary resets; it is built without basin labels.

Dysts $dt{\times}30$ training. Dysts rows are trained for the same seeds, $0, \ldots, 14$, with latent dimension $d_z = 256$, minibatches of 256, rollout length $L = 10$ windows, and 100,000 optimization steps.

Dysts trajectories come from deterministic native caches with 200 cached trajectories, 30,000 stored observations per trajectory, and a 2,000-step warm-up. The first cached trajectory starts from the Dysts default initial condition; remaining trajectories perturb that state with Gaussian noise at scale 0.2 times the coordinate-wise Dysts standard deviation. Train, validation, and test caches use separate deterministic namespaces, so held-out test windows are disjoint from training windows and reusable across model seeds. LISTA rows use the controlled-benchmark learning rates. MLP rows use learning rates 10^{-4} for encoder/decoder parameters and 10^{-5} for the latent transition. Sparse Dysts rows use $\lambda_{\text{sp}} = 6\times10^{-3}$, while Dense MLP uses $\lambda_{\text{sp}} = 0$.

Checkpoint selection. Checkpoint selection is fixed before test evaluation. Every 500 optimization steps, and again at the final step, the current model is rolled out for 200 stored steps from 16 fixed held-out initial states using every-step re-encoding. The checkpoint with the lowest final validation error under this proxy is used for reported evaluations. Final tables aggregate completed seeds; they do not select the best seed.

Training recipe selection. Exploratory sweeps varied LISTA depth, shrinkage scale, sparsity coefficient, sign-split encodings, transition structure, and MLP sparsity controls. These sweeps used shorter budgets or broader system lists and are not pooled with the final evidence. The paper-facing protocol is the fixed recipe in this appendix: 15 seeds per retained system, the training budgets above, held-out evaluation splits, and the statistical units in section I.

Forecasting experiments. Forecasting is evaluated on held-out trajectories at the stored observation cadence. A horizon h always means h stored observation steps, not a common physical time across systems. The no-reencoding rollout applies the learned latent transition for the full horizon before decoding. The periodic rollout advances the model's own predicted latent state, decodes the predicted state at a fixed period m, and re-encodes that predicted state; it never uses the true future state. Controlled multibasin forecasting uses 100 held-out initial conditions for each system and seed and reports the best score over the fixed period grid $m \in \{10, 25, 50, 100\}$. Dysts long-horizon forecasting uses 100 held-out test-cache windows for each system and seed and reports the best score over $m \in \{10, 25, 50, 100, 150, 200\}$. These grids are fixed before aggregation and are not tuned separately for each test trajectory.

Support–basin alignment. Support alignment is computed after training from held-out controlled-system states. For each state x, the basin-depth margin is the distance to the second-nearest benchmark attractor center minus the distance to the nearest attractor center. Within each represented benchmark basin, the evaluator keeps the top quartile of held-out states by this margin. This per-basin deep slice uses ground-truth geometry only to define the evaluation population. It is the clean slice for testing basin-support alignment because boundary states can legitimately have unstable or mixed supports.

On this slice, the evaluator encodes states and constructs $S_{\text{abs}}(x) = \{i : |z_i| > 10^{-3}\}$. Exact Boolean masks are grouped into F_{abs} support families by the deterministic greedy Jaccard rule in sections 3.4 and F with threshold 0.5. The main directional metric is $H(B \mid F_{\text{abs}})$, where B is the withheld benchmark basin label; lower values mean that knowing the learned support family leaves less uncertainty about basin identity. The companion count $\overline{|F_{\text{abs}}|}$ is descriptive and is interpreted relative to the number of represented basins, not as a monotone score to maximize.

Wrong-support functional diagnostic. The cross-system wrong-support diagnostic asks whether the selected active coordinates matter for prediction. For each testable controlled system and basin, the evaluator constructs a canonical deep-slice S_{abs} mask from the most frequent exact mask in that basin. During a held-out rollout, the inferred mask is replaced with a canonical mask from a different basin and held fixed during the masked latent rollout. Table 2 reports wrong-support/base error ratios at $H = 1$ and $H = 20$; larger ratios mean that using a basin-inconsistent support is more damaging.

Support-coordinate intervention figure. The representative coordinate-intervention experiment isolates active-coordinate identity from coefficient values. Starting from a held-out basin-interior state, the model encodes $z_0 = E_\theta(x_0)$, perturbs only z_0, and then runs the ordinary learned transition and decoder without retraining. In the coordinate-dropping condition, the active entries are ranked by $|z_{0,i}|$ and the top $k \in \{1, 2, 3, 5, 10\}$ are set to zero before rollout. In the random-support condition, active coefficient values are preserved but reassigned to randomly chosen inactive coordinates. The figure and table use 100 held-out basin-interior starts; the random-support condition uses 20 random inactive-coordinate reassignments for each start. Errors are accumulated through $H = 21$, with the full horizon curves shown in figure 2 and the $H = 21$ summary in table 13.

Periodic support-refresh diagnostic. The support-refresh experiment asks whether periodic re-encoding can re-select the support family appropriate to a new region of state space after a controlled transfer. The support object is F_{top8}, built by taking the eight largest-magnitude latent coordinates and merging exact top-8 masks with the same Jaccard-family rule. Each valid comparison starts from a controlled source-to-target transfer generated from benchmark annotations. Between scheduled refresh events, the latent state is advanced only with the learned global transition K. At each event, the stale family implied by the propagated latent is compared with the family obtained by encoding the current controlled-transfer state. The reported cells in table 14 are target-family fractions before re-encoding $\rightarrow$ after re-encoding for fixed periods $m \in \{1, 5, 10, 20\}$, plus a no-transfer false-target control. Basin labels are used to construct and score transfer instances, not to train a support-conditioned rollout.

Post-hoc routing diagnostic. The appendix support-conditioned predictor experiment is a second-stage diagnostic, not part of the main training procedure. After a Koopman autoencoder checkpoint is fixed, the encoder, decoder, and original global K are frozen. Routes are built from S_{top8} masks and merged into F_{top8} support families with Jaccard threshold 0.40. Local transition maps are fit by support family on a disjoint fitting split of 256 held-out trajectories of length 256, giving $256 \times 256 = 65{,}536$ adjacent latent transitions per checkpoint and support definition. A family with fewer than 50 current-state fitting transitions does not receive a trainable local map. For each retained family c, the centered affine map $T_c(z) = \bar{z}_c + K_c(z - \bar{z}_c)$ is initialized from the checkpoint's global K and optimized with decoded rollout MSE while the original autoencoder remains frozen. Route-balanced minibatches prevent the most frequent support families from monopolizing the second-stage updates. Evaluation uses a separate set of 128 held-out rollouts; at $H = 1000$, this contributes 128,000 one-step target positions along evaluated rollouts. Routes are recomputed from the current predicted latent state during rollout, and the routed rollout periodically decodes and re-encodes its own predicted state with period 5. If no fitted local route is available, the evaluator falls back to the same model's global transition. The reported routed/global ratio therefore compares a support-routed rollout with the same frozen checkpoint's own global rollout.

Aggregation and significance. Point estimates in forecasting and support tables summarize seeds within each system by interquartile mean and then average system summaries across systems. Multibasin forecasting, support-alignment, and wrong-support significance annotations use paired seed-level effects within each controlled system against Dense MLP, with one-sided alternatives fixed by the table direction and Holm correction across eligible systems. Dysts forecasting uses each Dysts system as the independent unit after within-system seed-IQM summarization and applies a paired one-sided exact sign test with Holm correction. Support-refresh and coordinate-intervention panels are descriptive mechanism diagnostics. Full statistical definitions and denominator conventions are in section I.

Hardware and resource details. Training was done on a SLURM cluster; RTX8000 was the main GPU being used. Training jobs load CUDA 12.6.0 and request one cluster GPU, 4 CPU cores, and 16 GB host memory per active training allocation. Controlled multibasin training uses single-run GPU allocations with a 24-hour time limit. Dysts $dt\times30$ training uses packed GPU allocations with up to 12 runs per allocation, one GPU, 4 CPU cores, 16 GB memory, and a 72-hour time limit per packed allocation. Dysts cache construction uses CPU allocations with 4 CPU cores, 16 GB memory, and a 24-hour time limit. Dysts long-horizon evaluation uses CPU allocations with 4 CPU cores, 24 GB memory, and a 6-hour time limit for the paper-facing $dt\times30$ evaluation queue. Controlled support diagnostics and support-routed analyses use CPU allocations with 4 CPU cores, 16–24 GB memory, and 8–12-hour time limits, depending on the diagnostic.

B Training-dynamics diagnostic

The main tables report held-out aggregate performance from validation-selected checkpoints. As a diagnostic for the optimization trajectory, figure 5 shows the metric histories for the six controlled multibasin model rows on one representative benchmark system, `gated_local_linear`, with seed 0. The curves use the same training protocol and validation rule described in section A: validation rollouts are evaluated every 500 optimization steps, and the saved checkpoint is the step with the lowest validation final error. The diagnostic is not used for model ranking; it is intended to show how the reported checkpoints arise along training.

The validation traces generally improve through training for the sparse rows, while the Dense MLP remains separated by a substantially larger validation error. The best validation checkpoints for the sparse rows occur late in training, between 179,500 and 193,500 steps in this trace, whereas the Dense MLP reaches its best validation value earlier. The spectral-radius panel shows that the learned transition remains close to the unit circle throughout training. This supports the qualitative reading that long-horizon behavior is tied to both validation error and the stability of the learned Koopman transition, while avoiding the stronger claim that $\rho(K)$ closest to one is always optimal.

For the external chaotic-flow benchmark, figure 6 gives the same diagnostic in aggregate form. Each curve uses seed 0 and summarizes the retained 10 Dysts $dt\times30$ systems by the median over finite per-system metric values at each logged step. We use a system median here rather than a single representative Dysts system because the individual chaotic flows have different scales and occasional very large finite validation rollout errors early in training. The resulting validation traces are therefore visibly noisier than the controlled-system trace, but they still show the checkpoint-selection path and the late-training behavior of the model families.

The Dysts diagnostic should be read more cautiously than the controlled benchmark diagnostic. The validation proxy can spike when a chaotic rollout leaves the attractor or compounds error early in training, and Dysts does not provide the controlled basin-label structure used by the main

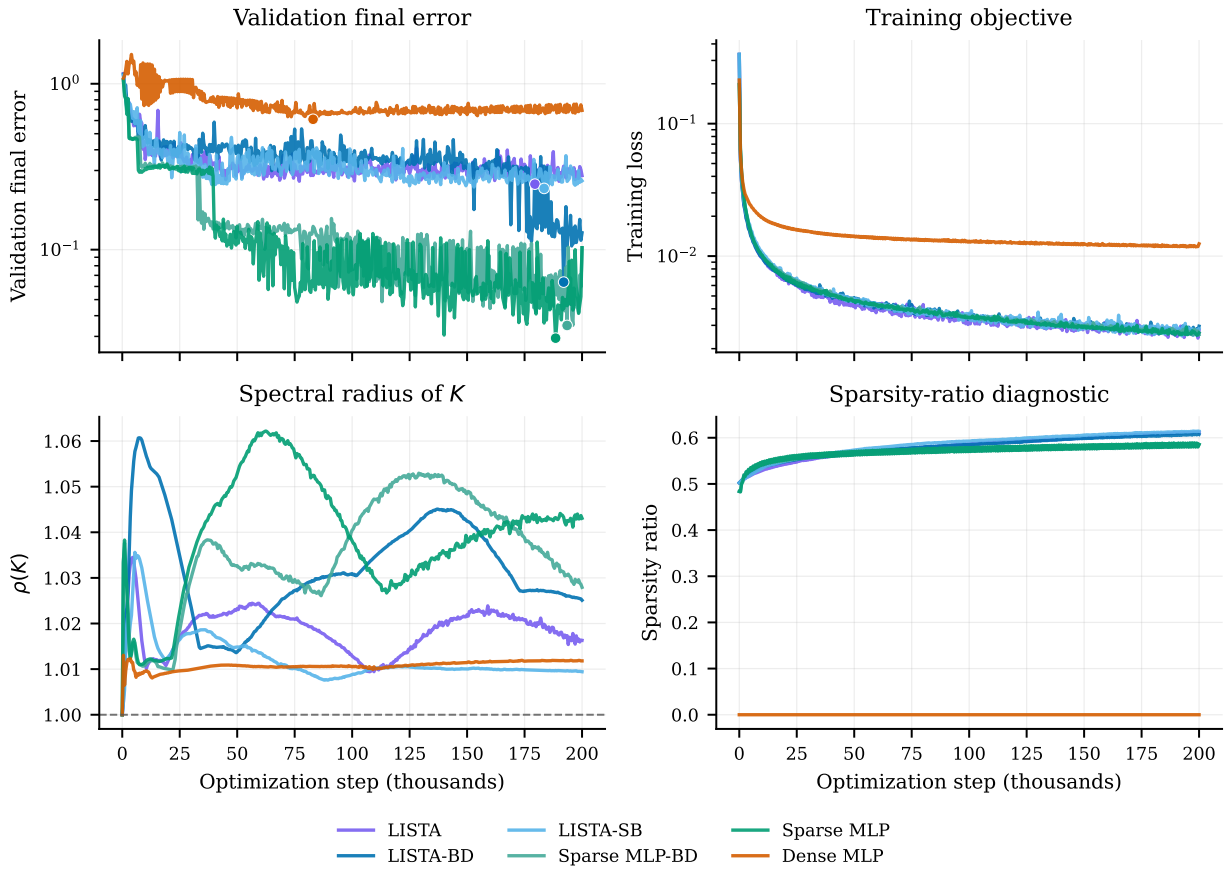


Figure 5: Representative training-dynamics diagnostic on `gated_local_linear`, seed 0, for the six controlled multibasin model rows. Validation final error is the held-out rollout error used by the checkpoint-selection rule; dots mark the minimum validation final error for each trace. The training objective, validation error, transition spectral radius $\rho(K)$, and sparsity-ratio diagnostic are plotted against optimization step. The horizontal dashed line marks $\rho(K) = 1$. These curves are a diagnostic of training progression and checkpoint selection, not an aggregate benchmark comparison.

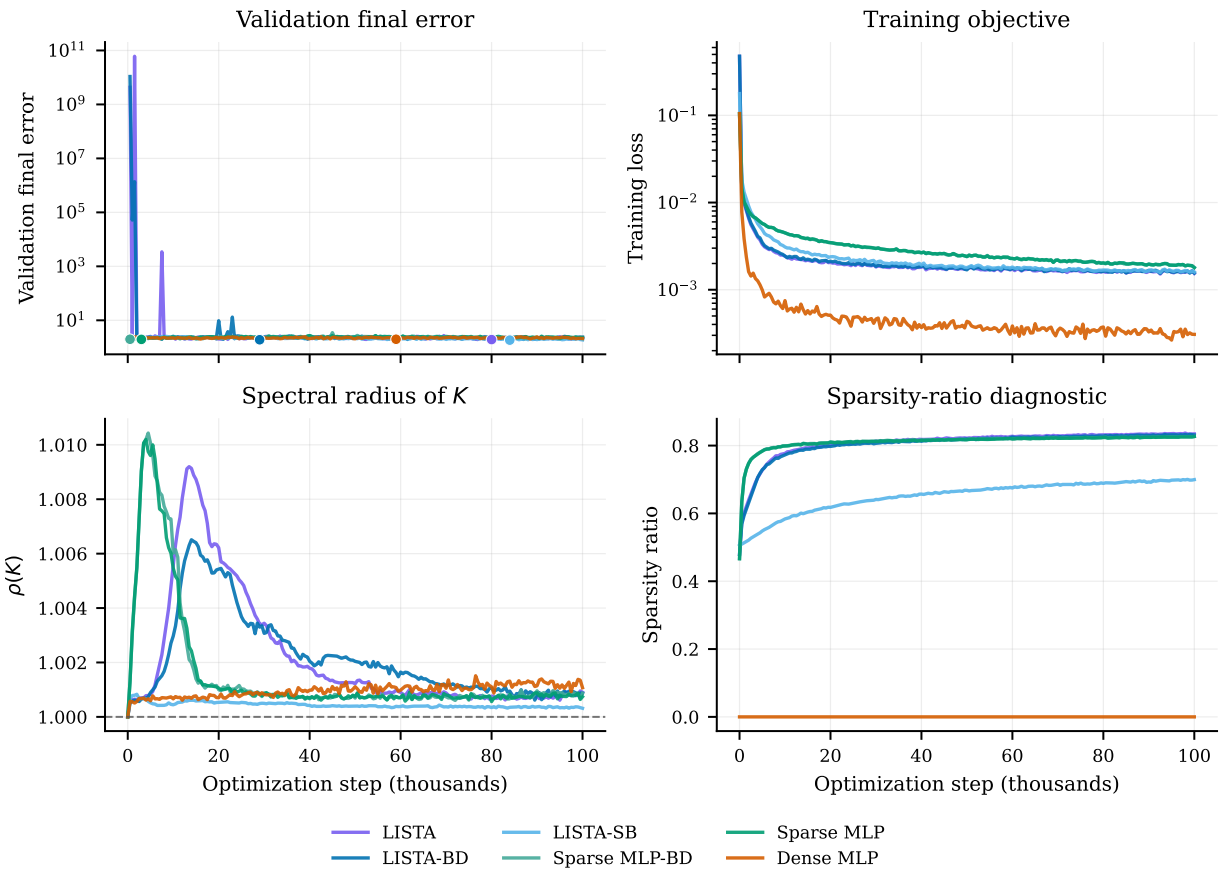


Figure 6: Dysts $dt\times30$ training-dynamics diagnostic. Curves summarize seed 0 over the retained 10 Dysts systems by the median over finite per-system metric values at each logged step. Validation final error is the same held-out rollout proxy used by checkpoint selection; dots mark the minimum median validation final error for each trace. The Dysts validation panel is noisier than the controlled trace because chaotic rollout errors can become very large early in training, so this figure is an appendix diagnostic of optimization and selection behavior rather than a main benchmark ranking.

support-alignment claims. Still, the median traces are useful for checking that the retained $dt\times30$ runs are not selected from isolated early accidents: validation curves generally settle after the initial transient, and the learned transition spectra remain close to the unit circle by the end of training, with final median $\rho(K)$ values roughly between 1.0003 and 1.0011 across rows. As above, this supports the qualitative stability diagnostic without making $\rho(K)$ closest to one a sufficient criterion for forecasting quality.

Table 4: The 15 multibasin benchmark systems.

Display name	System key	Generator family	Basins
Local-linear gates	`gated_local_linear`	piecewise local-linear gates	3
Transfer-gated local-linear	`gated_transfer_linear`	piecewise transfer gates	3
Arrested spiral	`arrested_spiral`	spiral capture with Gaussian traps	5
Asymmetric three-well	`cal_asymmetric_3`	Gaussian wells with independent rotation	3
High-cross three-well	`cal_high_cross_3`	Gaussian wells with stronger rotation	3
Hexagonal six-well	`cal_hexagon_6`	Gaussian wells with independent rotation	6
Octagonal eight-well	`cal_octagon_8`	Gaussian wells with independent rotation	8
Pentagonal five-well	`cal_pentagon_5`	Gaussian wells with independent rotation	5
Square four-well	`cal_square_4`	Gaussian wells with independent rotation	4
Triple-well Duffing	`duffing_triple_well`	triple-well Duffing oscillator	3
SNIC multi-attractor	`snic_multi`	polar SNIC-like phase dynamics	3
Transition-routes four-well	`transition_routes_4`	route-augmented Gaussian wells	4
Depth-gradient four-well	`var_depth_gradient_4`	Gaussian wells with depth gradient	4
Diamond four-well	`var_diamond_4`	Gaussian wells on a diamond layout	4
L-shaped five-well	`var_l_shape_5`	Gaussian wells on an L-shaped layout	5

C Multibasin benchmark inventory and system definitions

Table 4 lists the 15 two-dimensional multibasin systems used in this paper. Every row contributes to the retained-system aggregate in table 1, table 2, and figure 1a. The support-refresh subtable uses 12 retained systems for which the controlled-transfer construction produced eligible post-entry comparisons.

Basin labels are used only for benchmark annotation, controlled-transfer construction, and evaluation. Basin counts are benchmark metadata; for the structured-transition ablation rows, they are also used only to set a fixed architecture group count, as described in section A, and not for support extraction, support-family construction, routing, or rollout evaluation.

The 15 multibasin benchmark systems were generated procedurally with Claude Opus 4.5.

Stored-step convention. All retained systems are implemented as continuous-time vector fields $\dot{x} = f_s(x)$ and integrated with fourth-order Runge–Kutta to produce the stored observation map $F_{\Delta t}$ used in equation (1). The training windows contain only stored states x_k, not f_s, integration substeps, or basin assignments.

Ground-truth vector-field visualizations. Figure 7 shows the continuous-time ground-truth vector fields for all 15 retained two-dimensional multibasin systems. Streamlines show the direction of $f_s(x)$, and black crosses mark the generator attractor centers or analytic equilibrium references used for benchmark annotation. The streamline colors use a panel-local $\log_{10} \|f_s(x)\|_2$ scale to reveal structure within each system; they should not be read as comparable speed scales across panels. Detailed panels with per-system colorbars are provided in figures 8 to 12.

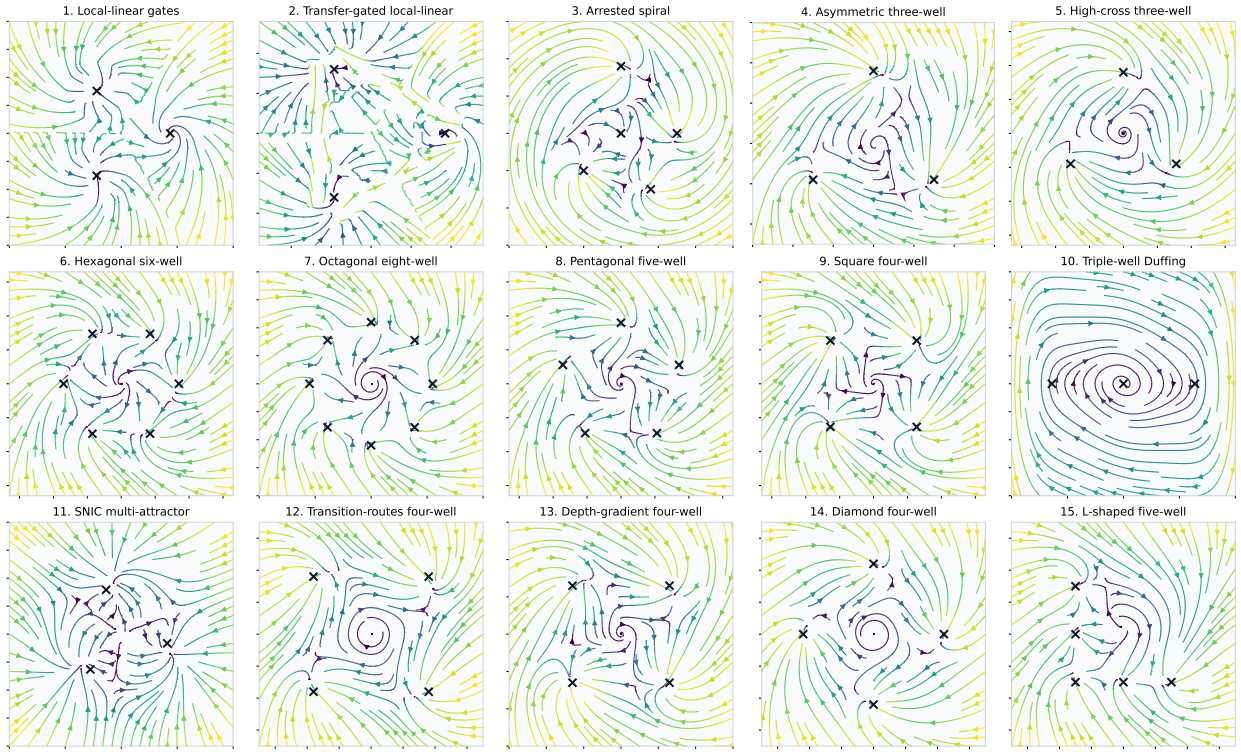
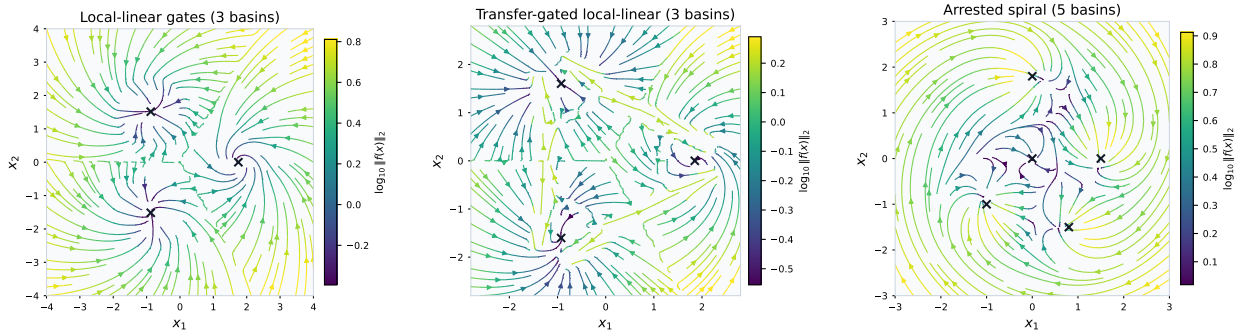
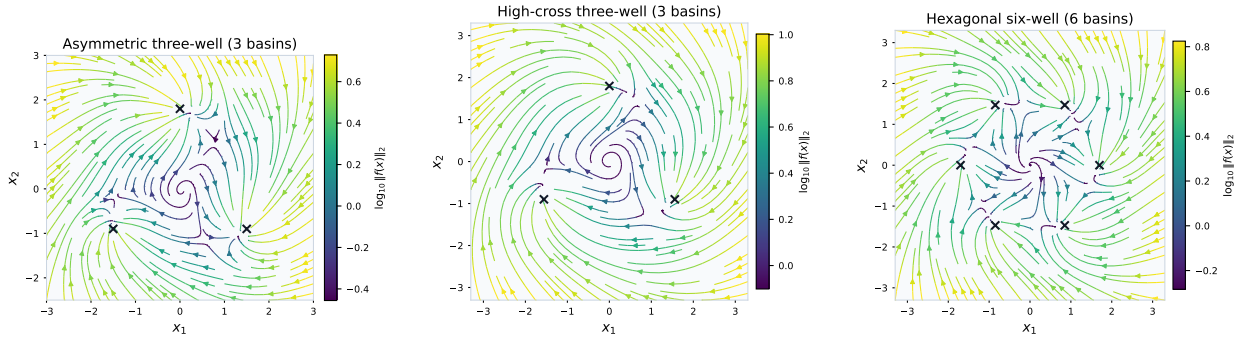


Figure 7: Ground-truth vector fields for the retained 15-system multibasin benchmark. The models are trained only on stored state trajectories; these continuous-time vector fields and attractor annotations are shown here only to document the benchmark geometry and are not provided to the learner.



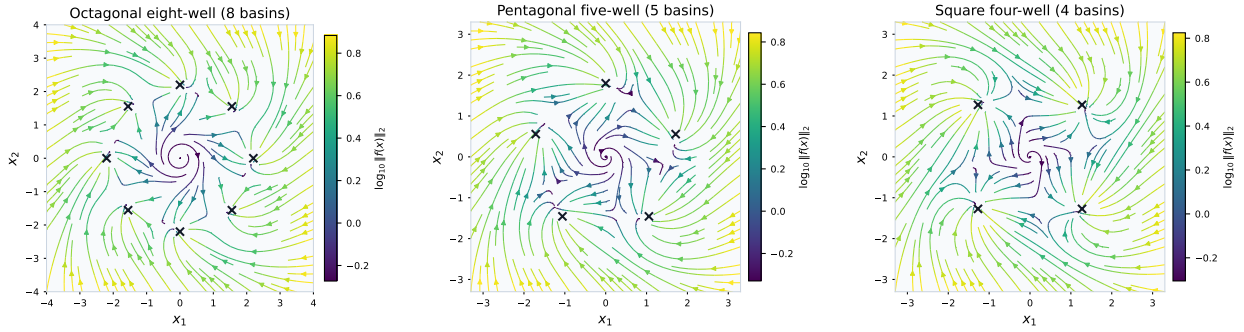
(a) Local-linear gates. (b) Transfer-gated local-linear. (c) Arrested spiral.

Figure 8: Detailed ground-truth vector-field panels for the first three retained multibasin systems. Colorbars report the panel-local $\log_{10} \|f_s(x)\|_2$ scale.



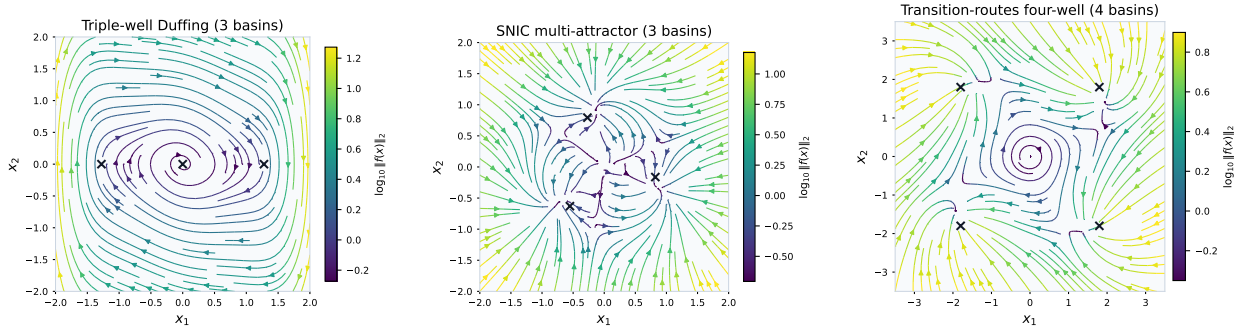
(a) Asymmetric three-well. (b) High-cross three-well. (c) Hexagonal six-well.

Figure 9: Detailed ground-truth vector-field panels for retained Gaussian-well systems. Colorbars report the panel-local $\log_{10}\|f_s(x)\|_2$ scale.



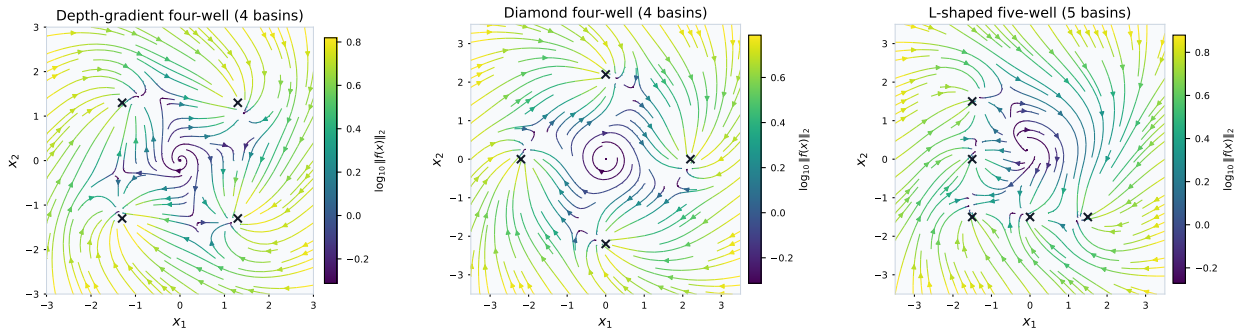
(a) Octagonal eight-well. (b) Pentagonal five-well. (c) Square four-well.

Figure 10: Detailed ground-truth vector-field panels for retained polygonal Gaussian-well systems. Colorbars report the panel-local $\log_{10}\|f_s(x)\|_2$ scale.



(a) Triple-well Duffing. (b) SNIC multi-attractor. (c) Transition-routes four-well.

Figure 11: Detailed ground-truth vector-field panels for specialized retained systems. Colorbars report the panel-local $\log_{10}\|f_s(x)\|_2$ scale.



(a) Depth-gradient four-well. (b) Diamond four-well. (c) L-shaped five-well.

Figure 12: Detailed ground-truth vector-field panels for retained geometry variants. Colorbars report the panel-local $\log_{10} \|f_s(x)\|_2$ scale.

Table 5: Parameters for retained Gaussian-well systems. Repeated pairs in the (a_i, σ_i) column apply to every listed center.

System	$\{c_i\}$	(a_i, σ_i)	(ω, γ)
High-cross three-well	$\{p_i^{(3)}(1.8, \pi/2)\}$	$(3.0, 0.5)$	$(2.0, 0.03)$
Hexagonal six-well	$\{p_i^{(6)}(1.7, 0)\}$	$(2.0, 0.5)$	$(1.0, 0.03)$
Octagonal eight-well	$\{p_i^{(8)}(2.2, 0)\}$	$(3.0, 0.5)$	$(0.90, 0.02)$
Pentagonal five-well	$\{p_i^{(5)}(1.8, \pi/2)\}$	$(2.0, 0.5)$	$(1.1, 0.03)$
Square four-well	$\{p_i^{(4)}(1.8, \pi/4)\}$	$(3.0, 0.5)$	$(1.0, 0.03)$
Diamond four-well	$\{(0, 2.2), (2.2, 0), (0, -2.2), (-2.2, 0)\}$	$(2.5, 0.5)$	$(1.0, 0.02)$
L-shaped five-well	$\{(-1.5, 1.5), (-1.5, 0), (-1.5, -1.5), (0, -1.5), (1.5, -1.5)\}$	$(2.5, 0.5)$	$(1.0, 0.03)$
Asymmetric three-well	$\{(0, 1.8), (-1.5, -0.9), (1.5, -0.9)\}$	$\{(2.5, 0.55), (1.5, 0.4), (2.0, 0.5)\}$	$(1.0, 0.03)$
Depth-gradient four-well	$\{(-1.3, 1.3), (1.3, 1.3), (1.3, -1.3), (-1.3, -1.3)\}$	$\{(2.2, 0.55), (2.5, 0.5), (3.0, 0.5), (3.5, 0.5)\}$	$(1.3, 0.03)$

Gaussian-well family. Most retained catalog systems use a common two-dimensional Gaussian potential with an independent rotational drift. For $x = (x_1, x_2) \in \mathbb{R}^2$, let $Rx = (x_2, -x_1)$ and

$$V_s(x) = \sum_{i=1}^{M_s} -a_{s,i} \exp\left(-\frac{\|x - c_{s,i}\|_2^2}{2\sigma_{s,i}^2}\right) + \gamma_s(x_1^4 + x_2^4).$$

The implemented vector field is

$$\dot{x} = -\nabla V_s(x) + \omega_s R x. \tag{14}$$

Define $p_i^{(M)}(r, \varphi) = r(\cos(2\pi i/M + \varphi), \sin(2\pi i/M + \varphi))$, $i = 0, \ldots, M-1$. The retained Gaussian-well systems instantiate equation (14) with the parameters in table 5.

The transition-routes system uses the same Gaussian-well form with centers

$$\{c_i\} = \{(-1.8, 1.8), (1.8, 1.8), (-1.8, -1.8), (1.8, -1.8)\},$$

$a_i = 3.0$, $\sigma_i = 0.6$, $\omega = 1.0$, and $\gamma = 0.03$, and adds a corridor-dependent rotation term:

$$\dot{x} = -\nabla V_s(x) + \big(1.0 + 0.3\, C_{\text{route}}(x_2)\big) R x, \qquad C_{\text{route}}(x_2) = e^{-(x_2 - 1.8)^2/0.3} + e^{-(x_2 + 1.8)^2/0.3}. \tag{15}$$

Native gated local-linear systems. Both native gated systems place three centers on a circular layout, but with different radii and region definitions. Let $Q(\alpha) = \left(\begin{smallmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{smallmatrix}\right)$ and $\alpha_b = 2\pi b/3$, $b \in \{0, 1, 2\}$.

For `gated_local_linear`, $c_b = 1.75(\cos\alpha_b, \sin\alpha_b)$. Inside the radius-1.05 core of basin b,

$$\dot{x} = A_b(x - c_b), \qquad A_b = Q(\alpha_b)\widetilde{A}_b Q(\alpha_b)^\top,$$

with

$$\widetilde{A}_0 = \left(\begin{smallmatrix} -0.9 & -1.2 \\ 1.2 & -0.9 \end{smallmatrix}\right), \quad \widetilde{A}_1 = \left(\begin{smallmatrix} -1.35 & 0.2 \\ -0.3 & -0.7 \end{smallmatrix}\right), \quad \widetilde{A}_2 = \left(\begin{smallmatrix} -0.7 & -0.1 \\ 0.5 & -1.2 \end{smallmatrix}\right).$$

Outside the cores, the active sector is the nearest angular sector $s(x)$ and the shared gate matrix

$$G = \left(\begin{smallmatrix} -1.35 & -0.9 \\ 0.9 & -1.35 \end{smallmatrix}\right)$$

drives the state as $\dot{x} = G(x - c_{s(x)})$.

For `gated_transfer_linear`, $c_b = 1.85(\cos\alpha_b, \sin\alpha_b)$. The implemented radii and widths are

$$\rho_{\text{core}} = 0.30, \quad \rho_{\text{source}} = 0.80, \quad \rho_{\text{exit}} = 0.60, \quad \beta_{\text{exit}} = 0.72,$$
$$w_{\text{chan}} = 0.22, \quad \delta_{\text{lane}} = 0.28, \quad \rho_{\text{hand}} = 0.45.$$

The core matrices use the same rotated form with templates

$$\left(\begin{smallmatrix} -1.0 & -1.1 \\ 1.1 & -1.0 \end{smallmatrix}\right), \quad \left(\begin{smallmatrix} -1.4 & 0.2 \\ -0.2 & -0.7 \end{smallmatrix}\right), \quad \left(\begin{smallmatrix} -0.8 & -0.3 \\ 0.5 & -1.3 \end{smallmatrix}\right).$$

For each ordered pair $p = (s, t)$, $s \neq t$, define

$$d_{s\to t} = \frac{c_t - c_s}{\|c_t - c_s\|_2}, \qquad n_{s\to t} = (-d_{s\to t,2}, d_{s\to t,1}), \qquad o_t = \frac{c_t}{\|c_t\|_2},$$

and $\chi_{s,t} = 1$ when $\sin(\alpha_s - \alpha_t) \geq 0$, otherwise $\chi_{s,t} = -1$. The implemented channel entry and handoff points are

$$\begin{aligned} e_p &= c_s + \rho_{\text{source}} d_{s\to t} + \chi_{s,t}\delta_{\text{lane}} n_{s\to t}, \\ h_p &= c_t + \rho_{\text{hand}} o_t + \chi_{s,t}\delta_{\text{lane}}(-o_{t,2}, o_{t,1}), \end{aligned}$$

with channel direction $d_p = (h_p - e_p)/\|h_p - e_p\|_2$, channel normal $n_p = (-d_{p,2}, d_{p,1})$, and channel length $\ell_p = (h_p - e_p) \cdot d_p$.

The vector field is piecewise analytic. Core regions $\|x - c_b\|_2 \leq \rho_{\text{core}}$ use $A_b(x - c_b)$. Source annuli use $0.65A_b(x - c_b)$, and other non-channel background regions use $0.50A_b(x - c_b)$ for the nearest center c_b. Exit wedges for $p = (s, t)$ are source-neighborhood states outside the core with $\|x - c_s\|_2 \geq \rho_{\text{exit}}$ and $(x - c_s) \cdot d_{s\to t} \geq \|x - c_s\|_2 \cos\beta_{\text{exit}}$; they use

$$\dot{x} = v_{\text{exit}} d_{s\to t} - \lambda_{\text{exit}}\big((x - c_s) \cdot n_{s\to t}\big) n_{s\to t}, \qquad v_{\text{exit}} = 1.0, \quad \lambda_{\text{exit}} = 1.8;$$

and channel rectangles satisfying

$$0 \leq (x - e_p) \cdot d_p \leq \ell_p, \qquad |(x - e_p) \cdot n_p| \leq w_{\text{chan}}$$

use

$$\dot{x} = v_{\text{chan}} d_p - \lambda_{\text{chan}}\big((x - e_p) \cdot n_p\big) n_p, \qquad v_{\text{chan}} = 1.55, \quad \lambda_{\text{chan}} = 2.8.$$

Other specialized systems. The arrested spiral system has a background spiral flow plus four Gaussian traps and an origin trap:

$$\dot{x} = -0.3x + 2.0Rx - 4.0\sum_{i=1}^{4} e^{-\|x-c_i\|_2^2/(2\cdot 0.25)} \frac{x - c_i}{0.25} - 1.5e^{-\|x\|_2^2/0.3} \frac{x}{0.15} - 0.02(x_1^3, x_2^3),$$

with $c_i \in \{(1.5, 0), (0, 1.8), (-1, -1), (0.8, -1.5)\}$. Its fifth basin is the origin trap, matching the benchmark manifest count.

The triple-well Duffing system uses $x = (q, p)$ and

$$V(q) = q^6/6 - q^4/2 + aq^2, \qquad V'(q) = q^5 - 2q^3 + 2aq,$$

with $a = 0.3$, damping $\delta = 0.5$, rotation strength $\omega = 1.0$, and soft cubic confinement $-\epsilon u^3/s^3$ with $\epsilon = 0.003$ and $s = 4.0$:

$$\begin{aligned} \dot{q} &= (1 + \omega)p - \epsilon q^3/s^3, \\ \dot{p} &= -V'(q) - \delta p - \omega q - \epsilon p^3/s^3. \end{aligned} \tag{16}$$

The SNIC system is implemented in polar form and then converted to Cartesian coordinates. With $\varepsilon_{\text{num}} = 10^{-8}$, $r^2 = x_1^2 + x_2^2 + \varepsilon_{\text{num}}$, $r = \sqrt{r^2}$, $\theta = \text{atan2}(x_2, x_1)$, $c_\theta = x_1/(r + \varepsilon_{\text{num}})$, and $s_\theta = x_2/(r + \varepsilon_{\text{num}})$,

$$\dot{r} = r(1 - r^2) - 0.3\cos(3\theta), \qquad \dot{\theta} = 1 - 1.2\cos(3\theta).$$

Table 6: The ten Dysts systems used in the $dt\times30$ forecasting benchmark. The "stored step" column is 30 times the native Dysts integration interval.

Display name	System key	Dimension	Native dt	Stored step $30dt$
Chua	`Chua`	3	$2.8474745791\times10^{-4}$	$8.5424237373\times10^{-3}$
Dadras	`Dadras`	3	$6.5782963827\times10^{-4}$	$1.9734889148\times10^{-2}$
Dequan Li	`DequanLi`	3	$1.6763993999\times10^{-5}$	$5.0291981997\times10^{-4}$
Hadley	`Hadley`	3	$2.9086847808\times10^{-4}$	$8.7260543424\times10^{-3}$
Lu–Chen–Cheng	`LuChenCheng`	3	$1.8469678280\times10^{-4}$	$5.5409034839\times10^{-3}$
Qi–Chen	`QiChen`	3	$7.8371061844\times10^{-5}$	$2.3511318553\times10^{-3}$
Sakarya	`Sakarya`	3	$9.9704617436\times10^{-4}$	$2.9911385231\times10^{-2}$
San–Um–Srisuchinwong	`SanUmSrisuchinwong`	3	$1.4933288881\times10^{-3}$	$4.4799866644\times10^{-2}$
Shimizu–Morioka	`ShimizuMorioka`	3	$2.4080013336\times10^{-3}$	$7.2240040007\times10^{-2}$
Wang–Sun	`WangSun`	3	$5.3924987498\times10^{-3}$	$1.6177496249\times10^{-1}$

The Cartesian vector field is

$$\dot{x}_1 = \dot{r}c_\theta - r\dot{\theta}s_\theta - 0.01x_1(x_1^2 + x_2^2) + 0.5x_2,$$
$$\dot{x}_2 = \dot{r}s_\theta + r\dot{\theta}c_\theta - 0.01x_2(x_1^2 + x_2^2) - 0.5x_1.$$

These analytic generators define the benchmark trajectories. Held-out evaluation annotations are produced by the benchmark label helpers or by endpoint-rollout basin labeling, and the learned models observe only stored states.

D Dysts benchmark inventory and system definitions

This appendix lists the ten Dysts systems used in the paper-facing $dt\times30$ forecasting benchmark. The systems are drawn from Dysts (Gilpin, 2021, 2023). We used the continuous-time right-hand sides and metadata from the pinned package version resolved in the project environment, `dysts` 0.96. All ten systems are three-dimensional autonomous flows. The equations below are written in unstandardized native Dysts coordinates (x, y, z); the training cache standardizes coordinates only after trajectories are generated.

Closed-form convention. For each system, the stored observations are generated from $\dot{x} = f_s(x)$ with observation interval $\Delta t = 30\, dt_{\text{native}}$, where dt_{native} is the system-specific Dysts integration interval in table 6. The learner observes only the resulting stored states, not the vector field, continuous-time solver, or substeps. All ten systems have explicit closed-form right-hand sides in Dysts. Most are polynomial vector fields. The Chua and San–Um–Srisuchinwong systems include absolute-value terms, so they are piecewise smooth rather than globally analytic at the corresponding switching surfaces.

Chua. The Chua system uses the piecewise-linear diode nonlinearity

$$h(x) = m_1 x + \frac{1}{2}(m_0 - m_1)\big(|x+1| - |x-1|\big),$$

with parameters

$$\alpha = 15.6, \qquad \beta = 28, \qquad m_0 = -1.142857, \qquad m_1 = -0.71429.$$

The vector field is

$$\begin{aligned}\dot{x} &= \alpha\big(y - x - h(x)\big),\\ \dot{y} &= x - y + z,\\ \dot{z} &= -\beta y.\end{aligned} \tag{17}$$

Dadras. With parameters

$$c = 2, \qquad e = 9, \qquad o = 2.7, \qquad p = 3, \qquad r = 1.7,$$

the Dadras system is

$$\begin{aligned}\dot{x} &= y - px + oyz,\\ \dot{y} &= ry - xz + z,\\ \dot{z} &= cxy - ez.\end{aligned} \tag{18}$$

Dequan Li. With parameters

$$a = 40, \qquad c = 1.833, \qquad d = 0.16, \qquad \varepsilon = 0.65, \qquad f = 20, \qquad k = 55,$$

the Dequan Li system is

$$\begin{aligned}\dot{x} &= a(y - x) + dxz,\\ \dot{y} &= kx + fy - xz,\\ \dot{z} &= cz + xy - \varepsilon x^2.\end{aligned} \tag{19}$$

Hadley. With parameters

$$a = 0.2, \qquad b = 4, \qquad f = 9, \qquad g = 1,$$

the Hadley system is

$$\begin{aligned}\dot{x} &= -y^2 - z^2 - ax + af,\\ \dot{y} &= xy - bxz - y + g,\\ \dot{z} &= bxy + xz - z.\end{aligned} \tag{20}$$

Lu–Chen–Cheng. With parameters

$$a = -10, \qquad b = -4, \qquad c = 18.1,$$

the Lu–Chen–Cheng system is

$$\begin{aligned}\dot{x} &= -\frac{ab}{a+b}x - yz + c,\\ \dot{y} &= ay + xz,\\ \dot{z} &= bz + xy.\end{aligned} \tag{21}$$

Qi–Chen. With parameters

$$a = 38, \qquad b = 2.666, \qquad c = 80,$$

the Qi–Chen system is

$$\begin{aligned}\dot{x} &= a(y - x) + yz,\\ \dot{y} &= cx + y - xz,\\ \dot{z} &= xy - bz.\end{aligned} \tag{22}$$

Sakarya. With parameters

$$a = -1, \qquad b = 1, \qquad c = 1, \qquad h = 1, \qquad p = 1, \qquad q = 0.4, \qquad r = 0.3, \qquad s = 1,$$

the Sakarya system is

$$\begin{aligned} \dot{x} &= ax + hy + syz, \\ \dot{y} &= -by - px + qxz, \\ \dot{z} &= cz - rxy. \end{aligned} \tag{23}$$

San–Um–Srisuchinwong. With parameter $a = 2$, the San–Um–Srisuchinwong system is

$$\begin{aligned} \dot{x} &= y - x, \\ \dot{y} &= -z \tanh(x), \\ \dot{z} &= -a + xy + |y|. \end{aligned} \tag{24}$$

Shimizu–Morioka. With parameters

$$a = 0.85, \qquad b = 0.5,$$

the Shimizu–Morioka system is

$$\begin{aligned} \dot{x} &= y, \\ \dot{y} &= x - ay - xz, \\ \dot{z} &= -bz + x^2. \end{aligned} \tag{25}$$

Wang–Sun. With parameters

$$a = 0.2, \qquad b = -0.01, \qquad d = -0.4, \qquad e = -1, \qquad f = -1, \qquad q = 1,$$

the Wang–Sun system is

$$\begin{aligned} \dot{x} &= ax + qyz, \\ \dot{y} &= bx + dy - xz, \\ \dot{z} &= ez + fxy. \end{aligned} \tag{26}$$

E High-dimensional and application benchmarks

The main experiments isolate basin-support alignment in controlled low-dimensional systems. The next benchmark additions should test the same claim in settings where autoencoding and regime discovery are more demanding, while preserving the paper's label-free training protocol. In these proposed benchmarks, labels are hidden from the encoder, decoder, transition model, sparsity rule, support-family construction, and checkpoint selection. Labels are used only after training to evaluate whether support families are useful regime variables.

E.1 Completed Lorenz–96 high-dimensional forecasting pilot

As a first scaling check, we ran a noisy $D = 128$ Lorenz–96 forecasting pilot with 64 training trajectories, 16 validation trajectories, 16 test trajectories, seeds $0, 1, 2$, and observation noise equal to 0.05 times the training-set channel standard deviation. The neural comparison used overcomplete latents with $d_z = 512 = 4D$, a 20-step training rollout window, dense learned K matrices throughout, and no K regularization. Sparse rows used latent sparsity through the sparse MLP encoder/objective or LISTA-family encoder nonlinearities. A short 120-step smoke run was not enough for this comparison; the paper-facing pilot uses a 2,000-step maximum budget with the same early-stopping rule for all neural rows.

With the longer budget, LISTA-sign-split was the strongest H50/H100 model. Its mean NRMSE was 0.8832 at H50 and 0.9525 at H100, compared with dense MLP KAE 0.9131/1.0081, DMD 0.9490/0.9741, and truncated-SVD DMD 0.9473/0.9775. Paired trajectory-bootstrap differences favored LISTA-sign-split over dense MLP KAE at H50/H100 by -0.0299 and -0.0556, over DMD by -0.0658 and -0.0216, and over truncated-SVD DMD by -0.0641 and -0.0250; all corresponding 95% intervals excluded zero. Other sparse rows were less consistent: Sparse-MLP $L_1 = 0.1$ improved H50 but not H100, LISTA-shrink approached dense MLP but remained slightly worse at H100, and HyperLISTA remained short-horizon accurate but long-horizon unstable.

This pilot supports a focused overcomplete scaling statement: under a fair longer training budget, the sign-split LISTA sparse encoder improves long-horizon noisy Lorenz–96 forecasting over the matched dense MLP KAE and over full-state DMD baselines in this $D = 128$ condition. The result is still not a blanket claim about all LISTA variants or all sparsity mechanisms, and it remains a full-state benchmark rather than a partial-observation test.

E.2 Spatialized multibasin reaction–diffusion fields

The low-risk high-dimensional benchmark is a spatialized version of the existing procedural multibasin generators. For a selected source system s, let $f_s : \mathbb{R}^2 \to \mathbb{R}^2$ be the same two-dimensional vector field used in the main benchmark, with known attractor centers used only for evaluation. The spatial field is

$$u(\xi, t) \in \mathbb{R}^2, \qquad \xi \in [0, 1]^2, \tag{27}$$

with dynamics

$$\partial_t u(\xi, t) = f_s(u(\xi, t)) + \nu \Delta u(\xi, t). \tag{28}$$

On an $N \times N$ periodic grid, the discretized dynamics are

$$\dot{u}_{i,j} = f_s(u_{i,j}) + \nu \left(u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j} \right), \tag{29}$$

so the observed state dimension is $2N^2$, e.g. 8192 for $N = 64$. The first pass should use $N = 64$, $\nu \in \{0.005, 0.01, 0.02\}$, RK4 internal step size 0.005 or 0.01, periodic boundary conditions, and stored observations every 10–20 integration steps. The model observes only stored fields, not vector fields or integration substeps. This follows the storage and high-dimensional forecasting style of PDEBench and The Well, which provide reference conventions and baseline context for time-dependent PDE datasets (Takamoto et al., 2022; Ohana et al., 2024).

The paper-facing high-dimensional multibasin variant should prioritize a multistable Allen–Cahn phase-field source, implemented as a two-channel order parameter with smooth multiwell potential W and dynamics $\partial_t u = \nu \Delta u - \nabla W(u)$. The wells are known only for evaluation, where final fields can be assigned global modal-well labels and per-pixel well maps. This keeps the benchmark close

to a recognizable multistable PDE while preserving the label-free training rule and the existing F_{abs} support-family evaluation.

Initial conditions should create spatially mixed local basin membership rather than independent ODE copies. For each trajectory, choose $r \in \{2, 3\}$ attractor centers $c_1, \ldots, c_r$, sample low-frequency Fourier noise fields $g_k(\xi)$, and convert them to soft masks

$$w_k(\xi) = \frac{\exp(g_k(\xi)/\tau)}{\sum_{m=1}^{r} \exp(g_m(\xi)/\tau)}. \tag{30}$$

The initial condition is

$$u(\xi, 0) = \sum_{k=1}^{r} w_k(\xi) c_k + \epsilon(\xi), \tag{31}$$

where $\epsilon(\xi)$ is small optionally smoothed Gaussian noise. Boundary-like cases can be oversampled by requiring each selected attractor to occupy a nontrivial initial area fraction. This produces fields with domain walls and coarsening-like transients.

Trajectory labels are computed after simulation and are never passed to the model. At a long label horizon T_{label}, each grid site is assigned to its nearest attractor center,

$$b_{i,j} = \arg\min_{m} \left\| u_{i,j}(T_{\text{label}}) - c_m \right\|_2, \tag{32}$$

and the trajectory-level final basin label is the modal site label B_{global}. The final majority fraction

$$\rho_{\text{maj}} = \max_{m} \frac{1}{N^2} \sum_{i,j} \mathbf{1}\{b_{i,j} = m\} \tag{33}$$

records how much of the final field belongs to the modal basin. The primary support-alignment read should use all held-out trajectories, with B_{global} hidden until evaluation, matching the main paper's all-held-out forecasting convention and avoiding an extra benchmark-specific filter. The majority fraction is a diagnostic stratification variable for asking whether support alignment is stronger on nearly single-basin fields than on mixed-domain fields; it is not a required filter for the main benchmark claim.

The initial dataset should cover five source systems, five seeds, and one grid size before expansion. Recommended source systems are `cal_square_4`, `cal_high_cross_3`, `transition_routes_4`, `gated_local_linear`, and `var_l_shape_5`; `cal_octagon_8`, `duffing_triple_well`, and `snic_multi` are escalation cases. Store HDF5 arrays as [trajectory, time, x, y, channel], with metadata for source system, attractor centers, final basin label, majority fraction, per-pixel final basin map, stored Δt, diffusion coefficient, and initial-condition seed.

Models should use convolutional encoders and decoders rather than flattening into the current MLP. Koopman-lifting runs should keep an overcomplete latent with $d_z \geq 4d_x$, where $d_x = 2N^2$; thus $N = 16$ requires $d_z \geq 2048$, $N = 32$ requires $d_z \geq 8192$, and the $N = 64$ setting above requires $d_z \geq 32768$. The main comparison is a convolutional LISTA SKAE with dense K, two or three LISTA refinement loops, and a convolutional decoder back to the two-channel field. Controls should include a dense Conv-KAE, a sparse Conv-MLP KAE without LISTA shrinkage, an FNO or U-Net autoregressive predictor, and an oracle basin-local Conv-KAE trained with evaluation labels only as an upper-bound diagnostic. Forecast metrics should include field MSE, gradient MSE, Fourier-shell MSE, and final-basin consistency. Support-family metrics should be computed out of sample: build F_{abs} representatives on validation states only, freeze them, assign test states by nearest Jaccard representative, and report $H(B_{\text{global}} \mid F_{\text{abs}})$, NMI, ARI, purity, $|F_{\text{abs}}|$, and $H(F_{\text{abs}} \mid B_{\text{global}})$.

Table 7: Allen–Cahn H200 no-reencoding pilot. Entries are mean held-out field MSE across three seeds; lower is better.

Model	H20 field	H50 field	H100 field	H200 field	H200 final
Dense Conv-KAE	0.611	0.920	1.066	1.174	1.363
LISTA Conv-KAE	0.655	0.931	1.061	**1.114**	**1.142**
Sparse-MLP Conv-KAE	**0.608**	**0.900**	**1.038**	1.147	1.372

Table 8: Allen–Cahn H200 field MSE by periodic re-encoding mode. Re-encoding decodes and re-encodes the model's own prediction; it never uses the future ground truth.

Rollout mode	Dense Conv-KAE	LISTA Conv-KAE	Sparse-MLP Conv-KAE
No re-encoding	1.174	**1.114**	1.147
Period 1	**1.314**	1.364	1.366
Period 5	1.296	1.313	**1.276**
Period 10	1.222	1.209	**1.205**
Period 20	1.175	1.195	**1.172**
Period 50	1.169	**1.135**	1.155

Completed Allen–Cahn H200 pilot. We completed a three-seed 16×16 two-channel multistable Allen–Cahn pilot on `allen_cahn_4`. The state dimension is $d_x = 512$, and all Koopman rows used $d_z = 2048 = 4d_x$, tanh convolutional hidden activations, dense latent K, no K-stability penalty, length-20 training windows, and 10,000 optimizer steps. Each seed used 48/12/12 train/validation/test trajectories and passed a preflight check confirming that all four final modal basins appeared in every split. The stored observation cadence is four RK4 substeps of size 0.005, so one forecast step is physical time 0.02 and $H200$ is physical time 4.0. In the autonomous no-reencoding rollout, Sparse-MLP had the lowest mean field MSE through $H100$, while LISTA had the lowest $H200$ average and final-step error: 1.114 average field MSE and 1.142 final-step MSE, compared with dense Conv-KAE 1.174 and 1.363. The seed-paired $H200$ no-reencoding field-MSE difference versus dense was -0.060 ± 0.053 for LISTA and -0.026 ± 0.027 for Sparse-MLP, where negative favors the sparse-latent row. Periodic re-encoding did not uniformly help in this 10,000-step pilot. Every-step and period-5 re-encoding worsened $H200$ error, period 10 favored Sparse-MLP, period 50 preserved a LISTA advantage over dense, and the best observed $H200$ LISTA score remained the autonomous no-reencoding score.

The longer 50,000-step run changes the endpoint interpretation. No-reencoding error improves for every model, but the improvement is largest for dense Conv-KAE and Sparse-MLP: $H200$ field MSE changes from 1.174 to 1.068 for dense, from 1.114 to 1.096 for LISTA, and from 1.147 to 1.073 for Sparse-MLP. Thus the 10,000-step autonomous LISTA advantage is not stable to a longer training budget. Under 50,000 steps, dense has the best no-reencoding $H100/H200$ average field error, Sparse-MLP is essentially tied with dense, and LISTA has the lowest $H200$ final-step error but worse average rollout error. Periodic re-encoding becomes more useful after longer training: period 20, matching the training-window length, gives the best $H200$ score for all three rows, and Sparse-MLP obtains the lowest $H200$ field MSE, 0.876, compared with dense 0.928 and LISTA 1.003. The seed-paired Sparse-MLP period-20 $H200$ difference versus dense is -0.052 ± 0.118, so this is promising high-dimensional forecasting evidence but not a strong statistical claim from only three seeds. The support-family readout still needs a stronger noncollapsed family structure before

Table 9: Allen–Cahn no-reencoding H200 field MSE under longer training. The 50,000-step run reuses the same train/validation/test trajectories as the 10,000-step run.

Model	10k H200	50k H200	Relative change
Dense Conv-KAE	1.174	1.068	-9.0%
LISTA Conv-KAE	1.114	1.096	-1.6%
Sparse-MLP Conv-KAE	1.147	1.073	-6.5%

Table 10: Allen–Cahn 50,000-step no-reencoding pilot. Entries are mean held-out field MSE across three seeds; lower is better.

Model	H20 field	H50 field	H100 field	H200 field	H200 final
Dense Conv-KAE	0.367	0.597	**0.896**	**1.068**	1.239
LISTA Conv-KAE	0.479	0.687	0.942	1.096	**1.225**
Sparse-MLP Conv-KAE	**0.356**	**0.596**	0.897	1.073	1.300

this benchmark can support the basin-support alignment claim.

The LISTA-only stiffness sweep shows that the weaker 50,000-step LISTA endpoint was partly a hyperparameter issue. The best validation-H50 LISTA setting, $q = 1, \alpha = 10^{-2}$, reached validation MSE 0.582, close to the dense and Sparse-MLP validation means 0.585, and much better than the original LISTA setting's 0.668. The per-horizon diagnostic bests in table 12 recover the no-reencoding H50–H200 gap and slightly edge the dense and Sparse-MLP references, but only by less than 1.3% in mean field MSE and with only three seeds. Under period-20 re-encoding, tuned LISTA improves substantially over the original LISTA row, but Sparse-MLP remains the lowest-error model at all displayed horizons. The optimal shrinkage is also horizon- and rollout-mode dependent: nearly no shrinkage is best for autonomous H200, while strong shrinkage is best for period-20 H200. We therefore interpret this sweep as evidence that LISTA shrinkage can be an effective regularizer after tuning, not as evidence that unfolded shrinkage is uniformly the best high-dimensional forecasting mechanism. If this benchmark is promoted beyond extension evidence, the next run should choose the LISTA setting by validation before test evaluation and increase the seed count.

E.3 Lorenz–96 overcomplete sparsity-screening pilot

A resource-aware seed-0 screen was run on noisy Lorenz–96 with $D = 128$, observation noise 0.05, $d_z = 512 = 4D$, dense K, no K-regularization, and length-20 training rollouts. With a 300-minibatch budget, horizon-100 NRMSE was 0.968 for DMD, 0.970 for truncated-SVD DMD, 0.964 for the best ReLU sparse-MLP KAE at latent sparsity coefficient 0.3, 0.988 for the best noncollapsed LISTA-shrink row at $\alpha = 0.03$, 1.083 for the dense tanh KAE, and 1.393 for persistence. A 1000-minibatch run improved the useful sparse rows: the best ReLU sparse-MLP row reached horizon-100 NRMSE 0.948, LISTA-shrink reached 0.958, and dense tanh reached 1.020. A 3000-minibatch higher-sparsity run improved further: Sparse-MLP at latent sparsity coefficient 1.0 reached horizon-100 NRMSE 0.937 with active density 0.854, Sparse-MLP at coefficient 3.0 reached 0.944 with active density 0.600, LISTA-shrink at $\alpha = 0.03$ reached 0.942, and dense tanh reached 0.946. Paired trajectory bootstrap differences in the 3000-update run favored Sparse-MLP 1.0 over DMD at horizon 100, with difference -0.031 and 95% CI $[-0.036, -0.027]$. Pushing sparsity further to Sparse-MLP coefficients 10 and 30 reduced active density to 0.272 and 0.200 but degraded horizon-100 NRMSE

Table 11: Allen–Cahn 50,000-step H200 field MSE by periodic re-encoding mode.

Rollout mode	Dense Conv-KAE	LISTA Conv-KAE	Sparse-MLP Conv-KAE
No re-encoding	**1.068**	1.096	1.073
Period 1	1.317	1.320	**1.251**
Period 5	**1.066**	1.119	1.075
Period 10	**0.948**	1.028	0.994
Period 20	0.928	1.003	**0.876**
Period 50	**1.060**	1.094	1.078

Table 12: Allen–Cahn 50,000-step LISTA shrinkage/depth diagnostic. The tuned LISTA column reports the best tested LISTA setting for each horizon and rollout mode over $q \in \{1, 2, 4\}$ refinement loops and $\alpha \in \{0, 10^{-4}, 3{\times}10^{-4}, 10^{-3}, 3{\times}10^{-3}, 10^{-2}\}$. Entries are mean held-out field MSE across three seeds; lower is better. Because the best setting is reported separately for each horizon and mode, this table is a diagnostic sweep summary rather than a final validation-selected model claim.

Mode	Horizon	LISTA setting	Tuned LISTA	Original LISTA	Dense Conv-KAE	Sparse-MLP Conv-KAE
No re-encoding	H50	$q = 1, \alpha = 10^{-2}$	**0.590**	0.687	0.597	0.596
No re-encoding	H100	$q = 4, \alpha = 3{\times}10^{-4}$	**0.889**	0.942	0.896	0.897
No re-encoding	H200	$q = 1, \alpha = 10^{-4}$	**1.063**	1.096	1.068	1.073
Period 20	H50	$q = 1, \alpha = 3{\times}10^{-4}$	0.538	0.655	0.531	**0.499**
Period 20	H100	$q = 1, \alpha = 10^{-2}$	0.723	0.828	0.717	**0.670**
Period 20	H200	$q = 1, \alpha = 10^{-2}$	0.922	1.003	0.928	**0.876**

to 0.968 and 0.990. This pilot therefore supports moderate sparse-activation regularization as a forecasting aid, but not extreme latent sparsity. LISTA-ReLU remained worse at long horizon for all noncollapsed thresholds. Very large LISTA thresholds collapsed the latent code to zero active coordinates and produced NRMSE exactly 1.0; those rows should be treated as degenerate stable predictors, not evidence for useful sparse dynamics. Future validation selection should include a noncollapse guard, such as a minimum reconstruction or one-step forecast quality constraint, before choosing the sparsest row within a long-horizon error tolerance.

A follow-up LISTA-shrink-only sweep used 30 thresholds and five seeds per threshold under the same $D = 128$, noise 0.05, $d_z = 512$, dense-K, no-K-regularization protocol, with a 3000-epoch maximum budget and early stopping. Validation H100 selected $\alpha = 0.015$, which had latent active density 0.990 and test H100 NRMSE 0.945 ± 0.010 across seeds. The seed-paired H100 difference versus DMD was -0.0316, with 95% seed-bootstrap interval $[-0.0354, -0.0255]$, and the difference versus truncated-SVD DMD was -0.0342, with interval $[-0.0403, -0.0267]$. This confirms that LISTA-shrink can beat DMD at long horizon in this condition, but the H100-optimal LISTA-shrink code is nearly dense. By contrast, the best H5–H50 LISTA-shrink rows used much sparser codes, with active density 0.55–0.62, so the optimal sparsity is horizon-dependent rather than a single universal target.

Lorenz–96 should only be used as multibasin evidence after a separate attractor audit. Some parameter regimes near bifurcations may exhibit coexisting periodic, quasiperiodic, or other attractors, but the paper must verify coexistence for the exact D, F pairs used. The audit criterion is numerical and conservative: launch diverse initial-condition families, discard long transients, cluster asymptotic tail features, and promote a parameter pair only when distinct reproducible tail clusters survive longer reruns and qualitative inspection. Otherwise Lorenz–96 remains a high-dimensional chaotic forecasting stress test rather than a basin-support alignment benchmark.

E.4 State-observation control world models

A lower-risk action-conditioned pilot should precede pixel observations and policy learning. The first pass uses state observations from four continuous-control tasks: cartpole swingup, finger spin, cheetah run, and walker walk. The model is trained offline from trajectories containing o_t, a_t, r_t, and a continuation variable c_t. No task labels, outcome labels, contact phases, basin counts, or trajectory clusters are used during training.

The controlled Koopman candidates use either

$$z_{t+1} = Kz_t + Ba_t \tag{34}$$

or

$$z_{t+1} = K_0 z_t + \sum_j a_{t,j} K_j z_t + Ba_t. \tag{35}$$

Both variants are trained with sparse and dense transition-matrix penalties. The dense no-sparsity baselines use tanh hidden activations by default. A parameter-matched residual MLP latent transition is the first non-Koopman control. Reward and continuation heads predict $\hat{r}_t = f_r(z_t, a_t)$ and $\hat{c}_t = f_c(z_t, a_t)$ from the same latent state and action used by the transition.

The primary readout is whether sparse transition matrices improve downstream world-model utility rather than only one-step prediction. Report one-step latent error, 5-, 10-, 20-, and 50-step open-loop observation error, reward error, continuation loss, rollout throughput, random-shooting planning latency, peak memory, unstable rollout fraction, base transition spectral radius, and effective matrix density. Data-efficiency slices should train on 10%, 25%, 50%, and 100% of the training trajectories. Robustness can then perturb mass, friction, action delay, or hidden dynamics parameters once the clean state-observation comparison is complete.

These experiments should be treated as extension evidence unless they show a clear advantage in data efficiency, planning latency, or stable long-horizon rollouts. A general claim that sparse Koopman autoencoders replace modern world-model agents would require online data collection, stochastic latent states, value or policy learning, pixel or history encoders, and broader task coverage.

E.5 Contact-rich robotic insertion in ManiSkill

The higher-risk application-style benchmark is robotic insertion in ManiSkill3 (Tao et al., 2024). The first task should be `PegInsertionSide-v1`, with `PlugCharger-v1` as a stretch task after the first signal is clear. These tasks contain randomized insertion geometry and demonstrations, making them suitable for testing whether sparse supports discover contact and outcome regimes rather than only synthetic basin labels.

Because the current KAE is autonomous, the robotics benchmark should use a controlled SKAE. Given observation o_t and action a_t,

$$z_t = E_\theta(o_t), \qquad z_{t+1}^{\text{roll}} = Kz_t + B\phi(a_t), \qquad \hat{o}_{t+1} = D_\phi(z_{t+1}^{\text{roll}}). \tag{36}$$

The support is still extracted only from z_t, not from labels or hand-coded contact modes. The minimum-change alternative is to replay a fixed policy and model the closed-loop observation dynamics autonomously, but the controlled version is the cleaner extension because it separates action effects from the discovered support variable.

The first dataset should be state-only, not pixel-based. Start from downloaded demonstrations, recreate each environment from its metadata, replay the demonstrated action sequence, and generate perturbed rollouts using action noise and fixed action or end-effector biases. The rollout set should

be deliberately balanced across successful insertions, rim jams, free-space misses, drops, and unstable partial insertions. Compact observations should include robot configuration and velocity, end-effector pose, gripper state, peg pose and velocity, box or receptacle pose, relative peg-hole pose, insertion depth, distance from peg tip to hole center, contact indicators when available, and the previous action. A first target scale is 10,000 rollouts, 100–150 steps each, split by reset seed and object geometry; after the state-only result works, RGB-D wrist and fixed-camera observations can be added.

Outcome and phase labels are evaluation-only. A trajectory-level outcome B can distinguish success, jammed-at-rim, dropped, missed-no-contact, and partial-insert-unstable outcomes using the simulator success flag plus contact persistence, peg-hole distance, insertion depth, gripper/peg relation, and final stability rules. A timestep-level contact phase C_t can label free-space approach, grasped transport, alignment, first contact, sliding insertion, jam, seated success, and drop or recovery. These labels are never used for training the model or constructing support families.

The robotics evaluation mirrors the basin-support protocol. Forecasting horizons should include 10, 25, 50, and 100 steps, with object-pose MSE, relative peg-hole pose MSE, insertion-depth error, contact-mode prediction F1, and outcome accuracy from the predicted rollout. Support-family evaluation should build F_{abs} on validation states only and assign all held-out test states by frozen Jaccard representatives, reporting $H(B \mid F_{\text{abs}})$, NMI, ARI, $H(C_t \mid F_{\text{abs}})$, and contact-phase purity. The most important mechanistic figure is a support-family timeline over an insertion episode, aligned with camera frames, insertion depth, and contact signal. The intervention analogue is a success-to-jam or jam-to-success support swap at first contact under the same action sequence, measuring changes in predicted insertion depth, success probability, and contact rollout.

Baselines should include a dense controlled KAE, a sparse MLP controlled KAE without LISTA shrinkage, a GRU or Transformer world model, a mixture-of-linear-dynamics model with a learned soft gate, and a contact-oracle local linear model as an upper bound. The benchmark is worth promoting into the main evidence only if SKAE improves long-horizon relative-pose or insertion-depth prediction over the dense controlled KAE, support families align with outcome or contact labels better than dense and sparse-MLP controls, support swaps change predicted insertion outcomes under fixed actions, and the support timeline visibly changes at contact events.

F Support definitions and sensitivity

Section 3.4 defines the notation used in the main text. This appendix records the implementation-level objects behind that notation and explains how to read the corresponding diagnostics. All support objects are computed after training from the learned encoder output $z = E_\theta(x)$. Basin labels, basin counts, and trajectory-to-basin assignments are not used to train the model, define a support, or build a support family; when labels appear below they are evaluation-only quantities used to score or display the learned objects.

From a latent vector to an exact support key. The reducers first convert each latent vector into a Boolean mask $m(x) \in \{0,1\}^{d_z}$. The exact support $S(x)$ is the index set of the true entries of this mask, but the code compares exact supports by serializing the Boolean mask itself: two states have the same exact support if and only if all d_z Boolean entries match. The main support rule used in the paper is

$$m_{\text{abs},i}(x) = \mathbf{1}\{|z_i| > 10^{-3}\}, \qquad S_{\text{abs}}(x) = \{i : m_{\text{abs},i}(x) = 1\}, \tag{37}$$

with $\epsilon = 10^{-12}$ in the implementation. The inequalities are strict. The epsilon only affects the degenerate all-zero relative-threshold case, where S_{rel} is empty. S_{abs} is a magnitude-thresholded active set, so $|S_{\text{abs}}|$ is a measured state-dependent statistic rather than a fixed sparsity budget.

From exact supports to support families. For a chosen support rule and evaluation collection, the implementation counts distinct exact support keys, sorts them from most to least frequent, and uses the serialized key as a deterministic tie-breaker. It then performs the leader-style greedy Jaccard merge described in section 3.4. Each family stores an actual Boolean support vector as its representative. A new exact mask joins the existing representative with the largest Jaccard similarity if that similarity is at least 0.5; otherwise it starts a new family. Representatives are not optimized, averaged, or updated as centroids. Family labels are therefore run- and collection-specific integer identifiers, and only their induced partition of states is meaningful.

Support-family creation algorithm. For completeness, the family construction used for both F_{abs} and F_{top8} is:

Input: evaluation states $\mathcal{X}$, a support rule q, and Jaccard threshold $\tau = 0.5$.
Output: family label $F_q(x)$ for each $x \in \mathcal{X}$, and family representatives $\{r_f\}$.

1. Compute one exact Boolean mask $m_q(x) \in \{0,1\}^{d_z}$ for every $x \in \mathcal{X}$.
2. Count the multiplicity $c(m)$ of each distinct exact mask m.
3. Sort the distinct masks in decreasing $c(m)$, using the serialized Boolean mask as a deterministic tie-breaker.
4. Initialize an empty list of families and representatives.
5. For each distinct mask m in the sorted order:
   a. If no family exists, create a new family f, set $r_f \leftarrow m$, and assign m to f.
   b. Otherwise compute $f^\star = \arg\max_f J(m, r_f)$ over existing family representatives.
   c. If $J(m, r_{f^\star}) \geq \tau$, assign every occurrence of m to family $f^\star$.
   d. If $J(m, r_{f^\star}) < \tau$, create a new family f, set $r_f \leftarrow m$, and assign every occurrence of m to f.
6. For each state x, return $F_q(x)$, the family assigned to its exact mask $m_q(x)$.

The representative update $r_f \leftarrow m$ occurs only when a new family is created. Existing representatives are never averaged, recomputed as majority masks, or moved toward later assigned masks.

For top-eight masks, $J(m, r) \geq 0.5$ is equivalent to at least six shared active coordinates because both masks have size eight. For S_{abs}, the same threshold is adaptive to mask size through the intersection-over-union denominator. A lower Jaccard threshold merges more exact supports and can collapse distinct basins; a higher threshold separates more masks and can make $H(B \mid F)$ small by over-fragmenting each basin. We therefore fix 0.5 rather than tuning it post hoc and interpret any entropy result together with the corresponding family count.

Stable support component construction. C_{stab} is used in the staged local-linearization experiment, not as the primary object in the basin-support alignment table. It also starts from the absolute mask $m_{\text{abs}}(x)$, but the first label is not F_{abs}. For this experiment, exact m_{abs} masks are merged into a finer base support-family label u_t with Jaccard threshold 0.8. Given a training-split trajectory, $u_{i,t}$ denotes the base label of trajectory i at stored time t. A transition $u \to v$ means that consecutive stored observations satisfy $u_{i,t} = u$ and $u_{i,t+1} = v$. The empirical transition graph is built from 512 trajectories of length 192 after the first half of LISTA training, while the encoder and decoder are held fixed.

The graph is filtered to retain sufficiently frequent transitions, its strongly connected components are computed, and recurrent components are selected when they appear in trajectory tails and have small outbound transition mass. Each observed base label u is then assigned to the recurrent component that its future support trajectory reaches with high empirical confidence. This assignment is $C_{\mathrm{stab}}(u)$. Thus C_{stab} groups base support families by support-flow fate; it is neither a ground-truth basin label nor an estimate of a state-space fixed point.

Dense tail-fate control. The dense control used to audit the local-linearization result deliberately removes the sparse support graph. It starts from the dense zero-sparsity MLP KAE, encodes route-fitting trajectories after the global training stage, and summarizes each dense latent trajectory by the mean, standard deviation, and final value of its tail. These continuous tail summaries are standardized, optionally PCA-compressed, and clustered with k-means using a silhouette-selected cluster count up to 12. All transitions from a trajectory inherit that trajectory's dense fate cluster. During forecasting, the future trajectory tail is unavailable, so the route is assigned by nearest centroid in the current dense latent space, using the mean current latent state for each fitted dense fate label. This is the intended contrast with C_{stab}: the dense control asks which dense fate centroid the current state is closest to, whereas C_{stab} asks which recurrent support-flow component the current sparse support pattern is assigned to. Both objects can contain basin-fate information, but only C_{stab} exposes an active-coordinate support graph and routes from the current support mask.

Which support object is used where.

S_{abs}.
: This is the strict active-coordinate object. It is used to build F_{abs}, to report exact-support diagnostics such as $H(B \mid S_{\mathrm{abs}})$, $H(S_{\mathrm{abs}} \mid B)$, exact-support uniqueness, and $|S_{\mathrm{abs}}|$, and to form canonical masks for wrong-support interventions. In the intervention code, the canonical mask for basin b is the most frequent S_{abs} key among candidate deep-slice states from basin b. The "wrong" condition replaces it with canonical masks from other represented basins and holds that mask fixed during the masked latent rollout.

F_{abs}.
: This is the main basin-support alignment object in table 2. It asks whether the many exact S_{abs} masks produced by a sparse encoder compress into basin-scale families. The primary directional metric is $H(B \mid F_{\mathrm{abs}})$: low values mean that knowing the support family leaves little uncertainty about the withheld basin label. The count $\overline{|F_{\mathrm{abs}}|}$ is deliberately non-directional and should be compared with the number of basin labels represented in the evaluated slice.

C_{stab}.
: This is the route object for the staged local affine maps in section K. During the second half of training, a latent source state is converted to m_{abs}, assigned to a base support-family label u_t, mapped to $c = C_{\mathrm{stab}}(u_t)$, and then used to choose the affine latent map $T_c(z) = d_c + K_c(z - \bar{z}_c)$. All local maps are trained in parallel with the encoder and decoder frozen. During forecasting with periodic re-encoding, the route is recomputed from the model's current predicted state. Exact support masks observed when C_{stab} was built map directly to their component; unseen masks are assigned to the nearest component prototype when their Jaccard overlap is at least 0.4; otherwise the rollout uses the frozen global K step.

How to read the diagnostics. A low $H(B \mid S_{\mathrm{abs}})$ or $H(B \mid F_{\mathrm{abs}})$ means that the support object predicts basin identity on the evaluated benchmark states. It does not by itself imply a compact

Table 13: Accumulated MSE at $H = 21$ for support-coordinate interventions. Lower is better; all intervention rows are worse than the standard rollout.

Rollout	Mean ± SD	Median [IQR]
Standard	0.0158±0.0127	0.0140 [0.0055, 0.0224]
Drop top-1	0.508±0.647	0.218 [0.148, 0.677]
Drop top-2	1.37±0.938	1.06 [0.802, 1.63]
Drop top-3	2.08±1.10	1.80 [1.34, 2.10]
Drop top-5	3.26±2.44	1.95 [1.61, 4.17]
Drop top-10	8.51±6.83	5.16 [4.58, 8.11]
Random support	1.69×10^3±2.19×10^3	874 [377, 2.06×10^3]

one-support-per-basin code: a model can achieve low $H(B \mid S_{\text{abs}})$ while assigning many different exact masks to the same basin. $H(S_{\text{abs}} \mid B)$, exact-support uniqueness, and $|S_{\text{abs}}|$ diagnose this fragmentation, while $H(B \mid F_{\text{abs}})$ and $\overline{|F_{\text{abs}}|}$ ask whether the fragmented exact masks merge into a basin-scale family structure.

The takeaways are as follows. F_{abs} is the object for the claim that sparse supports identify basin interiors. S_{abs} is the intervention object for testing whether the active coordinates are functionally used. C_{stab} is the route object for the staged local-linearization result; it is built from support transitions after stage-one training and then used to select local affine latent dynamics during stage-two training and periodic-reencoding rollout.

G Support-coordinate intervention table

Figure 2 plots the full horizon curves for the representative support-coordinate intervention experiment. Table 13 reports the corresponding accumulated MSE summary at $H = 21$. The coordinate-dropping rows use 100 held-out basin-interior starts. The random-support row summarizes 20 random inactive-coordinate reassignments for each of the same 100 starts.

H Support refresh after controlled transfer

Table 14 reports the controlled-transfer support-refresh diagnostic described in section A. The stale value is the F_{top8} family implied by advancing the latent with the learned global transition before a scheduled refresh. The re-encoded value is the family obtained by encoding the current controlled-transfer state at the same event. Higher target-family rates after re-encoding indicate that the encoder can re-select the target-basin support family from the current state without using a support-conditioned local map.

I Statistical testing protocol

This appendix describes exactly what the significance annotations test. Point estimates and statistical annotations answer different questions. Point estimates in the forecasting and support-diagnostic tables first summarize completed training seeds within each benchmark system by an interquartile mean (IQM), then average those system summaries across systems. The main exception is the descriptive family-count column $\overline{|F_{\text{abs}}|}$, which averages per-system seed means because it is a count

Table 14: Controlled-transfer support-refresh diagnostic. Cells report target-family fraction before re-encoding → after re-encoding for period m; the no-transfer column is a false-target control where lower is better.

	Target F_{top8}: stale → re-encoded				No-transfer
Model	$m = 1$	$m = 5$	$m = 10$	$m = 20$	max ↓
LISTA	0.999→0.999	0.997→0.999	0.994→1.000	0.963→1.000	0.011
LISTA-BD	0.999→0.999	0.995→0.998	0.991→0.999	0.873→1.000	0.013
LISTA-SB	0.998→0.999	0.996→0.998	0.989→0.999	0.865→1.000	0.009
Sparse MLP, BD	0.996→0.997	0.990→0.994	0.988→0.998	0.833→0.999	0.011
Sparse MLP	0.996→0.997	0.993→0.994	0.991→0.998	0.835→0.999	0.015
Dense MLP	0.999→1.000	0.977→0.999	0.885→1.000	0.675→1.000	0.211

diagnostic rather than a directional performance metric. Significance annotations instead ask whether a paired effect is reproducible under the independent unit implied by the experimental design. We therefore do not pool seeds, trajectories, transfers, and systems into one sample.

Common paired effects. For MSE comparisons against the dense-latent MLP baseline in table 1, the paired effect for system s, replicate u, and horizon h is

$$\Delta_{s,u,h} = \log_{10} E^{\text{cand}}_{s,u,h} - \log_{10} E^{\text{Dense}}_{s,u,h} = \log_{10}\left(E^{\text{cand}}_{s,u,h}/E^{\text{Dense}}_{s,u,h}\right), \tag{38}$$

where E is the held-out best-periodic MSE and the one-sided alternative is $\Delta < 0$. For routed/global MSE ratios in table 19, the same log-ratio convention is used with E^{routed} and the same model's E^{global}. Log ratios are used only for positive MSE ratios. They match the multiplicative scientific question and reduce the influence of the heavy-tailed seed-level MSE scale.

Support diagnostics use raw paired differences unless the displayed quantity is already an MSE ratio. Mean $|S_{\text{abs}}|$ and $H(B \mid F_{\text{abs}})$ are tested against Dense MLP with alternative candidate $<$ baseline. Wrong-support over base ratios are tested with alternative candidate $>$ baseline, because a larger ratio means that replacing a basin's canonical S_{abs} mask with a different basin's mask is more damaging. The family-count diagnostic $\overline{|F_{\text{abs}}|}$ is not assigned a one-sided test because closeness to the represented basin count is not monotone in either direction.

Holm correction. All Wilcoxon families use Holm step-down correction at $\alpha = 0.05$. If $p_{(1)} \leq \cdots \leq p_{(m)}$ are the eligible raw p-values in a test family, the adjusted value at rank i is the running maximum of $\min\{1, (m - j + 1)p_{(j)}\}$ for $j \leq i$, then mapped back to the original cell order. The implementation uses SciPy's one-sided Wilcoxon signed-rank test with `zero_method="wilcox"`; all-zero or otherwise degenerate paired differences are not counted as Holm passes.

Controlled multibasin forecasting and support diagnostics. For the controlled multibasin columns of table 1 and the support-diagnostic columns of table 2, the replicate unit is the training seed. Candidate and baseline values are paired by the same benchmark system and seed. For each candidate–metric–horizon–subset cell, the table-generating scripts run a separate one-sided paired Wilcoxon signed-rank test within each system over the finite paired seed deltas. MSE tests require positive finite MSEs before the log transform. The paper-facing table builders require at least four finite paired seeds for a system to be eligible for these within-system tests.

The raw per-system p-values are Holm-corrected across eligible systems for that cell. A compact superscript $*$ means the Holm-corrected test passes at 0.05 in the prespecified direction. Appendix tables may instead display the same information as K/N, where K is the number of systems whose Holm-corrected p-value is below 0.05 and N is the number of eligible systems for that diagnostic. The multibasin forecasting cells in the compact main table suppress the displayed K/N because every shown non-Dense multibasin forecasting cell passes on all 15/15 retained systems.

Dysts forecasting superscripts. The Dysts columns in table 1 use benchmark systems, not seeds, as the independent inferential units. For each Dysts system and horizon, seeds are first summarized within system:

$$I_{s,h}^{\mathrm{cand}} = \mathrm{IQM}_r\left\{E_{s,r,h}^{\mathrm{cand}}\right\}, \qquad I_{s,h}^{\mathrm{Dense}} = \mathrm{IQM}_r\left\{E_{s,r,h}^{\mathrm{Dense}}\right\}. \tag{39}$$

Each system then contributes one paired log effect

$$d_{s,h} = \log_{10} I_{s,h}^{\mathrm{cand}} - \log_{10} I_{s,h}^{\mathrm{Dense}}. \tag{40}$$

The compact Dysts table superscripts come from an exact one-sided sign test on the number of systems with $d_{s,h} < 0$. Equivalently, for n retained systems and w systems improved over Dense, the raw p-value is

$$\Pr\{\mathrm{Binomial}(n, 1/2) \geq w\}.$$

These sign-test p-values are Holm-corrected across the non-Dense model–horizon comparisons in the Dysts analysis family, and the compact table reads the corrected sign-test values. The signed-rank and t-test values kept in the Dysts sidecar CSVs are audit diagnostics only; they do not determine the compact table superscripts. With 10 retained Dysts systems and this correction, a displayed Dysts superscript corresponds to improvement on all 10/10 systems in that cell.

System-level support-conditioned routing. The F_{top8}-local routing stars in table 19 test a cross-system claim: whether the post-hoc support-family routed predictor reduces MSE relative to the same checkpoint's global-K rollout. Seeds are nested inside systems, so they are not treated as independent systems.

For each system and horizon, the evaluator first keeps mutually finite positive seed-level routed/global pairs and computes

$$\log_{10}\left(E_{s,r,h}^{\mathrm{routed}} / E_{s,r,h}^{\mathrm{global}}\right).$$

These finite seed-level log ratios are summarized within system by IQM, giving $D_{s,h}$. The table reports $10^{\mathrm{mean}_s\, D_{s,h}}$, so values below one favor routing. The confirmatory p-value is a one-sided exact sign-flip test over the finite system effects $D_{s,h}$ with alternative $\mathrm{mean}_s\, D_{s,h} < 0$. The test enumerates all 2^n sign assignments of the observed magnitudes $|D_{s,h}|$, compares their signed mean with the observed signed mean, and divides the number at least as favorable to the alternative by 2^n. These raw p-values are Holm-corrected over the displayed all-state F_{top8}-local model–horizon cells; $*$ and $**$ denote corrected $p < 0.05$ and $p < 0.01$, respectively.

The companion "system wins" column is directional and diagnostic, not the confirmatory p-value. It uses a censored all-seed-slot comparison so invalid long rollouts do not disappear from the count. For horizon h, a cap

$$C = 1 + \left\lceil \max\left(12, \max_{\mathrm{finite}} \left|\log_{10}(E^{\mathrm{routed}}/E^{\mathrm{global}})\right|\right)\right\rceil$$

is chosen outside the observed finite log-ratio range. Finite positive routed and global MSEs use the ordinary log ratio. A finite routed MSE paired with an invalid same-model global MSE is encoded as $-C$; the reverse is encoded as $+C$. Both invalid, missing, or both-zero seed slots are encoded as 0. The per-system direction is then summarized from these censored seed slots.

Routing robustness and partition-control brackets. The route-form robustness table in table 28 and the partition-control table in table 20 use the same censored seed-slot rule as the routing diagnostic counts above. Their displayed ratios are finite routed/global MSE ratios summarized by seed-IQM within each system and arithmetic mean across systems. Their bracketed K/N counts are within-system one-sided Wilcoxon/Holm counts over the censored seed-level log deltas, with alternative routed $<$ global. These brackets are diagnostics for route-form and partition-control robustness; the main confirmatory routing stars remain the system-level finite-pair sign-flip tests described above.

Support refresh. Table 14 is a descriptive mechanism check rather than a hypothesis-test table. The current paper-facing support-refresh diagnostic uses controlled source-to-target transfers on the retained 15 multibasin systems, training seeds $0, \ldots, 14$, F_{top8} support families, and refresh periods $m \in \{1, 5, 10, 20\}$. Between scheduled events, the latent is advanced only with the learned global K. At each event after target entry, the "stale" label is the target-family indicator for the F_{top8} family of the globally propagated latent before re-encoding, and the "re-encoded" label is the target-family indicator after encoding the current controlled transfer state. Each period cell reports the aggregate target-family fraction before re-encoding $\rightarrow$ after re-encoding. The no-transfer column reports the maximum after-reencoding target-family fraction over matched no-transfer controls. No support-conditioned local map, support-gated transition, or statistical significance test is used for this main refresh table.

Support-coordinate interventions. Figure 2 is also a mechanism diagnostic rather than a cross-system significance table. The coordinate-dropping panel reports mean MSE curves over the selected held-out starts, with 95% bootstrap intervals from 2000 bootstrap resamples of starts. The random support-shuffle panel preserves active coefficient values, moves them to random inactive coordinates, repeats the shuffle procedure, and displays the interquartile range, median, and mean over the shuffle outcomes. These intervals describe intervention variability for the representative system and seed; they are not Holm-corrected hypothesis-test annotations.

J Normalized-decoder audit tables

K Support-stable local-linearization diagnostic

The main text reports the simpler F_{abs}-routed local affine predictor, because matched route controls show that instantaneous support families are already sufficient to improve over the global-K LISTA baseline and are comparable to, or slightly better than, the more elaborate stable support component route. We keep the C_{stab} construction as an appendix diagnostic because it asks a different mechanistic question: whether support families induced by the sparse encoder have a coherent support-flow fate.

Stable support components. The object C_{stab} starts from the same absolute mask $m_{\text{abs}}(x) = \mathbf{1}\{|z_i| > 10^{-3}\}$, but first forms a finer base support-family label u_t using Jaccard threshold 0.8.

Table 15: A/B forecasting check for normalized linear decoder atoms on the controlled multibasin benchmark. The left block repeats the current paper table. The two normalized-decoder blocks normalize linear decoder atoms for every KAE row and differ only in whether the sparsity penalty is applied to rollout latents or encoded latents. Unlike the current table, these normalized-decoder rows are partial seed-3 diagnostics with 13–15 represented systems depending on the row and 1–3 available seeds per system; values are cross-system means after averaging available seeds within each system.

	Current table			Norm. decoder, rollout-z sparsity			Norm. decoder, encoded-z sparsity		
Model	H100	H500	H1000	H100	H500	H1000	H100	H500	H1000
LISTA	0.0407	0.144	0.166	0.0344	0.101	0.116	0.0416	0.164	0.188
LISTA-BD	0.0411	0.118	0.135	0.0598	0.277	0.332	0.0438	0.207	0.252
LISTA-SB	0.0387	0.114	0.130	0.0219	0.0954	0.125	0.0337	0.184	0.233
Sparse MLP, BD	0.0473	0.136	0.159	0.0262	0.0981	0.121	0.0393	0.117	0.138
Sparse MLP	0.0397	0.0940	0.107	0.0410	0.108	0.123	0.0261	0.149	0.194
Dense MLP	0.830	2.68	2.93	0.932	3.12	3.36	0.982	3.16	3.45

Table 16: A/B forecasting check for normalized linear decoder atoms on the retained Dysts $dt\times30$ benchmark. The left block repeats the current paper table, while the two normalized-decoder blocks compare rollout-latent and encoded-latent sparsity targets. These rows are shown as a replacement candidate audit rather than a table replacement because the normalized-decoder packets still have missing retained-system seed runs: 7 for rollout-latent sparsity and 3 for encoded-latent sparsity.

	Current table			Norm. decoder, rollout-z sparsity			Norm. decoder, encoded-z sparsity		
Model	H100	H2000	H4000	H100	H2000	H4000	H100	H2000	H4000
LISTA	2.53×10^{-4}	0.0972	0.452	2.89×10^{-4}	0.0909	0.460	2.89×10^{-4}	0.117	0.459
LISTA-BD	3.38×10^{-4}	0.117	0.485	3.72×10^{-4}	0.135	0.522	3.88×10^{-4}	0.140	0.554
LISTA-SB	4.85×10^{-4}	0.165	0.686	4.00×10^{-4}	0.162	0.532	2.84×10^{-4}	0.115	0.499
Sparse MLP, BD	3.96×10^{-4}	0.113	0.436	3.11×10^{-4}	0.0900	0.433	2.66×10^{-4}	0.0878	0.472
Sparse MLP	2.32×10^{-4}	0.120	0.497	3.31×10^{-4}	0.130	0.572	3.15×10^{-4}	0.114	0.510
Dense MLP	0.00110	0.224	0.754	0.00248	0.292	0.888	0.00245	0.293	0.865

Table 17: A/B support-diagnostic check for normalized linear decoder atoms on the controlled multibasin per-basin-deep slice. The left block repeats the current paper table. The normalized-decoder blocks use the same absolute-threshold support-family diagnostic F_{abs} but are partial seed-3 diagnostics rather than full table replacements.

	Current table		Norm. decoder, rollout-z sparsity		Norm. decoder, encoded-z sparsity	
Model	$H(B\,\vert\,F_{\mathrm{abs}})$	$\overline{\vert F_{\mathrm{abs}}\vert}$	$H(B\,\vert\,F_{\mathrm{abs}})$	$\overline{\vert F_{\mathrm{abs}}\vert}$	$H(B\,\vert\,F_{\mathrm{abs}})$	$\overline{\vert F_{\mathrm{abs}}\vert}$
LISTA	0.130	4.0	0.140	3.9	0.172	3.8
LISTA-BD	0.156	3.8	0.171	3.6	0.188	3.6
LISTA-SB	0.153	3.8	0.152	3.8	0.112	4.0
Sparse MLP, BD	0.358	3.0	0.276	3.2	0.299	3.3
Sparse MLP	0.223	3.3	0.269	3.2	0.324	3.3
Dense MLP	1.28	1.0	1.23	1.0	1.26	1.0

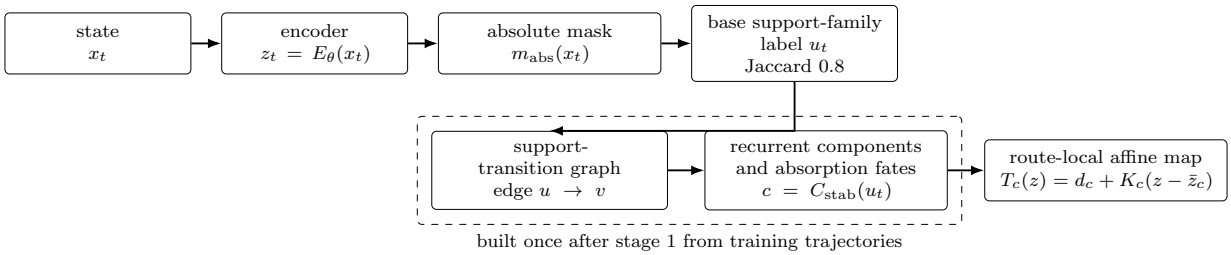


Figure 13: C_{stab} routing used by the staged local-linearization diagnostic. Exact support masks define base labels u_t, transitions define a support-flow graph, recurrent support-flow components define C_{stab}, and $c = C_{\text{stab}}(u_t)$ selects the local affine latent map.

On route-fitting trajectories, consecutive labels define an empirical support-transition graph with edges $u \to v$. We filter this graph by transition frequency, compute strongly connected components, retain recurrent components that appear in trajectory tails and have small outbound probability, and assign each observed base label u to the recurrent component into which its future support trajectory is absorbed with high empirical confidence. Thus $C_{\text{stab}}(u)$ is a support-flow fate label, not a basin label, fixed-point estimate, or prescribed switching mode. Basin labels and basin counts are not used to construct C_{stab}; they are used only afterward for benchmark audits.

Staged training. The staged training recipe is the same as the main F_{abs}-routed local-map experiment. Given total budget $T = 200{,}000$, the first 100,000 steps train the LISTA Koopman autoencoder with a shared encoder E_θ, decoder D_ϕ, and global latent matrix K. We then freeze E_θ, D_ϕ, and K, build C_{stab} from 512 training-split trajectories of length 192, and train one affine latent map for each retained component for the final 100,000 steps. For component c, the predictor is

$$T_c(z) = d_c + K_c(z - \bar{z}_c), \tag{41}$$

where $\bar{z}_c$ is the mean source latent assigned to the component. Each K_c is initialized from the frozen global K, and d_c is initialized to $K\bar{z}_c$ and then learned. If the current support mask cannot be assigned to a trained stable component, the predictor falls back to the frozen global map for that step.

Forecasting result and matched-route interpretation. Against the matched same-budget global-K LISTA baseline, the C_{stab}-routed local affine maps win 188/225 paired rows at $H1000$, with median staged/global ratio 0.399. This confirms that support-derived routes can select useful local affine latent dynamics. However, the matched route-baseline packet shows that this is not specific to C_{stab}: instantaneous F_{abs} support-family routing wins 189/225 rows at $H1000$, with median ratio 0.328, and latent k-means routing is also comparable. We therefore treat C_{stab} as an interpretable support-flow diagnostic rather than as the primary forecasting route.

L Post-hoc support-conditioned predictor diagnostics

This appendix records the post-hoc support-conditioned predictor experiments. These experiments are diagnostic: they ask whether learned support families can select fitted transition maps after the Koopman autoencoder has already been trained. They are not used for the main paper's basin-identifiability claim or for the main forecasting table.

Table 18: C_{stab}-routed staged local affine forecasting on the controlled multibasin benchmark. The baseline is the matched same-budget global-K LISTA model, with both models evaluated using the same periodic re-encoding grid. Lower staged/global ratios are better.

Metric	H100	H500	H1000	All horizons
Paired staged wins	189/225	188/225	188/225	176/225
Geometric staged/global MSE ratio	0.269	0.202	0.182	–
Systems won by seed-geometric ratio	14/15	14/15	14/15	14/15
Two-sided paired sign-test p	3.0×10^{-26}	1.6×10^{-25}	1.6×10^{-25}	5.3×10^{-18}

Support-conditioned rollouts. Support-conditioned prediction is a second-stage procedure applied after the Koopman autoencoder has been trained with a single global transition K. In the second stage, the encoder E_θ, decoder D_ϕ, and original K are frozen. A separate centered transition K_c is optimized for each retained F_{top8} support family c, while the original K remains the fallback for unretained or unassigned routes. The comparison point is the same frozen checkpoint's global rollout,

$$\hat{z}^{\text{global}}_{t+1} = K\hat{z}_t, \qquad \hat{x}^{\text{global}}_{t+1} = D_\phi(\hat{z}^{\text{global}}_{t+1}). \tag{42}$$

Routes are constructed without basin labels. On disjoint fitting trajectories, we encode states with the frozen encoder, compute exact S_{top8} supports, and merge them into support families by the Jaccard rule in section 3.4 with threshold 0.40. Families with fewer than 50 current-state fitting transitions are not given trainable maps. For each retained family $c \in \mathcal{C}_{\text{fit}}$, the fixed center $\bar{z}_c$ is the mean current latent assigned to that family, and the map is initialized from the checkpoint transition, $K_c \leftarrow K$. The route-local transition is

$$T_c(z) = \bar{z}_c + K_c(z - \bar{z}_c). \tag{43}$$

The reported local maps are trained jointly as one bank of matrices by gradient descent on decoded rollout MSE. For a second-stage training window $(x_{b,0}, \dots, x_{b,H})$, let $\hat{z}_{b,0} = E_\theta(x_{b,0})$, where H is the checkpoint's short training horizon. At rollout offset ℓ, the current predicted latent is detached only for the discrete route assignment:

$$c_{b,\ell} = F_{\text{top8}}(\hat{z}_{b,\ell}).$$

If this assigned family is retained, the next latent uses $T_{c_{b,\ell}}$; otherwise it falls back to the frozen global transition:

$$\tilde{z}_{b,\ell+1} = \begin{cases} T_{c_{b,\ell}}(\hat{z}_{b,\ell}), & c_{b,\ell} \in \mathcal{C}_{\text{fit}}, \\ K\hat{z}_{b,\ell}, & c_{b,\ell} \notin \mathcal{C}_{\text{fit}}. \end{cases} \tag{44}$$

The decoded prediction is $\hat{x}_{b,\ell+1} = D_\phi(\tilde{z}_{b,\ell+1})$. Every m predicted steps, with $m = 5$ in the selected runs, the next latent state is refreshed by the frozen encoder,

$$\hat{z}_{b,\ell+1} = E_\theta(\hat{x}_{b,\ell+1}),$$

and otherwise $\hat{z}_{b,\ell+1} = \tilde{z}_{b,\ell+1}$. The active setting recomputes $c_{b,\ell}$ before every one-step map application, so route switching can occur at any predicted step, not only at refresh boundaries.

The second-stage objective is

$$\min_{\{K_c:\, c\in\mathcal{C}_{\text{fit}}\}} \frac{1}{BH} \sum_{b=1}^{B} \sum_{\ell=1}^{H} \|\hat{x}_{b,\ell} - x_{b,\ell}\|_2^2 . \tag{45}$$

Table 19: Appendix support-conditioned predictor diagnostic. Entries are label-free F_{top8}-local routed/global MSE ratios and system-win counts on all states; lower ratios and higher win counts are better. Superscripts denote system-level significance tests: $*$, $p_{\text{Holm}} < 0.05$; $**$, $p_{\text{Holm}} < 0.01$.

	$H100$		$H1000$	
Model	F_{top8} ratio	system wins	F_{top8} ratio	system wins
LISTA	0.239^{*}	**13/15**	$\mathbf{1.41 \times 10^{-11}}^{**}$	**15/15**
LISTA-SB	0.021^{*}	**13/15**	$2.42 \times 10^{-7*}$	14/15
LISTA-BD	$\mathbf{0.011}^{*}$	**13/15**	$6.96 \times 10^{-5*}$	**15/15**
Sparse MLP, BD	0.032^{**}	12/15	6.53×10^{-8}	**15/15**
Sparse MLP	0.038^{**}	**13/15**	5.19×10^{-4}	**15/15**
Dense MLP	154	1/15	85.7	1/15

There is no basin-label loss, support-classification loss, latent consistency loss, or regularizer in this stage. Minibatches are route-balanced: a training step samples support-family buckets approximately uniformly and then samples windows inside the selected buckets. Thus frequent support families do not monopolize the updates, and a particular K_c receives gradients only on examples and rollout steps where it is selected.

At evaluation time no parameters are updated. Starting from a held-out initial state, the same recurrence in equation (44) is run for the forecast horizon using only predicted states. Exact supports seen in the codebook map directly to their family; unseen exact supports are assigned to the nearest family prototype only when the Jaccard similarity reaches the same threshold, and otherwise use the global fallback. Exact S_{top8}-local routing uses the same rollout structure with exact supports as routes and is retained as an ablation in section P.

M Falsification and partition controls

The partition-control experiments are falsification checks for the support-routing interpretation in table 19. They separate four questions. First, do post-hoc route-local centered affine maps $T_c(z) = \bar{z}_c + K_c(z - \bar{z}_c)$ improve over the same model's autonomous global-K rollout at all? Second, can the model-produced F_{top8} support family select those maps without basin labels or a known basin count? Third, would an oracle basin partition, a generic latent-space partition, or an arbitrary count-matched partition explain the same effect? Fourth, does improvement over the same model's autonomous global-K rollout imply that the post-hoc routed predictor is also better than the best-periodic re-encoding forecaster used in the main forecasting tables?

All rows in table 20 use the current $d_z = 256$ artifacts and freeze the trained encoder E_θ, decoder D_ϕ, and learned global transition K. The route-operator fitting split contains 256 held-out trajectories with 256 adjacent encoded transitions each. For each selector, the current latent states define route classes c; for every class with at least 128 fitting transitions, we fit a centered ridge slope K_c, with $\eta = 10^{-4}$, and roll out the affine map T_c in equations (43) and (44). Missing or underpopulated routes fall back to the same model's global K. The forecast split is disjoint from the fitting split, contains 128 held-out rollouts, and recomputes the route from the model's current predicted latent state during rollout. Thus the table changes only the route selector while holding the trained representation, local-map fitting rule, fallback rule, and global baseline fixed.

The selectors differ only in how they define the route class c_t. The support-family selector uses

$c_t = F_{\text{top8}}(z_t)$, constructed by greedy Jaccard merging of exact S_{top8} masks at threshold 0.5, and uses no basin labels or basin count. The oracle-basin selector uses benchmark basin labels on the fitting trajectories and, during rollout, assigns the decoded current prediction $D_\phi(z_t)$ to the nearest evaluation basin center used by the labeling routine. This is an evaluation-only control: it uses information that is intentionally unavailable in the training/deployment setting. The latent-cluster selector fits k-means clusters in the learned latent space with k matched to the number of occupied F_{top8} families for that checkpoint. The random selector permutes count-matched labels on the fitting transitions and routes later states by nearest random-label latent center. The reported quantity is the routed/global MSE ratio at $h = 100$, so values below one mean that the selected local maps improve over the same model's global rollout. Because this appendix table reports arithmetic means of per-system seed-IQM finite ratios, rather than the appendix routing table's system-level mean log effect, isolated catastrophic finite rollouts can dominate all-state cells; the deep-state slice is the cleaner partition audit.

The control table rules out the strongest but false interpretation of the routing result: basin labels alone are not a guaranteed route to a stable closed-loop local predictor in an arbitrary learned latent space. Oracle basin-label maps are very strong for the LISTA-style rows in this audit table, which shows that useful local centered laws exist for those learned representations. The same oracle partition is unstable for the MLP controls, including the dense no-sparsity MLP. Thus the supported conclusion is narrower and more precise: route-local centered maps can help, but the partition, the latent geometry, and the autonomous rollout dynamics must be compatible.

The non-oracle support-family rows show why F_{top8} remains a meaningful mechanism variable. On basin-interior initial states, F_{top8}-local routing gives large improvements for the LISTA-style rows in the table, while the dense no-sparsity MLP support-family route fails catastrophically. This supports the main claim that sparse encoders produce inspectable active-coordinate families that can act as label-free routing variables. The latent-cluster controls show that generic latent geometry can also support useful local maps, especially for sparse MLP controls, so the result should not be read as "only support families can ever route." The random count-matched controls can give modest gains in some rows, but they do not reproduce the strongest deep-state support-family or latent-cluster effects and they fail for the dense MLP. The take-away is therefore comparative rather than absolute: F_{top8} is a label-free, interpretable selector produced by the sparse representation, whereas oracle basins, k-means clusters, and random matched partitions are diagnostic controls rather than deployment-time support objects.

A separate routed-forecasting add-on compared the same post-hoc F_{top8} family-local centered predictor against the same model's best-periodic paper-table forecasts. The fully autonomous routed version was negative for absolute forecasting superiority: it produced complete merged outputs for the retained controlled multibasin benchmark and Dysts benchmark with 0 failures, but for every $d_z = 256$ LISTA-family model and evaluated horizon, routed rows won 0/15 controlled systems and 0/10 Dysts systems against the same-model best-periodic baseline. A follow-up that periodically decoded and re-encoded the routed rollout every 5, 10, 20, or 30 steps also completed with 0 failures. Periodic refresh reduced many autonomous blow-ups and made route coverage high on the controlled systems, but it did not reverse the forecasting conclusion: every aggregate routed/best-periodic ratio remained above one, with only isolated system wins in individual cells. This negative result is important because the appendix routing table compares routed rollouts to the same model's autonomous global-K rollout, whereas the paper-table forecasting baseline is stronger because it periodically decodes and re-encodes. We therefore use support-family routing as appendix mechanism evidence that F_{top8} can select useful local predictors relative to the same model's global latent rollout, not as evidence that the post-hoc routed predictor replaces best-periodic forecasting in the aggregate forecasting tables.

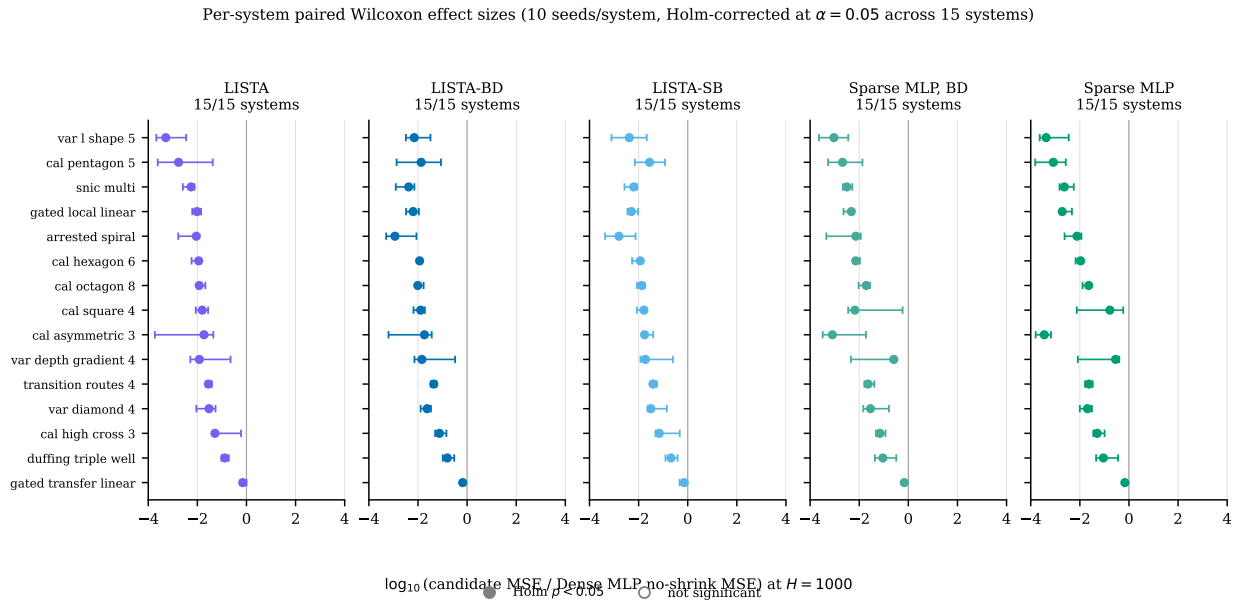


Figure 14: Per-system effect sizes at H=1000 versus the Dense MLP baseline. Each point is one benchmark system; the x-coordinate is the per-system median paired $\log_{10}$-MSE difference (candidate minus baseline) over the common completed seeds, up to 15 per system, with whiskers showing the 95% bootstrap interval. Filled markers clear Holm-corrected α=0.05. Numerical values are in table 23.

N Full per-system multibasin tables

Tables 21–23 render the per-(system, candidate) effect sizes and Holm-corrected Wilcoxon p-values that underlie the multibasin K/N forecasting counts in table 1 and the forest plot in figure 14. Each cell gives the per-system median paired $\log_{10}$-MSE difference (candidate minus matched-sampling Dense MLP, no-shrink baseline) over the common completed seeds, up to 15 per system, and the corresponding Holm-corrected one-sided paired Wilcoxon p-value. Bold entries indicate Holm-corrected $p < 0.05$ in favor of the candidate.

Per-system audit of the interpretability columns

Why the wrong-support denominator is 11 and not 15. The wrong-support ablation builds, for each system, a dictionary mapping each basin to its canonical support mask, defined as the most-common exact support among the deep-slice states of that basin. The ablated rollout draws a mask from a basin different from the state's own basin and holds that wrong mask fixed, so the test requires at least two distinct deep-slice canonical masks. The main support-diagnostic table uses the per-basin deep slice, which keeps all represented basins in the diagnostic population. Earlier global top-quartile slices can have smaller denominators when a system's deepest states concentrate in a single basin; the corresponding full per-system support-intervention tables remain useful as robustness diagnostics.

Tables 24–27 render the full per-(system, candidate) breakdown of the retained active-count diagnostic and the three directional interpretability columns of table 2. The alternative direction is "candidate < baseline" for $|S_{\text{abs}}|$ and $H(B \mid F_{\text{abs}})$, and "candidate > baseline" for the wrong-support ablation ratios. The main-text family-count column is intentionally not included in these one-sided

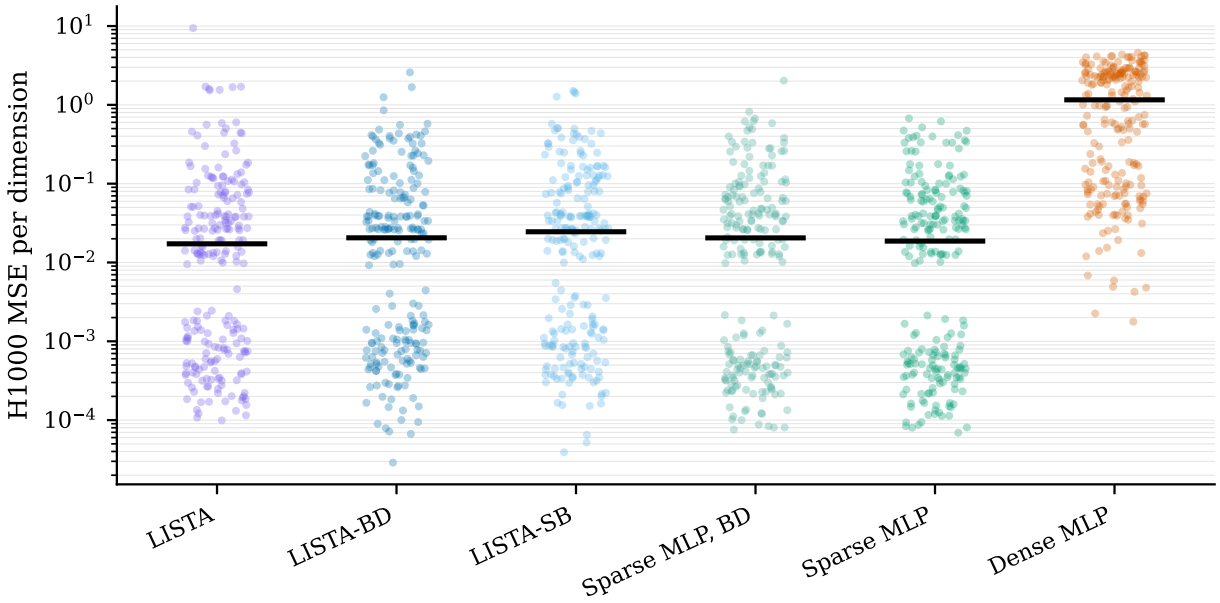


Figure 15: Per-(system, seed) forecasting MSE at $H100$, $H500$, and $H1000$ on the multibasin benchmark. Black bars mark the mean across per-system seed-IQM values. The Dense MLP has a much heavier upper tail at every horizon.

tests because $|F_{\text{abs}}|$ is a non-directional count.

O Per-seed distributions

Tables 1 and 2 report arithmetic means across retained systems after per-system seed-IQM summaries. The corresponding per-(system, seed) distributions are shown below. Figure 15 shows the paper-facing strip plot for the matched boundary-emphasized rows. Figure 16 plots per-(system, seed) histograms of forecasting MSE at every paper-facing horizon. Figure 17 plots per-(system, seed) histograms of $H(B \mid S_{\text{abs}})$, $H(S_{\text{abs}} \mid B)$, and exact-support uniqueness on the deep-state slice. Figure 18 plots the same per-seed distributions for the Dysts $dt{\times}30$ benchmark. The histograms show retained-system and seed-level dispersion around the paper-facing means.

P Support-routing robustness

Table 28 reports appendix route-form diagnostics. The deep slice removes boundary-adjacent states, so it is useful as a robustness check rather than the primary deployment setting. The exact S_{top8} columns should be read as ablations of the support-family router rather than as the headline routing object.

For the gated exact-support diagnostic, let $m_c \in \{0,1\}^{d_z}$ be the eight-coordinate mask of the exact route $c = S_{\text{top8}}(z_t)$, and let $\bar{z}_c$ be the corresponding fitting-set mean. The gated rule masks the centered latent state before applying the learned global transition:

$$\hat{z}_{t+1} = \bar{z}_c + K\big(m_c \odot (z_t - \bar{z}_c)\big). \tag{46}$$

This diagnostic asks whether the active coordinates select useful columns or subspaces of the learned

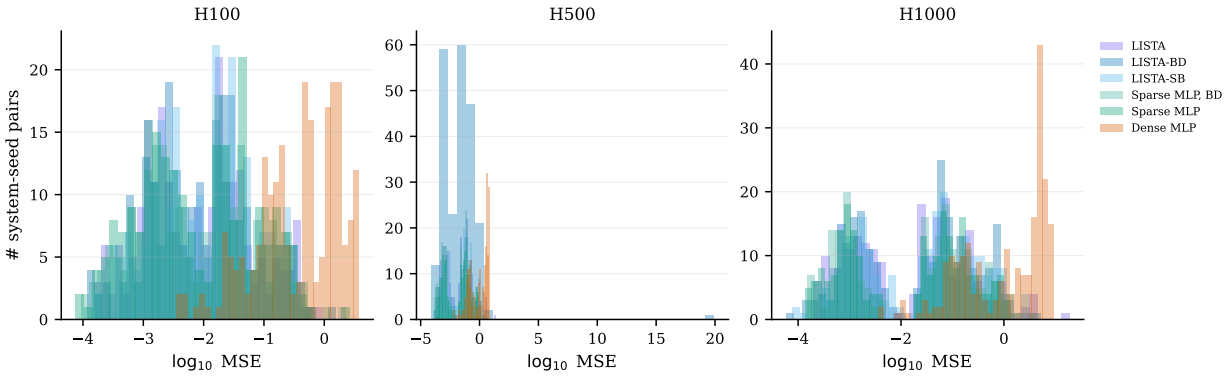


Figure 16: Per-(system, seed) forecasting MSE distributions on the multibasin benchmark. Each row is a horizon; histograms are over the $\log_{10}$ raw MSE.

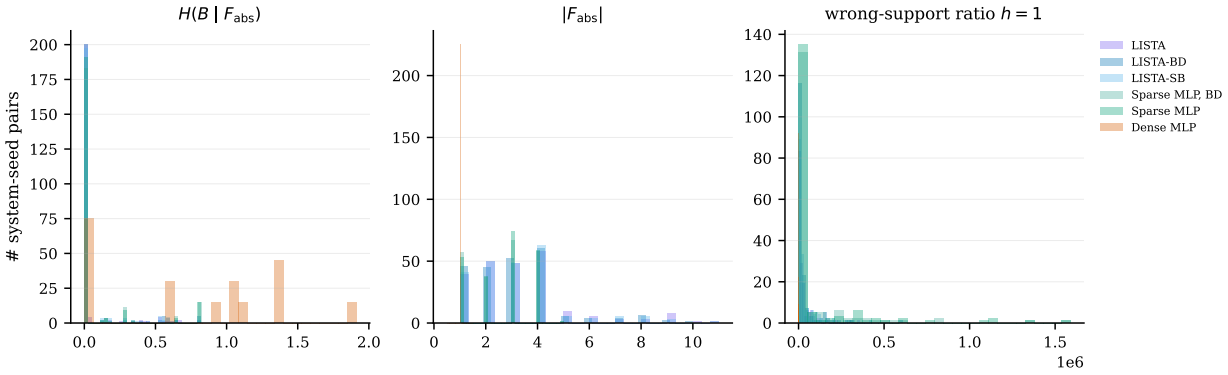


Figure 17: Per-(system, seed) support–basin diagnostic distributions on the deep-state slice. Histograms show $H(B \mid S_{\text{abs}})$, $H(S_{\text{abs}} \mid B)$, and exact-support uniqueness for the rows summarized in table 2.

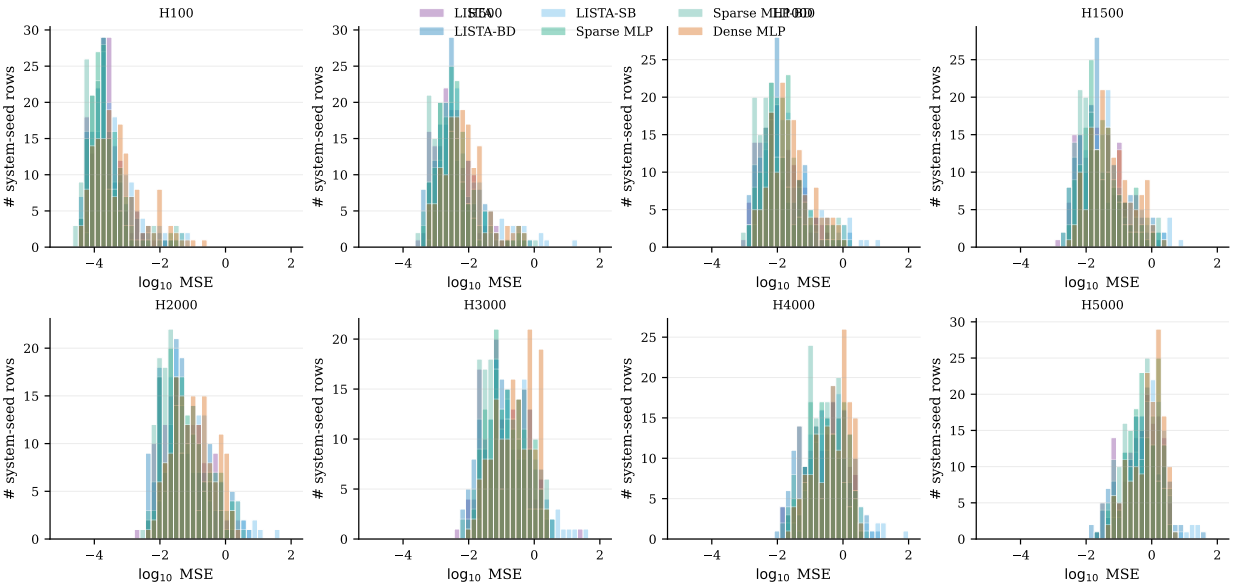


Figure 18: Per-(system, seed) forecasting MSE distributions on the Dysts $dt\times30$ benchmark.

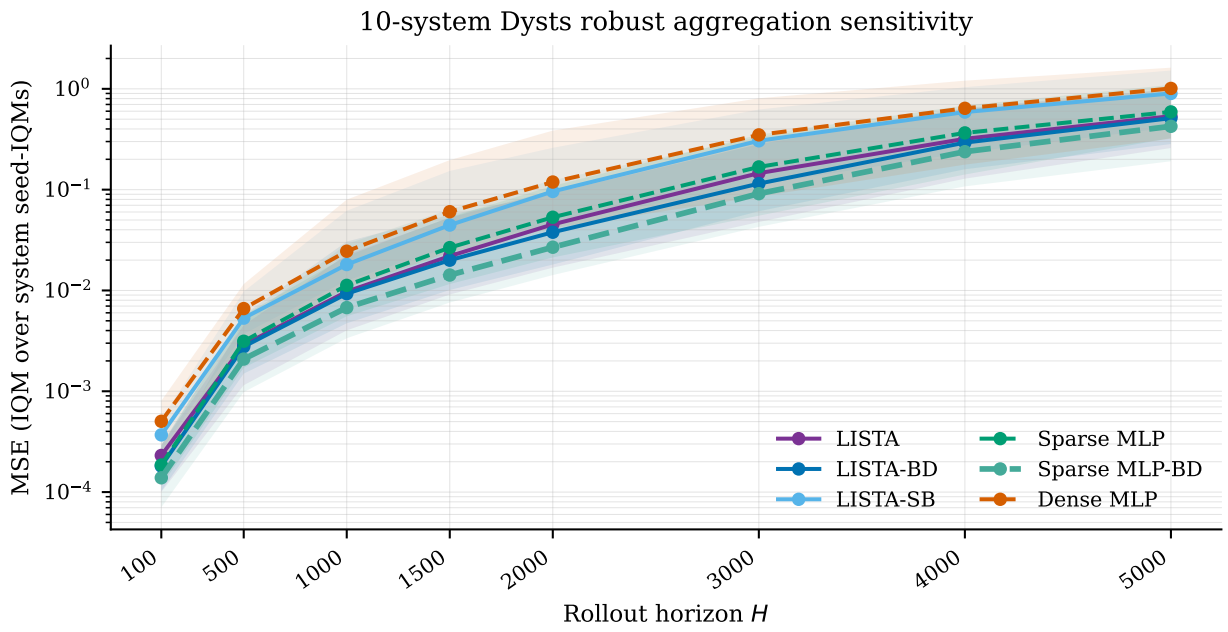


Figure 19: Dysts $dt\times30$ IQM-over-IQM horizon sensitivity. Lines show IQM across retained systems after first summarizing seeds within each system by IQM. Shaded bands show the inter-system IQR of the per-system seed-IQM MSE values, not a confidence interval. With the repaired block-diagonal Sparse MLP run, Sparse MLP-BD is the best robust aggregate at every displayed horizon.

transition. The family-local route in section L asks a different question: whether the same support family can key a post-hoc local linear map.

Q Dysts forecasting diagnostics

The per-(system, seed) Dysts MSE values are released in the collected `forecasting_rows.csv` artifacts. Table 1 reports arithmetic means across systems after first summarizing seeds within each system by IQM. The $dt\times30$ packet is fully finite in median coverage at every reported horizon, but the per-seed distributions remain heavy-tailed on several systems, so the main table and figures 1b and 18 should be read together: the table gives actual-MSE point estimates and paired system-level exact sign-test markers, while the figures show horizon-scale uncertainty and per-seed dispersion. The soft-block LISTA diagnostic improves on Dense MLP in aggregate but does not match the primary sparse forecasting rows and does not survive the main Dysts significance correction. The older tiny-step $H\leq60000$ packet is useful as a stress-test diagnostic because it exposed divergent block-diagonal LISTA rollouts, but it is not pooled with the coarser-step table.

As a robustness check on the cross-system aggregation, table 29 and figure 19 replace the final arithmetic mean across systems with an IQM across the retained systems, after the same within-system seed IQM used by the main table. This IQM-over-IQM view asks for the typical retained-system performance and trims both unusually easy and unusually hard chaotic-flow systems. With the repaired block-diagonal Sparse MLP run, Sparse MLP-BD has the lowest MSE at every displayed horizon from $H100$ through $H5000$. This should be read as an aggregation-sensitivity result for the external forecasting benchmark, not as a replacement for the average-case Dysts table in the main text.

R Additional background

R.1 Koopman operators and finite-dimensional approximations

Consider the stored-step map used in section 2,

$$x_{k+1} = F_{\Delta t}(x_k), \qquad x_k \in \mathcal{X} \subseteq \mathbb{R}^{d_x}. \tag{47}$$

Koopman operator theory studies the evolution of observables rather than the evolution of the state directly. An observable is a measurement function $g : \mathcal{X} \to \mathbb{R}$, and the Koopman operator $\mathcal{K}$ acts by composition:

$$(\mathcal{K}g)(x) = g(F_{\Delta t}(x)). \tag{48}$$

The map $F_{\Delta t}$ may be nonlinear, but $\mathcal{K}$ is linear on the function space of observables. If a finite vector of observables

$$\Phi(x) = \left[g_1(x), \ldots, g_{d_z}(x)\right]^\top \tag{49}$$

spans a Koopman-invariant subspace, then there is a matrix $K \in \mathbb{R}^{d_z \times d_z}$ such that

$$\Phi(x_{k+1}) = \Phi(F_{\Delta t}(x_k)) = \mathcal{K}\Phi(x_k) = K\Phi(x_k). \tag{50}$$

In that case the nonlinear dynamics are exactly linear in the observable coordinates $z_k = \Phi(x_k)$. Learning Koopman models can therefore be viewed as searching for observables whose span is invariant enough to support useful finite-dimensional linear prediction.

In practice, the infinite-dimensional operator is approximated from data. Given snapshot pairs $\{(x_k, x_{k+1})\}$ and a chosen feature map Φ, dynamic mode decomposition and extended dynamic mode decomposition (eDMD) estimate a finite transition by least squares (Schmid, 2010; Rowley et al., 2009; Tu et al., 2014; Williams et al., 2015, 2016):

$$K = \underset{\widetilde{K} \in \mathbb{R}^{d_z \times d_z}}{\arg\min} \sum_k \left\| \Phi(x_{k+1}) - \widetilde{K}\Phi(x_k) \right\|_2^2. \tag{51}$$

DMD is the special case $\Phi(x) = x$, so it fits a linear state-space surrogate without a learned nonlinear lift. eDMD extends this by choosing a richer observable dictionary. Koopman autoencoders replace the fixed observable dictionary with a learned encoder and decoder, which is the setting used in this paper.

R.2 Why multibasin systems require local structure

The theoretical obstruction in multibasin systems is not the absence of local linear descriptions. Near appropriate attracting sets, classical linearization and Koopman eigenfunction constructions can produce useful local or basin-restricted coordinates. The obstruction is instead global: as stated in the main text, multi-attractor systems should not generally be expected to admit a single finite-dimensional global linearization with a continuous encoder-decoder pair (Lan and Mezić, 2013; Brunton et al., 2016; Pan and Duraisamy, 2024; Liu et al., 2025). A single continuous, exact, state-reconstructing finite-dimensional Koopman representation is therefore compatible only with restricted global attractor structure.

For the present paper, this motivates a cautious formulation. A learned finite-dimensional Koopman representation on a multibasin system should not be expected to be one exact global linearization that reconstructs every basin simultaneously. It must either approximate across incompatible regions, or it must contain some implicit information about which local description is active. Sparse supports are useful because they make one such piece of information discrete and inspectable: the support can identify a basin, basin part, or local predictive chart while the coefficient values carry within-chart state information.

R.3 Koopman autoencoders and periodic re-encoding

A standard Koopman autoencoder learns an encoder $E_\theta : \mathbb{R}^{d_x} \to \mathbb{R}^{d_z}$, decoder $D_\phi : \mathbb{R}^{d_z} \to \mathbb{R}^{d_x}$, and latent transition K so that

$$z_k = E_\theta(x_k), \qquad z_{k+1} \approx K z_k, \qquad x_k \approx D_\phi(z_k). \tag{52}$$

A schematic one-step objective has the form

$$\min_{E_\theta, D_\phi, K} \sum_k \|x_k - D_\phi(E_\theta(x_k))\|_2^2 + \lambda_{\text{lin}} \|E_\theta(x_{k+1}) - K E_\theta(x_k)\|_2^2 . \tag{53}$$

This objective is useful as a reference point, but it is not sufficient for the present study. It does not directly train decoded multi-step rollouts, does not impose a sparse support object, and does not test whether the support itself selects a useful local predictive rule. The training objective in section 3.3 therefore keeps the autoencoding and latent-linearity terms but adds decoded rollout prediction over windows, sparsity on the rolled latent states, and optional transition-structure penalties.

Periodic re-encoding interrupts a latent rollout after a fixed number of stored observation steps. For period m,

$$\tilde{z}_{k+m} = K^m z_k, \qquad \tilde{x}_{k+m} = D_\phi(\tilde{z}_{k+m}), \qquad z_{k+m} = E_\theta(\tilde{x}_{k+m}). \tag{54}$$

This is the same mechanism defined in equation (12). It uses only the model's own predicted state and is therefore not a teacher-forced correction. The point is to refresh the latent embedding and, in sparse models, to refresh the support as the predicted state moves through regions where a different local linear description may be more appropriate (Fathi et al., 2024).

R.4 Sparse coding and LISTA

Classical sparse coding seeks a code z that reconstructs a signal from a small set of active dictionary atoms:

$$z^\star(x) = \arg\min_{z \in \mathbb{R}^{d_z}} \frac{1}{2} \|x - Dz\|_2^2 + \lambda_{\text{sc}} \|z\|_1 , \qquad \lambda_{\text{sc}} > 0, \tag{55}$$

where D is a dictionary and λ_{sc} controls sparsity (Olshausen and Field, 1996; Chen et al., 1998; Donoho and Elad, 2003). For a fixed dictionary, and under the usual uniqueness and non-boundary conditions for the Lasso solution, the active set and signs are locally constant over regions of the input space, while the coefficient values vary continuously within a fixed active set. The decoder reconstruction then lies in the span of the selected dictionary columns, giving a union-of-subspaces geometry. Each exact support can index a local chart, and the greedy Jaccard procedure in section 3.4 groups nearby Boolean support vectors into support families when threshold noise or boundary effects fragment the exact masks.

LISTA replaces an iterative sparse-coding solver with a learned finite-depth shrinkage network (Gregor and LeCun, 2010). In the notation of equation (6), the encoder forms a pre-code from the input and repeatedly applies learned affine refinement followed by soft thresholding. In this paper, the encoder is trained jointly with the Koopman rollout objective rather than solving the sparse-coding problem at evaluation time. The thresholding operation is the important ingredient: it turns the latent code into both continuous coefficients and an explicit support. The main experiments then ask whether F_{abs} groups absolute-threshold supports S_{abs} in a way that aligns with withheld benchmark basin labels, whether S_{abs} is functionally used by the predictor, and whether refreshed S_{top8} supports track controlled basin transfer. Appendix diagnostics additionally test post-hoc support-routed predictors.

Table 20: Partition-control local forecasting for the top-8 support-family comparison. Entries are routed/global MSE ratios, summarized by seed-IQM within each system and arithmetic mean across systems; lower is better. Bracketed cells give within-system Wilcoxon/Holm counts when present. The LISTA-SB row uses the matched p256 (d_z=256) control artifact.

		$H100$	
Model	Selector	All	Deep
LISTA-SB	Oracle basin labels	0.0176	0.00679
	Support family	0.168	0.0374
	Latent clusters	0.179	0.0471
	Random matched	0.857	0.869
LISTA-BD	Oracle basin labels	0.0330 [15/15]	0.0306 [14/15]
	Support family	2.18 [11/15]	0.135 [12/15]
	Latent clusters	2.83 [11/15]	0.368 [12/15]
	Random matched	0.861 [13/15]	0.887 [12/15]
Sparse MLP, BD	Oracle basin labels	2.57×10^{12} [8/15]	2.60×10^{12} [7/15]
	Support family	1.20 [8/15]	0.779 [8/15]
	Latent clusters	0.241 [13/15]	0.230 [13/15]
	Random matched	0.929 [14/15]	0.961 [8/15]
Sparse MLP	Oracle basin labels	6.00×10^{9} [11/15]	1.19×10^{10} [10/15]
	Support family	0.639 [6/15]	0.443 [5/15]
	Latent clusters	0.279 [14/15]	0.221 [13/15]
	Random matched	0.925 [12/15]	0.956 [11/15]
Dense MLP	Oracle basin labels	1.75×10^{33} [4/15]	1.36×10^{33} [4/15]
	Support family	3.32×10^{3} [0/15]	2.61×10^{3} [0/15]
	Latent clusters	5.73×10^{9} [0/15]	9.57×10^{8} [2/15]
	Random matched	179 [0/15]	322 [0/15]

Table 21: Per-system effect sizes and Holm-corrected Wilcoxon p-values at $H = 100$ on the 15-system multibasin benchmark, against the matched-sampling Dense MLP, no-shrink baseline. Δ is the per-system median paired $\log_{10}$-MSE difference (candidate − baseline); p_{H} is the one-sided paired Wilcoxon signed-rank p-value, Holm-corrected across retained systems within each candidate column. Bold Δ: Holm-corrected $p < 0.05$ in favor of the candidate.

System	LISTA		LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}
var_l_shape_5	**−2.23**	$<10^{-3}$	**−1.94**	$<10^{-3}$	**−1.90**	$<10^{-3}$	**−2.05**	$<10^{-3}$	**−2.23**	$<10^{-3}$
cal_pentagon_5	**−1.86**	$<10^{-3}$	**−1.58**	$<10^{-3}$	**−1.20**	$<10^{-3}$	**−1.89**	$<10^{-3}$	**−2.16**	$<10^{-3}$
snic_multi	**−2.01**	$<10^{-3}$	**−2.03**	$<10^{-3}$	**−1.91**	$<10^{-3}$	**−2.09**	$<10^{-3}$	**−2.13**	$<10^{-3}$
gated_local_linear	**−2.16**	$<10^{-3}$	**−2.19**	$<10^{-3}$	**−2.20**	$<10^{-3}$	**−2.35**	$<10^{-3}$	**−2.52**	$<10^{-3}$
arrested_spiral	**−1.85**	$<10^{-3}$	**−2.09**	$<10^{-3}$	**−2.03**	$<10^{-3}$	**−1.93**	$<10^{-3}$	**−1.90**	$<10^{-3}$
cal_hexagon_6	**−1.76**	$<10^{-3}$	**−1.73**	$<10^{-3}$	**−1.85**	$<10^{-3}$	**−1.96**	$<10^{-3}$	**−1.83**	$<10^{-3}$
cal_octagon_8	**−1.85**	$<10^{-3}$	**−1.90**	$<10^{-3}$	**−1.80**	$<10^{-3}$	**−1.68**	$<10^{-3}$	**−1.63**	$<10^{-3}$
cal_square_4	**−1.44**	$<10^{-3}$	**−1.62**	$<10^{-3}$	**−1.58**	$<10^{-3}$	**−1.39**	$<10^{-3}$	**−1.02**	$<10^{-3}$
cal_asymmetric_3	**−2.00**	$<10^{-3}$	**−1.92**	$<10^{-3}$	**−1.75**	$<10^{-3}$	**−2.00**	$<10^{-3}$	**−2.44**	$<10^{-3}$
var_depth_gradient_4	**−1.39**	$<10^{-3}$	**−1.33**	$<10^{-3}$	**−1.37**	$<10^{-3}$	**−1.05**	$<10^{-3}$	**−0.82**	$<10^{-3}$
transition_routes_4	**−2.21**	$<10^{-3}$	**−1.77**	$<10^{-3}$	**−1.90**	$<10^{-3}$	**−2.36**	$<10^{-3}$	**−2.47**	$<10^{-3}$
var_diamond_4	**−2.00**	$<10^{-3}$	**−1.80**	$<10^{-3}$	**−1.84**	$<10^{-3}$	**−1.46**	$<10^{-3}$	**−1.67**	$<10^{-3}$
cal_high_cross_3	**−1.05**	$<10^{-3}$	**−1.12**	$<10^{-3}$	**−1.19**	$<10^{-3}$	**−1.10**	$<10^{-3}$	**−1.17**	$<10^{-3}$
duffing_triple_well	**−0.54**	$<10^{-3}$	**−0.51**	0.008	**−0.35**	0.007	**−0.81**	0.002	**−0.78**	$<10^{-3}$
gated_transfer_linear	**−0.27**	0.001	**−0.21**	0.008	**−0.31**	0.007	**−0.36**	$<10^{-3}$	**−0.27**	0.004
K/15 *Holm* $p < 0.05$	K=15 / N=15		K=15 / N=15		K=15 / N=15		K=15 / N=15		K=15 / N=15	

Table 22: Per-system effect sizes and Holm-corrected Wilcoxon p-values at $H = 500$ on the 15-system multibasin benchmark. Format identical to table 21.

System	LISTA		LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}
var_l_shape_5	**−2.98**	$<10^{-3}$	**−2.04**	$<10^{-3}$	**−2.03**	$<10^{-3}$	**−2.73**	$<10^{-3}$	**−2.96**	$<10^{-3}$
cal_pentagon_5	**−2.48**	$<10^{-3}$	**−2.10**	$<10^{-3}$	**−1.61**	$<10^{-3}$	**−2.42**	$<10^{-3}$	**−2.77**	$<10^{-3}$
snic_multi	**−2.30**	$<10^{-3}$	**−2.45**	$<10^{-3}$	**−2.31**	$<10^{-3}$	**−2.44**	$<10^{-3}$	**−2.49**	$<10^{-3}$
gated_local_linear	**−2.01**	$<10^{-3}$	**−2.24**	$<10^{-3}$	**−2.27**	$<10^{-3}$	**−2.29**	$<10^{-3}$	**−2.52**	$<10^{-3}$
arrested_spiral	**−2.26**	$<10^{-3}$	**−2.57**	$<10^{-3}$	**−2.57**	$<10^{-3}$	**−2.40**	$<10^{-3}$	**−2.33**	$<10^{-3}$
cal_hexagon_6	**−2.01**	$<10^{-3}$	**−2.01**	$<10^{-3}$	**−2.18**	$<10^{-3}$	**−2.32**	$<10^{-3}$	**−2.16**	$<10^{-3}$
cal_octagon_8	**−1.94**	$<10^{-3}$	**−2.09**	$<10^{-3}$	**−1.91**	$<10^{-3}$	**−1.81**	$<10^{-3}$	**−1.74**	$<10^{-3}$
cal_square_4	**−1.60**	$<10^{-3}$	**−1.89**	$<10^{-3}$	**−1.81**	$<10^{-3}$	**−1.67**	$<10^{-3}$	**−1.17**	$<10^{-3}$
cal_asymmetric_3	**−2.27**	$<10^{-3}$	**−2.12**	$<10^{-3}$	**−1.94**	$<10^{-3}$	**−2.53**	$<10^{-3}$	**−3.15**	$<10^{-3}$
var_depth_gradient_4	**−1.63**	$<10^{-3}$	**−1.45**	$<10^{-3}$	**−1.47**	$<10^{-3}$	**−1.21**	$<10^{-3}$	**−0.92**	$<10^{-3}$
transition_routes_4	**−1.73**	$<10^{-3}$	**−1.52**	$<10^{-3}$	**−1.49**	$<10^{-3}$	**−1.68**	$<10^{-3}$	**−1.86**	$<10^{-3}$
var_diamond_4	**−1.77**	$<10^{-3}$	**−1.58**	$<10^{-3}$	**−1.40**	$<10^{-3}$	**−1.41**	$<10^{-3}$	**−1.75**	$<10^{-3}$
cal_high_cross_3	**−0.93**	$<10^{-3}$	**−0.97**	$<10^{-3}$	**−1.03**	$<10^{-3}$	**−1.17**	$<10^{-3}$	**−1.26**	$<10^{-3}$
duffing_triple_well	**−0.83**	$<10^{-3}$	**−0.81**	$<10^{-3}$	**−0.65**	$<10^{-3}$	**−1.07**	$<10^{-3}$	**−1.05**	$<10^{-3}$
gated_transfer_linear	**−0.08**	0.013	**+1.19**	0.008	**−0.21**	$<10^{-3}$	**−0.18**	$<10^{-3}$	**−0.18**	$<10^{-3}$
K/15 *Holm* $p < 0.05$	K=15 / N=15		K=15 / N=15		K=15 / N=15		K=15 / N=15		K=15 / N=15	

Table 23: Per-system effect sizes and Holm-corrected Wilcoxon p-values at $H = 1000$ on the 15-system multibasin benchmark. This is the numerical version of the forest plot in figure 14. Format identical to table 21.

System	LISTA		LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}
var_l_shape_5	**−3.29**	$<10^{-3}$	**−2.15**	$<10^{-3}$	**−2.38**	$<10^{-3}$	**−3.03**	$<10^{-3}$	**−3.37**	$<10^{-3}$
cal_pentagon_5	**−2.76**	$<10^{-3}$	**−1.87**	$<10^{-3}$	**−1.55**	$<10^{-3}$	**−2.68**	$<10^{-3}$	**−3.08**	$<10^{-3}$
snic_multi	**−2.26**	$<10^{-3}$	**−2.38**	$<10^{-3}$	**−2.20**	$<10^{-3}$	**−2.50**	$<10^{-3}$	**−2.63**	$<10^{-3}$
gated_local_linear	**−2.01**	$<10^{-3}$	**−2.20**	$<10^{-3}$	**−2.29**	$<10^{-3}$	**−2.32**	$<10^{-3}$	**−2.72**	$<10^{-3}$
arrested_spiral	**−2.04**	$<10^{-3}$	**−2.94**	$<10^{-3}$	**−2.80**	$<10^{-3}$	**−2.14**	$<10^{-3}$	**−2.12**	$<10^{-3}$
cal_hexagon_6	**−1.95**	$<10^{-3}$	**−1.93**	$<10^{-3}$	**−1.93**	$<10^{-3}$	**−2.14**	$<10^{-3}$	**−1.97**	$<10^{-3}$
cal_octagon_8	**−1.92**	$<10^{-3}$	**−2.01**	$<10^{-3}$	**−1.87**	$<10^{-3}$	**−1.72**	$<10^{-3}$	**−1.64**	$<10^{-3}$
cal_square_4	**−1.80**	$<10^{-3}$	**−1.89**	$<10^{-3}$	**−1.78**	$<10^{-3}$	**−2.18**	$<10^{-3}$	**−0.78**	$<10^{-3}$
cal_asymmetric_3	**−1.73**	$<10^{-3}$	**−1.74**	$<10^{-3}$	**−1.76**	$<10^{-3}$	**−3.09**	$<10^{-3}$	**−3.45**	$<10^{-3}$
var_depth_gradient_4	**−1.92**	$<10^{-3}$	**−1.84**	$<10^{-3}$	**−1.72**	$<10^{-3}$	**−0.59**	$<10^{-3}$	**−0.54**	$<10^{-3}$
transition_routes_4	**−1.55**	$<10^{-3}$	**−1.36**	$<10^{-3}$	**−1.42**	$<10^{-3}$	**−1.64**	$<10^{-3}$	**−1.63**	$<10^{-3}$
var_diamond_4	**−1.52**	$<10^{-3}$	**−1.63**	$<10^{-3}$	**−1.50**	$<10^{-3}$	**−1.54**	$<10^{-3}$	**−1.69**	$<10^{-3}$
cal_high_cross_3	**−1.28**	$<10^{-3}$	**−1.13**	$<10^{-3}$	**−1.16**	$<10^{-3}$	**−1.16**	$<10^{-3}$	**−1.29**	$<10^{-3}$
duffing_triple_well	**−0.88**	$<10^{-3}$	**−0.80**	$<10^{-3}$	**−0.68**	$<10^{-3}$	**−1.04**	$<10^{-3}$	**−1.04**	$<10^{-3}$
gated_transfer_linear	**−0.15**	0.013	**−0.18**	$<10^{-3}$	**−0.14**	$<10^{-3}$	**−0.16**	$<10^{-3}$	**−0.17**	$<10^{-3}$
$K/15$ *Holm* $p < 0.05$	K=15 / N=15		K=15 / N=15		K=15 / N=15		K=15 / N=15		K=15 / N=15	

Table 24: Per-system effect sizes and Holm-corrected Wilcoxon p-values for $|S_{\mathrm{abs}}|$, the mean active-coordinate count under the absolute-threshold support on the deep slice. Alternative: candidate $|S_{\mathrm{abs}}|$ < baseline $|S_{\mathrm{abs}}|$.

System	LISTA		LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}
snic_multi	**−175.52**	$<10^{-3}$	**−174.21**	$<10^{-3}$	**−171.07**	$<10^{-3}$	**−170.02**	$<10^{-3}$	**−169.93**	$<10^{-3}$
gated_local_linear	**−168.98**	$<10^{-3}$	**−167.67**	$<10^{-3}$	**−167.29**	$<10^{-3}$	**−159.44**	$<10^{-3}$	**−159.44**	$<10^{-3}$
duffing_triple_well	**−163.57**	$<10^{-3}$	**−163.31**	$<10^{-3}$	**−163.67**	$<10^{-3}$	**−148.02**	$<10^{-3}$	**−148.80**	$<10^{-3}$
transition_routes_4	**−167.29**	$<10^{-3}$	**−162.38**	$<10^{-3}$	**−163.99**	$<10^{-3}$	**−152.85**	$<10^{-3}$	**−152.73**	$<10^{-3}$
cal_asymmetric_3	**−161.77**	$<10^{-3}$	**−158.00**	$<10^{-3}$	**−156.12**	$<10^{-3}$	**−163.00**	$<10^{-3}$	**−162.97**	$<10^{-3}$
cal_pentagon_5	**−162.15**	$<10^{-3}$	**−156.55**	$<10^{-3}$	**−157.12**	$<10^{-3}$	**−141.89**	$<10^{-3}$	**−142.43**	$<10^{-3}$
gated_transfer_linear	**−155.81**	$<10^{-3}$	**−152.76**	$<10^{-3}$	**−155.40**	$<10^{-3}$	**−158.88**	$<10^{-3}$	**−158.43**	$<10^{-3}$
cal_hexagon_6	**−159.19**	$<10^{-3}$	**−155.01**	$<10^{-3}$	**−156.11**	$<10^{-3}$	**−142.86**	$<10^{-3}$	**−142.86**	$<10^{-3}$
arrested_spiral	**−156.00**	$<10^{-3}$	**−151.91**	$<10^{-3}$	**−153.06**	$<10^{-3}$	**−154.19**	$<10^{-3}$	**−154.19**	$<10^{-3}$
cal_square_4	**−153.23**	$<10^{-3}$	**−152.11**	$<10^{-3}$	**−153.07**	$<10^{-3}$	**−140.53**	$<10^{-3}$	**−140.78**	$<10^{-3}$
var_l_shape_5	**−157.68**	$<10^{-3}$	**−151.90**	$<10^{-3}$	**−155.03**	$<10^{-3}$	**−142.74**	$<10^{-3}$	**−142.74**	$<10^{-3}$
var_diamond_4	**−160.07**	$<10^{-3}$	**−144.83**	$<10^{-3}$	**−157.41**	$<10^{-3}$	**−149.83**	$<10^{-3}$	**−149.50**	$<10^{-3}$
var_depth_gradient_4	**−154.59**	$<10^{-3}$	**−149.53**	$<10^{-3}$	**−152.15**	$<10^{-3}$	**−140.31**	$<10^{-3}$	**−140.15**	$<10^{-3}$
cal_octagon_8	**−152.81**	$<10^{-3}$	**−147.55**	$<10^{-3}$	**−152.64**	$<10^{-3}$	**−143.20**	$<10^{-3}$	**−143.34**	$<10^{-3}$
cal_high_cross_3	**−148.93**	$<10^{-3}$	**−143.99**	$<10^{-3}$	**−146.03**	$<10^{-3}$	**−127.04**	$<10^{-3}$	**−133.00**	$<10^{-3}$
$K/15$ *Holm* $p < 0.05$	K=15 / N=15		K=15 / N=15		K=15 / N=15		K=15 / N=15		K=15 / N=15	

Table 25: Per-system effect sizes and Holm-corrected Wilcoxon p-values for $H(B \mid F_{\text{abs}})$ on the deep slice. Alternative: candidate $H(B \mid F_{\text{abs}}) <$ baseline.

System	LISTA		LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}
transition_routes_4	**−1.38**	$<10^{-3}$	**−1.38**	$<10^{-3}$	**−1.38**	$<10^{-3}$	**−1.38**	$<10^{-3}$	**−1.38**	$<10^{-3}$
cal_square_4	**−1.37**	$<10^{-3}$	**−1.37**	$<10^{-3}$	**−1.37**	$<10^{-3}$	**−1.37**	$<10^{-3}$	**−1.37**	$<10^{-3}$
var_diamond_4	**−1.37**	$<10^{-3}$	**−1.37**	$<10^{-3}$	**−1.37**	$<10^{-3}$	**−1.37**	$<10^{-3}$	**−1.37**	$<10^{-3}$
cal_octagon_8	**−1.37**	$<10^{-3}$	**−1.33**	$<10^{-3}$	**−1.36**	$<10^{-3}$	**−1.09**	0.001	**−1.09**	$<10^{-3}$
snic_multi	**−1.09**	$<10^{-3}$	**−1.09**	$<10^{-3}$	**−1.09**	$<10^{-3}$	**−1.09**	$<10^{-3}$	**−1.09**	$<10^{-3}$
gated_local_linear	**−1.07**	$<10^{-3}$	**−1.07**	$<10^{-3}$	**−1.07**	$<10^{-3}$	**−1.07**	$<10^{-3}$	**−1.07**	$<10^{-3}$
gated_transfer_linear	**−1.06**	$<10^{-3}$	**−1.06**	$<10^{-3}$	**−1.06**	$<10^{-3}$	**−1.06**	$<10^{-3}$	**−1.06**	$<10^{-3}$
cal_hexagon_6	**−0.92**	$<10^{-3}$	**−0.92**	$<10^{-3}$	**−0.82**	$<10^{-3}$	**−0.66**	0.001	**−0.66**	$<10^{-3}$
cal_pentagon_5	**−0.64**	$<10^{-3}$	**−0.64**	$<10^{-3}$	**−0.64**	$<10^{-3}$	**−0.64**	0.002	**−0.64**	$<10^{-3}$
var_depth_gradient_4	**−0.62**	$<10^{-3}$	**−0.62**	$<10^{-3}$	**−0.62**	$<10^{-3}$	**−0.62**	$<10^{-3}$	**−0.62**	$<10^{-3}$
cal_high_cross_3	**−0.01**	$<10^{-3}$	**−0.01**	$<10^{-3}$	**−0.01**	$<10^{-3}$	**−0.00**	0.007	**−0.01**	$<10^{-3}$
cal_asymmetric_3	+0.00	–	+0.00	–	+0.00	–	+0.00	–	+0.00	–
duffing_triple_well	+0.00	–	+0.00	–	+0.00	–	+0.00	1.000	+0.00	–
arrested_spiral	+0.00	–	+0.00	–	+0.00	–	+0.00	–	+0.00	–
var_l_shape_5	+0.00	–	+0.00	–	+0.00	–	+0.00	–	+0.00	–
K/15 *Holm* $p < 0.05$	K=11 / N=11		K=11 / N=11		K=11 / N=11		K=11 / N=12		K=11 / N=11	

Table 26: Per-system effect sizes and Holm-corrected Wilcoxon p-values for the wrong-support-over-base MSE ratio at $H1$ on the deep slice. Alternative: candidate ratio > baseline.

System	LISTA		LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}
cal_high_cross_3	$\mathbf{+4.98\times10^{3}}$	$<10^{-3}$	$\mathbf{+1.47\times10^{3}}$	$<10^{-3}$	$\mathbf{+6.29\times10^{3}}$	$<10^{-3}$	$\mathbf{+2.93\times10^{3}}$	$<10^{-3}$	$\mathbf{+3.16\times10^{3}}$	$<10^{-3}$
cal_pentagon_5	$\mathbf{+5.58\times10^{3}}$	$<10^{-3}$	$\mathbf{+5.22\times10^{3}}$	$<10^{-3}$	$\mathbf{+4.33\times10^{3}}$	$<10^{-3}$	$\mathbf{+1.36\times10^{3}}$	$<10^{-3}$	$\mathbf{+1.23\times10^{3}}$	$<10^{-3}$
var_depth_gradient_4	$\mathbf{+4.68\times10^{3}}$	$<10^{-3}$	$\mathbf{+4.57\times10^{3}}$	$<10^{-3}$	$\mathbf{+8.59\times10^{3}}$	$<10^{-3}$	$\mathbf{+1.04\times10^{3}}$	$<10^{-3}$	$\mathbf{+1.04\times10^{3}}$	$<10^{-3}$
cal_octagon_8	$\mathbf{+5.02\times10^{3}}$	$<10^{-3}$	$\mathbf{+6.47\times10^{3}}$	$<10^{-3}$	$\mathbf{+8.70\times10^{3}}$	$<10^{-3}$	$\mathbf{+3.83\times10^{3}}$	$<10^{-3}$	$\mathbf{+3.78\times10^{3}}$	$<10^{-3}$
gated_transfer_linear	$\mathbf{+3.67\times10^{3}}$	$<10^{-3}$	$\mathbf{+4.36\times10^{3}}$	$<10^{-3}$	$\mathbf{+5.44\times10^{3}}$	$<10^{-3}$	$\mathbf{+9.16\times10^{3}}$	$<10^{-3}$	$\mathbf{+7.65\times10^{3}}$	$<10^{-3}$
cal_square_4	$\mathbf{+1.06\times10^{4}}$	$<10^{-3}$	$\mathbf{+1.31\times10^{4}}$	$<10^{-3}$	$\mathbf{+1.71\times10^{4}}$	$<10^{-3}$	$\mathbf{+5.82\times10^{3}}$	$<10^{-3}$	$\mathbf{+5.40\times10^{3}}$	$<10^{-3}$
var_diamond_4	$\mathbf{+1.47\times10^{4}}$	$<10^{-3}$	$\mathbf{+5.46\times10^{3}}$	$<10^{-3}$	$\mathbf{+1.62\times10^{4}}$	$<10^{-3}$	$\mathbf{+8.87\times10^{3}}$	$<10^{-3}$	$\mathbf{+1.07\times10^{4}}$	$<10^{-3}$
cal_hexagon_6	$\mathbf{+1.33\times10^{4}}$	$<10^{-3}$	$\mathbf{+7.89\times10^{3}}$	$<10^{-3}$	$\mathbf{+9.78\times10^{3}}$	$<10^{-3}$	$\mathbf{+1.22\times10^{4}}$	$<10^{-3}$	$\mathbf{+1.22\times10^{4}}$	$<10^{-3}$
snic_multi	$\mathbf{+2.79\times10^{4}}$	$<10^{-3}$	$\mathbf{+1.85\times10^{4}}$	$<10^{-3}$	$\mathbf{+2.35\times10^{4}}$	$<10^{-3}$	$\mathbf{+2.09\times10^{5}}$	$<10^{-3}$	$\mathbf{+3.23\times10^{5}}$	$<10^{-3}$
transition_routes_4	$\mathbf{+7.07\times10^{4}}$	$<10^{-3}$	$\mathbf{+1.10\times10^{5}}$	$<10^{-3}$	$\mathbf{+7.64\times10^{4}}$	$<10^{-3}$	$\mathbf{+3.21\times10^{4}}$	$<10^{-3}$	$\mathbf{+2.75\times10^{4}}$	$<10^{-3}$
gated_local_linear	$\mathbf{+7.77\times10^{4}}$	$<10^{-3}$	$\mathbf{+5.51\times10^{4}}$	$<10^{-3}$	$\mathbf{+1.13\times10^{5}}$	$<10^{-3}$	$\mathbf{+2.81\times10^{5}}$	$<10^{-3}$	$\mathbf{+3.63\times10^{5}}$	$<10^{-3}$
K/15 *Holm* $p < 0.05$	K=11 / N=11		K=11 / N=11		K=11 / N=11		K=11 / N=11		K=11 / N=11	

Table 27: Per-system effect sizes and Holm-corrected Wilcoxon p-values for the wrong-support-over-base MSE ratio at $H20$. Alternative: candidate > baseline.

System	LISTA		LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_H	Δ	p_H	Δ	p_H	Δ	p_H	Δ	p_H
cal_high_cross_3	+1.91	0.076	−0.338	0.756	+0.790	0.094	**+3.46**	0.036	**+13.9**	0.008
var_depth_gradient_4	**+4.38**	0.035	**+0.919**	0.018	**+3.01**	$<10^{-3}$	**+13.6**	$<10^{-3}$	**+13.6**	$<10^{-3}$
cal_octagon_8	**+9.78**	$<10^{-3}$	**+4.45**	$<10^{-3}$	**+2.99**	$<10^{-3}$	**+6.57**	$<10^{-3}$	**+8.59**	$<10^{-3}$
cal_square_4	**+7.85**	$<10^{-3}$	**+9.39**	$<10^{-3}$	**+3.43**	$<10^{-3}$	**+10.6**	$<10^{-3}$	**+10.6**	$<10^{-3}$
cal_pentagon_5	**+11.0**	$<10^{-3}$	**+12.3**	$<10^{-3}$	**+7.70**	0.001	**+11.0**	$<10^{-3}$	**+8.97**	$<10^{-3}$
gated_transfer_linear	**+8.94**	0.002	**+22.3**	0.001	**+14.5**	$<10^{-3}$	**+27.5**	$<10^{-3}$	**+22.1**	$<10^{-3}$
cal_hexagon_6	**+29.3**	$<10^{-3}$	**+26.0**	$<10^{-3}$	**+7.23**	$<10^{-3}$	**+45.8**	$<10^{-3}$	**+45.8**	$<10^{-3}$
var_diamond_4	**+55.7**	$<10^{-3}$	**+1.75**	0.004	**+40.9**	$<10^{-3}$	**+49.4**	$<10^{-3}$	**+49.2**	$<10^{-3}$
transition_routes_4	**+308**	$<10^{-3}$	**+176**	$<10^{-3}$	**+214**	$<10^{-3}$	**+209**	$<10^{-3}$	**+154**	$<10^{-3}$
snic_multi	**+214**	$<10^{-3}$	**+282**	$<10^{-3}$	**+186**	$<10^{-3}$	**+573**	$<10^{-3}$	**+626**	$<10^{-3}$
gated_local_linear	**+568**	$<10^{-3}$	**+362**	$<10^{-3}$	**+1.17×10³**	$<10^{-3}$	**+2.95×10³**	$<10^{-3}$	**+4.07×10³**	$<10^{-3}$
$K/15$ *Holm* $p<0.05$	K=10 / N=11		K=10 / N=11		K=10 / N=11		K=11 / N=11		K=11 / N=11	

Table 28: Deep-state support-routed forecasting route-form diagnostics. S_{top8} columns use exact eight-index supports; F_{top8} columns use merged families of those exact supports. Entries are arithmetic means across systems after within-system seed-IQM summarization, with diagnostic within-system Wilcoxon/Holm counts in brackets. LISTA-SB is the matched d_z=256 soft-block LISTA ablation; lower is better.

*H*100

Model	Gated S_{top8} equation (46)	S_{top8}-local equation (43)	F_{top8}-local equation (43)
LISTA	0.783 [4/15]	0.771 [5/15]	0.165 [9/15]
LISTA-SB	0.861 [1/15]	0.861 [1/15]	0.00372 [13/15]
LISTA-BD	0.915 [1/15]	0.919 [1/15]	**0.00207** [12/15]
Sparse MLP, BD	**0.797** [0/15]	0.770 [0/15]	0.00941 [6/15]
Sparse MLP	0.813 [0/15]	**0.736** [0/15]	0.0104 [5/15]
Dense MLP	0.991 [0/15]	0.990 [0/15]	102 [0/15]

*H*1000

Model	Gated S_{top8} equation (46)	S_{top8}-local equation (43)	F_{top8}-local equation (43)
LISTA	0.681 [8/15]	0.676 [7/15]	$\mathbf{8.03\times10^{-14}}$ [11/15]
LISTA-SB	0.817 [7/15]	0.809 [6/15]	6.30×10^{-11} [13/15]
LISTA-BD	0.690 [6/15]	0.709 [7/15]	1.19×10^{-7} [13/15]
Sparse MLP, BD	$\mathbf{1.48\times10^{-5}}$ [1/15]	$\mathbf{2.99\times10^{-4}}$ [1/15]	1.49×10^{-6} [14/15]
Sparse MLP	0.804 [2/15]	0.738 [1/15]	0.126 [14/15]
Dense MLP	0.849 [0/15]	0.965 [0/15]	78.6 [0/15]

Table 29: Robust Dysts dt×30 forecasting aggregation. Entries are IQMs across retained systems of each system's seed-IQM best-periodic MSE; lower is better. **Bold** marks the best row in each horizon column.

Model	H100	H500	H1000	H1500	H2000	H3000	H4000	H5000
LISTA	2.31×10^{-4}	0.00288	0.00975	0.0219	0.0453	0.146	0.319	0.536
LISTA-BD	1.81×10^{-4}	0.00277	0.00929	0.0200	0.0378	0.115	0.293	0.512
LISTA-SB	3.69×10^{-4}	0.00532	0.0181	0.0446	0.0960	0.305	0.588	0.900
Sparse MLP	1.86×10^{-4}	0.00313	0.0112	0.0266	0.0532	0.168	0.365	0.590
Sparse MLP-BD	$\mathbf{1.39\times10^{-4}}$	**0.00209**	**0.00676**	**0.0142**	**0.0268**	**0.0912**	**0.238**	**0.425**
Dense MLP	5.04×10^{-4}	0.00661	0.0245	0.0604	0.119	0.349	0.641	1.01

Table 30: Scale-normalized Dysts forecasting summary. Entries are arithmetic means across systems of the candidate per-system seed-IQM MSE divided by the Dense MLP per-system seed-IQM MSE; lower than 1 favors the candidate. This table gives the average-case ratio view corresponding to the actual-MSE table in table 1. LISTA-SB is retained as a diagnostic row.

Model	H100	H500	H1000	H1500	H2000	H3000	H4000	H5000
LISTA	0.509	0.464	0.459	0.426	0.440	0.489	0.565	0.595
LISTA-BD	0.530	0.587	0.502	0.445	0.443	0.498	0.574	0.612
LISTA-SB	0.972	1.026	1.019	1.088	1.157	1.264	1.086	1.006
Sparse MLP	0.462	0.499	0.501	0.563	0.566	0.610	0.685	0.739
Sparse MLP-BD	0.580	0.601	0.497	0.396	0.395	0.431	0.495	0.527
Dense MLP	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000

S NeurIPS checklist details

S.1 Checklist: Claims

Checklist question. Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

Answer. Yes.

Concise justification. The abstract and introduction state bounded methodological and empirical claims: sparse Koopman autoencoders are trained without basin labels or regime annotations; sparse supports are treated as post-hoc, model-produced regime variables; sparse-latent models improve forecasting relative to dense-latent Koopman autoencoder controls on the reported benchmarks; active-coordinate identities matter for prediction; and LISTA support families identify held-out basin-interior states better than dense-latent controls. These claims match the stated experimental design, reported metrics, and discussion caveats.

Evidence. The training/deployment claim is explicitly limited to the unsupervised representation setting: the model is not given basin labels, basin counts, or trajectory-to-basin assignments during training, and benchmark basin labels are reserved for post-hoc scoring or diagnostic interventions. The paper also distinguishes this from the block-diagonal and soft-block transition ablations, where benchmark metadata is used only to set fixed latent transition groups.

The forecasting claim is supported by the main forecasting table over 15 controlled multibasin systems and 10 Dysts $dt\times30$ systems. On the multibasin benchmark, the dense MLP baseline reports MSEs 0.830, 2.68, and 2.93 at $H100$, $H500$, and $H1000$, while all sparse rows report much lower values with Holm-corrected significance annotations. On Dysts, the dense MLP baseline reports 0.00110, 0.224, and 0.754 at $H100$, $H2000$, and $H4000$, while the primary sparse rows remain below the baseline at the displayed horizons, with the strongest annotations for the LISTA, sparse MLP, and sparse MLP-BD rows.

The functional-support claim is supported by the support-coordinate intervention experiment. The standard rollout has mean accumulated MSE 0.0158 at $H = 21$, while dropping the top 1/2/3/5/10 active coordinates increases the corresponding errors to 0.508/1.37/2.08/3.26/8.51, and random reassignment of active coefficient values to inactive coordinates is described as even more destructive. This directly tests active-coordinate identity rather than only latent amplitude shrinkage.

The basin-support alignment claim is supported by the held-out per-basin deep-slice diagnostic. The main table reports $H(B \mid F_{\text{abs}}) = 0.130$ for LISTA and 1.28 for the dense MLP baseline, with LISTA-family counts close to the benchmark basin-count summary reported in the text. The diagnostic is post-hoc: F_{abs} support families are built from learned absolute-threshold supports by a fixed Jaccard merge rule, and basin labels are used only afterward to evaluate conditional entropy.

Remaining caveat. The paper does not claim that a single sparse model learns a globally smooth finite-dimensional linearization of the whole multibasin state space. The support-identification evidence is strongest on two-dimensional procedural systems and on held-out states deep inside basins; it does not settle behavior near separatrices, rare transitions, partial observation, or high-dimensional physical systems. The Dysts experiments broaden the forecasting evidence but are not basin-identification tests, and the support-family construction depends on fixed threshold choices that must be interpreted together with conditional entropy and family count.

S.2 Checklist: Limitations

Question. Does the paper discuss the limitations of the work performed by the authors?

Answer. Yes.

Justification. The Discussion explicitly identifies what the experiments do and do not establish. The paper claims that sparse Koopman autoencoders can expose support families that act as label-free, inspectable regime variables, and it supports this claim with forecasting, support-coordinate interventions, and support–basin alignment diagnostics. It also states that the evidence is strongest in deliberately controlled settings and should not be read as a proof of a globally smooth finite-dimensional Koopman linearization for a multibasin state space.

Concrete limitation details and interpretation.

- **Controlled multibasin scope.** The strongest basin-identification evidence comes from procedurally generated two-dimensional systems whose attractor geometry is known. These systems are appropriate for isolating whether learned sparse supports can align with basin-scale structure, but they do not by themselves establish the same behavior for high-dimensional physical systems, partially observed systems, or systems whose attractors and basin geometry are unknown.
- **Basin-interior evaluation slice.** The main support–basin alignment diagnostic is evaluated on held-out states in the per-basin top quartile of a basin-depth margin. This is an intentionally clean slice for asking whether support families identify basin interiors. It does not settle behavior near separatrices, during rare transitions, or in regions where small perturbations can change basin assignment or sparse support.
- **Evaluation labels are unavailable at deployment.** Basin labels, basin counts, and attractor locations are not used to train the sparse encoder, fit the Koopman transition, or construct support families in the intended setting. They are used only for benchmark evaluation and diagnostic slicing. Consequently, the reported entropy and family-count metrics should be interpreted as post-hoc evidence that the learned supports align with known benchmark basins, not as a supervised routing procedure.
- **External chaotic-flow evidence is forecasting-only.** The Dysts $dt\times30$ experiments broaden the forecasting evidence beyond the procedural generator and show that sparse-latent models remain competitive on nontrivial chaotic systems. They are not basin-identification tests, because those systems do not provide the same evaluated basin-label structure used for the controlled support–basin diagnostics.
- **Support-family construction has fixed design choices.** The main support-family object F_{abs} starts from an absolute latent activation threshold and then merges exact supports using a fixed Jaccard threshold. These choices affect fragmentation and merging: a loose merge can collapse distinct regimes, while a strict merge can create many families and lower conditional entropy by over-fragmenting basins. The family count must therefore be interpreted together with $H(B \mid F_{\mathrm{abs}})$, rather than optimized or read in isolation.
- **Forecasting protocol is a benchmark diagnostic.** Forecast horizons and re-encoding periods are measured in stored observation steps, and equal horizon values need not correspond to equal physical time across systems. Periodic re-encoding uses only the model's predicted state, not the true future or an oracle basin label, but the best-periodic summaries should still be read

as controlled deployment diagnostics rather than as a guarantee of autonomous long-horizon stability in all regimes.

S.3 Checklist: Theory assumptions and proofs

Question. For the theory-assumptions-and-proofs checklist item, are there original theoretical results for which the manuscript must state complete assumptions and proofs?

Answer. NA.

Justification. The current manuscript does not state any original theorem, lemma, proposition, corollary, or proof. The theory-facing material is used to frame the empirical study: it defines the stored-step dynamical map, basins of attraction, Koopman operators, Koopman autoencoders, sparse codes, LISTA-style encoders, support sets, support families, training losses, and re-encoding rollouts. The claims about local linearization, Koopman eigenfunction constructions, and the obstruction to a single finite-dimensional global linearization in multibasin systems are attributed to prior work rather than presented as new results proved in this paper.

The paper's original contribution is therefore methodological and empirical: it proposes sparse supports as inspectable, label-free regime variables for sparse Koopman autoencoders, then evaluates forecasting quality, support-coordinate interventions, and basin-support alignment on benchmark systems. These contributions are supported by experiment protocols, diagnostics, and statistical tests, not by new formal proof obligations.

Caveats. This answer relies on the current manuscript text. The preamble defines theorem-like environments, but they are not used in the current paper or included appendices. If a later revision adds a formal theorem, proposition, identifiability statement, convergence guarantee, recovery guarantee, or impossibility result, this checklist item should be revisited and the paper should provide all assumptions and complete proofs. Methodological statements such as sparse supports "can act" as regime variables or sparsity "encourages" local support separation should remain framed as modeling motivation or empirically tested hypotheses unless they are converted into formal claims with proofs.

S.4 Experimental result reproducibility

Question. Are the main experimental results reproducible from the released paper, code, configuration surfaces, scripts, generated benchmark definitions, random seeds, and output artifacts?

Answer. Yes.

Justification. The paper-facing protocol specifies the benchmark populations, model families, training budgets, evaluation horizons, statistical units, and seed aggregation rules. The repository also contains executable configuration surfaces that make the protocol reproducible: `pyproject.toml` and `uv.lock` define the environment, `tools/train.py` is the single training entry point, `skae/config.py` defines model and optimizer defaults, and the paper launchers write task tables whose rows become the exact command-line arguments passed to `tools/train.py`. Each training run persists its resolved `config.json`, checkpoints, metrics, and evaluation outputs, so a reproduction can audit the actual hyperparameters used by any displayed row.

The only qualification is that exact bitwise equality is not promised across different GPU models, CUDA libraries, or PyTorch kernels. Reproducibility should therefore be interpreted as reproducing the same task table, seeds, data generation protocol, checkpoint-selection rule, and statistical summaries, with small numerical differences possible across hardware/software stacks.

Environment. Use the locked project environment:

- Python: ≥ 3.10, managed with `uv`.
- Dependency source: `pyproject.toml` plus `uv.lock`.
- Core dependencies: `torch>=2.2`, `numpy`, `scipy`, `matplotlib`, `dysts>=0.96`, `numba`, and `scikit-learn`. The Dysts appendix records the paper-facing resolved Dysts version as `0.96`.
- GPU jobs: the benchmark array runner uses SLURM on the `long` partition with one GPU, four CPUs, 16 GB memory, and `cuda/12.6.0`. CPU reducers and collectors are launched as SLURM jobs as well.

  The setup command is:

```
uv sync
```

All Python entry points should be run through `uv run`; paper-scale training and evaluation should be submitted through the SLURM scripts rather than run on a login node.

Core training and evaluation surfaces. The central command constructed by the launchers is:

```
uv run python tools/train.py \
  --config <config_name> \
  --env <env_name> \
  --env_dt <stored_step> \
  --num_steps <steps> \
  --batch_size 256 \
  --target_size 256 \
  --res_coeff 1.0 \
  --reconst_coeff 0.03 \
  --pred_coeff 1.0 \
  --sparsity_coeff <lambda_sp> \
  --sequence_length <L> \
  --eval_profile full \
  --seed <seed> \
  --device cuda \
  --log_dir <run_root>
```

The model rows are selected by `--config`, `--k_structure`, `--soft_block`, LISTA flags, and the sparsity coefficient:

- LISTA: `lista_parity_generic_sparse` with `lista_alpha=0.15`, `lista_num_loops=2`, `lista_final_op=sign_split`, dense K, and $\lambda_{\text{sp}} = 0.003$ on the multibasin benchmark.
- LISTA-BD: the same LISTA encoder, with `k_structure=block_diagonal`. The block count is set from benchmark metadata only for this architecture diagnostic.
- LISTA-SB: the same LISTA encoder, dense K, and a soft off-block penalty. The soft-block count is likewise architecture metadata, not a training label.

- Sparse MLP: `generic_sparse`, dense K, and the paper sparsity coefficient.
- Sparse MLP-BD: `generic_sparse` with block-diagonal K.
- Dense MLP: `generic_no_shrink` with $\lambda_{\text{sp}} = 0$ and no final ReLU sparsifying gate.

Multibasin benchmark route. The paper-facing controlled benchmark contains 15 two-dimensional multibasin systems:

```
gated_local_linear
gated_transfer_linear
claude:arrested_spiral
claude:cal_asymmetric_3
claude:cal_high_cross_3
claude:cal_hexagon_6
claude:cal_octagon_8
claude:cal_pentagon_5
claude:cal_square_4
claude:duffing_triple_well
claude:snic_multi
claude:transition_routes_4
claude:var_depth_gradient_4
claude:var_diamond_4
claude:var_l_shape_5
```

They are continuous-time vector fields integrated with fourth-order Runge–Kutta to produce stored observations. Training observes only stored states; vector fields, integration substeps, basin labels, basin counts, and trajectory-to-basin assignments are not supplied to the encoder or transition model. Basin metadata is used after training for evaluation and, only for the block-diagonal and soft-block diagnostic rows, to set the fixed number of transition groups.

The multibasin paper settings are:

- Seeds: $0, 1, \ldots, 14$.
- Latent dimension: $d_z = 256$.
- Rollout-window length: $L = 8$, i.e. nine adjacent stored states.
- Optimization: 200,000 steps, batch size 256, AdamW, encoder and decoder learning rate 5×10^{-5}, Koopman-matrix learning rate 5×10^{-6}, and weight decay 10^{-4} on non-transition parameters.
- Loss weights: $\lambda_{\text{pred}} = 1$, $\lambda_{\text{rec}} = 0.03$, $\lambda_{\text{lin}} = 1$, $\lambda_{\text{sp}} = 0.003$ for sparse rows, and $\lambda_{\text{sp}} = 0$ for Dense MLP.
- Training-state distribution: boundary-emphasized hard-initial-condition oversampling enabled for all six controlled rows; 50% of windows start from a retained pool of 1024 states selected from 4096 candidates using 32-step, label-free perturbation/transient scores, with four perturbations, perturbation scale 0.04, transient window 8, transient weight 0.5, and jitter scale 0.25.
- Checkpoint rule: every 500 steps, and at the final step, evaluate a 200-step every-step-reencoded validation rollout from 16 held-out initial states; save the checkpoint with lowest final validation error as `checkpoint.pt`.

The reproducibility route is to regenerate task tables and submit them through the existing array runner:

```
uv run python tools/build_transition_rich_basin_partition_tasks.py \
  --phase_label transition_rich_basin_partition \
  --output_tsv <task_tsv> \
  --output_manifest_json <manifest_json> \
  --systems_csv <the 15-system comma-separated list above> \
  --model_variants_csv lista_blockdiag_signsplit_hardinit_basin_partition,\
mlp_sparse_blockdiag_hardinit_basin_partition_control,\
mlp_sparse_hardinit_basin_partition_control,\
mlp_zero_sparse_hardinit_basin_partition_control \
  --seeds_csv 0,1,2,3,4,5,6,7,8,9,10,11,12,13,14 \
  --num_steps_override 200000

TASK_TSV=<task_tsv> BASE_OUT=<scratch_output_root> \
  sbatch --array=0-<num_tasks_minus_1> scripts/run_paper_benchmark_array.sh
```

The current repository also includes paper-facing convenience launchers for the matched-dimension LISTA, LISTA-SB, and backfill rows:

```
sbatch scripts/queue_transition_rich_lista_sb_p256_hardinit_fairness.sh
sbatch scripts/queue_transition_rich_lista_dense_p256_hardinit_table123.sh
sbatch scripts/queue_transition_rich_table2_5model_seed_backfill.sh
sbatch scripts/queue_sparse_mlp_bd_repaired_table1.sh
bash scripts/queue_per_basin_deep_current_table1_eval.sh
```

Some older or broader scripts still know about 17-system exploratory packets. For reproducing the claims in this paper, use the 15-system roster above or the filtered paper-facing figure/statistics scripts.

Dysts $dt \times 30$ forecasting route. The paper-facing Dysts benchmark contains these 10 three-dimensional systems:

```
dysts:Chua
dysts:Dadras
dysts:DequanLi
dysts:Hadley
dysts:LuChenCheng
dysts:QiChen
dysts:Sakarya
dysts:SanUmSrisuchinwong
dysts:ShimizuMorioka
dysts:WangSun
```

For each system, stored observations use 30 times the native Dysts integration interval, and coordinates are standardized after trajectory generation. The full Dysts cache profile uses 200 cached trajectories, 30000 stored observations per trajectory, and a 2000-step warm-up. Initial conditions are the Dysts default initial condition plus Gaussian perturbations with scale 0.2 times the coordinate-wise Dysts standard deviation. Train, validation, and test caches are split by cache namespace; held-out test windows are disjoint from training windows.

The Dysts paper settings are:

- Seeds: $0, 1, \ldots, 14$.
- Latent dimension: $d_z = 256$.
- Rollout-window length: $L = 10$.
- Optimization: 100,000 steps and batch size 256.
- LISTA learning rates: 5×10^{-5} for encoder/decoder and 5×10^{-6} for K.
- MLP learning rates: 10^{-4} for encoder/decoder and 10^{-5} for K.
- Sparse-row Dysts coefficient: $\lambda_{\text{sp}} = 0.006$; Dense MLP: $\lambda_{\text{sp}} = 0$.
- Long-horizon evaluation: 100 held-out test windows per system and seed, horizons $100, 500, 1000, 1500, 2000, 3000, 4000, 5000$, and periodic re-encoding periods $10, 25, 50, 100, 150, 200$.

Use the Dysts queue with the 10-system paper list explicitly, because the task builder also contains additional stress-test systems:

```
SYSTEMS_CSV=dysts:Chua,dysts:Dadras,dysts:DequanLi,dysts:Hadley,\
dysts:LuChenCheng,dysts:QiChen,dysts:Sakarya,\
dysts:SanUmSrisuchinwong,dysts:ShimizuMorioka,dysts:WangSun \
SEEDS_CSV=0,1,2,3,4,5,6,7,8,9,10,11,12,13,14 \
NUM_STEPS=100000 \
sbatch scripts/queue_dysts_dt30_basinblock_p256_seeds0to14.sh
```

That launcher builds the task table with `tools/build_dysts_dt30_basinblock_tasks.py`, prebuilds Dysts `train`/`val` caches, runs packed training arrays through `scripts/run_paper_benchmark_packed_array.sh`, and queues `scripts/queue_dysts_long_horizon_eval.sh` for the held-out test cache evaluation.

Support and mechanism diagnostics. Support definitions are fixed in code and prose:

- S_{abs}: active coordinates with absolute threshold 10^{-3}.
- F_{abs}: greedy Jaccard families of S_{abs}, with threshold 0.5, used for the main basin-support alignment diagnostic.
- S_{top8} and F_{top8}: eight largest-magnitude coordinates and their support families, used for routing and refresh appendix diagnostics.

The per-basin deep-state slice is evaluation-only: within each benchmark basin, states are ranked by the basin-depth margin, defined as the distance to the second-nearest attractor center minus the distance to the nearest attractor center, and the top quartile is retained. Support reducers use `eval_seed=42` by default, 128 trajectories of length 128, endpoint rollouts of 5000 steps for basin labeling when needed, and `DEPTH_SLICE_MODE=per_basin` for the current paper table.

The coordinate-intervention experiment uses the LISTA dense-transition model, the `gated_local_linear` system, training seed 0, 100 held-out basin-interior starts, horizons $1, 3, \ldots, 21$, top-k coordinate drops up to $k = 10$, and 20 random inactive-coordinate reassignments. The route is:

```
NUM_INITIAL_POINTS=100 \
RANDOM_SUPPORT_REPEATS=20 \
ROWS_CSV=<paper_forecasting_rows.csv> \
```

```
OUT_DIR=<support_intervention_output_dir> \
sbatch scripts/run_support_coordinate_interventions.sh
```

Randomness and seeds. Training seeds are the explicit model seeds $0, \ldots, 14$. Within a run, `tools/train.py` sets `torch.manual_seed(cfg.SEED)` and, when available, the CUDA seed. Online training batches use deterministic `torch.Generator` instances derived from the run seed. The fixed validation set uses seed `cfg.SEED` + 999999. Standardized evaluation uses seed `cfg.SEED` + 12345. Dysts cache generation is intentionally independent of the training seed and uses split-specific deterministic cache seeds: train, validation, and test are separate namespaces. Support reducers use default evaluation seed 42, and random-support shuffling uses random seed 123 unless overridden.

Outputs and paper regeneration. Each run directory contains:

- `config.json`: resolved configuration and command-line overrides.
- `checkpoint.pt`: best checkpoint by the validation rule.
- `last.pt`: final checkpoint.
- `metrics_history.jsonl`, `metrics_summary.json`, `final_metrics.json`, and `training_metrics.png`.
- `evaluation_results_best.json`, `evaluation_results_last.json`, `evaluation_summary.json`, and per-system evaluation directories with optional rollout artifacts.

Collectors and reducers then produce paper-facing tables:

- Forecasting rows: `forecasting_rows.csv`, produced by `tools/collect_forecasting_roots.py` through `scripts/collect_transition_rich_basin_partition.sh`, `scripts/collect_paper_benchmark.sh`, or the Dysts long-horizon collector.
- Support diagnostics: `interpretability_rows.csv`, produced by `scripts/reduce_transition_rich_interpretability_metrics.sh` and merged by `scripts/merge_transition_rich_interpretability_shards.sh`.
- Dysts long-horizon rows and compact rollout artifacts from `tools/evaluate_dysts_long_horizon_run.py` and `scripts/collect_dysts_long_horizon_forecasting.sh`.
- Final paper figures and LaTeX tables under `docs/figures/neurips_paper_2026/` and `docs/figures/neurips_paper_2026/_tables/`, generated by `scripts/build_per_system_stats_and_forest.py` and `scripts/build_paper_seed_stats_and_figures.py`.

Point estimates are interquartile means over seeds within each system followed by arithmetic means across systems, except for explicitly descriptive count columns. Multibasin forecasting uses paired within-system Wilcoxon signed-rank tests with Holm correction. Dysts forecasting uses per-system seed-IQM effects and a one-sided exact sign test across the 10 systems with Holm correction.

S.5 Checklist: Open access to data and code

Question. Does the paper provide open access to the data and code needed to reproduce the main experimental results?

Answer (Yes/No/NA). Yes.

Justification. The empirical setting uses generated dynamical-system observations rather than a restricted or proprietary dataset. The controlled multibasin benchmark consists of procedurally specified two-dimensional systems with known attractor geometry, and the Dysts benchmark uses public chaotic-flow systems from the external `dysts` package. The released artifact will include the implementation used to generate observations, train sparse and dense Koopman autoencoders, evaluate held-out rollouts, compute support diagnostics, and build the paper-facing tables and figures. Basin labels, basin counts, and attractor coordinates are included only as benchmark metadata for evaluation and diagnostics; the training code does not require them for sparse-support selection.

Exactly what will be released and included.

- The core source package, including model definitions, sparse encoders, Koopman transition variants, configuration presets, dynamical-system environments, benchmark adapters, and evaluation utilities.
- Command-line tools and batch scripts for training, checkpoint evaluation, benchmark task construction, result collection, statistical summaries, and figure/table generation.
- Environment and reproducibility metadata, including `pyproject.toml`, `uv.lock`, the project README, and the package entry points for training and evaluation.
- The paper source and paper-facing generated artifacts needed to inspect the reported claims, including the LaTeX tables, figure files, aggregate summaries, and collected CSV/JSON metric files used to produce the main forecasting and support-diagnostic displays.
- Benchmark definitions for the 15 controlled multibasin systems and the 10 Dysts $dt \times 30$ forecasting systems, including system keys, dimensions, stored-step conventions, and the evaluation-only metadata needed to score basin-support alignment on the controlled benchmark.
- Instructions for regenerating trajectories, rerunning the training and evaluation pipeline, and rebuilding the paper summaries from released artifacts.

Data generation and access. For the controlled multibasin benchmark, there is no separate private dataset. Training, validation, and test windows are generated from released vector-field implementations by applying the stored observation map with fourth-order Runge–Kutta integration. The learner observes stored states only; it does not observe continuous-time vector fields, integration substeps, basin labels, or trajectory-to-basin assignments during training. Evaluation-only basin metadata is used after training to define diagnostic slices, controlled transfer pairs, and support–basin agreement scores.

For the Dysts benchmark, trajectories are generated from public Dysts systems through the released adapter code and the dependency version resolved by the project environment. The paper-facing setup uses standardized coordinates, native Dysts trajectory generation, deterministic train/validation/test cache split namespaces, 200 cached trajectories, 30,000 stored observations per trajectory, and a 2,000-step warm-up before cached observations are used. The released code can rebuild these caches from the public Dysts right-hand sides; the cache files themselves are generated artifacts rather than required external data.

Anonymization. The experiments do not use human-subject data, personal data, private text, images, medical records, user logs, or other sensitive data. The generated states, labels, checkpoints, and metrics contain only synthetic dynamical-system values and model outputs. During anonymous

review, the release will remove or neutralize author-identifying metadata, local usernames, private scratch paths, and cluster-specific job records that are not needed for reproduction. The final public release can restore standard project attribution and licensing.

What is not released. The release will not include private cluster scratch directories, raw SLURM stdout/stderr logs, queue bookkeeping, editor/agent state, exploratory planning notes, failed pilot runs, or the full `runs/` tree of transient training outputs. Large generated Dysts cache tensors and exhaustive checkpoint files for exploratory sweeps are not required for access to the benchmark data, because they can be regenerated from the released code and public dependencies. The paper-facing aggregate CSV/JSON summaries, tables, figures, and scripts needed to audit the reported results will be included.

S.6 Checklist: experimental setting and details

Question. Are the experimental setting and implementation details specified well enough to interpret and reproduce the main empirical claims, including data splits, seeds, model classes, optimizer settings, evaluation horizons, and support metrics?

Answer (Yes/No/NA). Yes.

Justification. The main paper defines the training objective, the label-free basin-discovery setting, the model families, the evaluation-only role of basin annotations, and the primary forecasting and support-alignment metrics. The current training and evaluation entry points further specify the final paper-facing hyperparameters, held-out evaluation populations, checkpoint-selection rule, and support-family construction. The only training-time use of benchmark basin metadata is for the diagnostic block-structured transition rows, where the fixed number of transition groups is set from the benchmark generator or scroll/lobe count; basin labels and trajectory-to-basin assignments are not used to train encoders, select supports, construct support families, or choose rollouts.

Summarized experimental setting.

1. **Benchmarks.** The controlled benchmark contains 15 procedurally generated two-dimensional multibasin systems. These systems have known attractor geometry, used only after training for evaluation slices, support-basin scoring, and controlled interventions. The external Dysts forecasting benchmark contains 10 three-dimensional chaotic systems: Chua, Dadras, Dequan Li, Hadley, Lu–Chen–Cheng, Qi–Chen, Sakarya, San–Um–Srisuchinwong, Shimizu–Morioka, and Wang–Sun. The Dysts benchmark uses stored observations at 30 times each system's native integration timestep, with standardized coordinates.
2. **Training data and splits.** Controlled-system training samples online windows of adjacent stored observations. Training uses a boundary-emphasized, label-free reset distribution: 4096 candidate initial conditions are scored by short-rollout perturbation sensitivity and late transient motion, the top 1024 are retained, and 50% of training windows start from this pool with jitter. The hard-state score uses 32 probe steps, 4 perturbations, perturbation scale 0.04, transient window 8, transient weight 0.5, and jitter scale 0.25. Controlled forecasting is evaluated on 100 held-out initial conditions per system and training seed, not on training windows. Support-basin alignment uses the evaluation-only per-basin deep slice: within each represented basin, states in the top quartile of the basin-depth margin are scored.

3. **Dysts caches and splits.** Dysts training uses native trajectory caches with 200 trajectories, 30,000 stored observations per trajectory, and a 2,000-step warm-up. The first cached trajectory starts from the Dysts default initial condition; remaining trajectories perturb it with Gaussian noise at scale 0.2 times the coordinate-wise Dysts standard deviation. Train, validation, and test caches use separate deterministic split namespaces. Long-horizon Dysts evaluation samples 100 test-cache windows per system and seed, rather than sampling from the training-cache namespace.

4. **Seeds.** The final paper-facing comparisons train each model on 15 random seeds, $0, \ldots, 14$, for every reported system. Within training, model and batch sampling use the run seed; the fixed validation set uses an offset seed; standardized forecasting evaluation uses a separate offset seed. Dysts cache construction is deterministic and independent of the model seed so that cache files can be reused across model seeds while preserving train/validation/test separation.

5. **Model rows.** The main table compares six model rows: LISTA, LISTA-BD, LISTA-SB, Sparse MLP, Sparse MLP-BD, and Dense MLP. All final rows use latent dimension $d_z = 256$. MLP encoders use two hidden layers of width 64, a linear decoder, and a learned latent transition K. The Dense MLP baseline uses a tanh encoder without an output ReLU gate and sets the latent sparsity penalty to zero. Sparse MLP rows use the sparse MLP preset with an L_1 sparsity penalty and either dense or block-diagonal K. LISTA rows use a learned sparse encoder with threshold scale $\alpha = 0.15$, a normalized linear decoder dictionary, and dense, hard block-diagonal, or soft-block-regularized K. The controlled multibasin LISTA launchers use two LISTA refinement loops and a sign-split final operation; the Dysts $dt \times 30$ task builder uses one LISTA loop and a ReLU final operation. Block-diagonal and soft-block rows set the number of transition groups from benchmark metadata as an architecture diagnostic, not as a training label.

6. **Optimizer and controlled-system hyperparameters.** Controlled multibasin runs train for 200,000 optimization steps on rollout windows of length $L = 8$, i.e. 9 adjacent stored states, with minibatches of 256 windows. They use AdamW with learning rate 5×10^{-5} for encoder/decoder parameters, 5×10^{-6} for K, and weight decay 10^{-4} on non-K parameters. Loss weights are $\lambda_{\text{pred}} = 1$, $\lambda_{\text{rec}} = 0.03$, $\lambda_{\text{lin}} = 1$, and $\lambda_{\text{sp}} = 3 \times 10^{-3}$ for sparse rows; the Dense MLP row sets $\lambda_{\text{sp}} = 0$. Soft-block rows add an L_1 off-block penalty with weight 10^{-4}.

7. **Optimizer and Dysts hyperparameters.** Dysts $dt \times 30$ runs train for 100,000 optimization steps on rollout windows of length $L = 10$, i.e. 11 adjacent stored states, with minibatches of 256 windows and $d_z = 256$. LISTA rows use learning rates 5×10^{-5} for encoder/decoder parameters and 5×10^{-6} for K. MLP rows use 10^{-4} and 10^{-5}, respectively. Weight decay is 10^{-4}. Sparse Dysts rows use $\lambda_{\text{sp}} = 6 \times 10^{-3}$, while Dense MLP uses $\lambda_{\text{sp}} = 0$. Soft-block Dysts rows use an L_1 off-block penalty with weight 10^{-4}.

8. **Checkpoint selection.** During training, every 500 optimization steps and at the final step, the model is rolled out for 200 steps from 16 fixed held-out initial states using every-step re-encoding. The saved best checkpoint is the one with the lowest final validation error under this fixed validation proxy. Final tables aggregate completed seeds; they do not select the best seed.

9. **Forecasting evaluation.** Horizons are counted in stored observation steps. Controlled-system evaluation computes MSE on held-out rollouts and reports the main horizons $H = 100, 500, 1000$; horizon curves use the same stored-step convention. The reported periodic forecast score is the best over fixed re-encoding periods $m \in \{10, 25, 50, 100\}$, where re-encoding uses the model's own predicted state and never the true future state. Dysts $dt \times 30$ evaluation uses held-out test-cache windows, horizons through $H = 5000$ with the main table reporting $H = 100, 2000, 4000$, and

fixed re-encoding periods $m \in \{10, 25, 50, 100, 150, 200\}$.

10. **Support metrics.** For a learned latent z, the absolute support is $S_{\text{abs}}(x) = \{i : |z_i| > 10^{-3}\}$. Exact absolute supports are merged into F_{abs} support families by a deterministic greedy leader rule over Boolean support masks, using Jaccard similarity threshold 0.5. The main support-alignment metric is $H(B \mid F_{\text{abs}})$ on the per-basin deep held-out slice, where B is the withheld benchmark basin label; lower values mean support families leave less uncertainty about basin identity. The companion count $\overline{|F_{\text{abs}}|}$ is descriptive and is interpreted relative to the number of represented basins. Functional support diagnostics include wrong-support/base error ratios at short horizons and coordinate interventions that drop the largest active coordinates or move active coefficient values to inactive coordinates. The intervention summary reports accumulated MSE at $H = 21$ using 100 held-out basin-interior starts; the random-support condition uses 20 random inactive-coordinate reassignments per start.

11. **Secondary support diagnostics.** The fixed-size support S_{top8} contains the eight largest-magnitude latent coordinates and is merged into F_{top8} families with the same Jaccard threshold 0.5. F_{top8} is used for appendix support-routed forecasting and support-refresh diagnostics. Support-routed diagnostics fit post-hoc local maps on a disjoint fitting split and evaluate on separate held-out rollouts. Support refresh uses controlled source-to-target transfers with 32 pre-transfer steps, 32 bridge steps, 128 post-transfer steps, and refresh periods $m \in \{1, 5, 10, 20\}$; it compares stale propagated families with families obtained by re-encoding the current controlled-transfer state.

12. **Aggregation and tests.** Main point estimates summarize seeds within each system by interquartile mean and then average system summaries. Multibasin forecasting and support-alignment significance tests use paired seed-level effects within each system against the Dense MLP baseline with Holm correction across systems. Dysts forecasting uses the benchmark system as the independent unit after seed-IQM summarization and applies paired one-sided sign tests with Holm correction. The family-count column is descriptive rather than tested as a monotone improvement metric.

S.7 Checklist: statistical significance

Question. Do the experiments report point estimates, uncertainty bands or error bars, statistical tests, multiple-comparison corrections, and the assumptions needed to interpret them?

Answer (Yes/No/NA). Yes.

Justification. The main forecasting and support-alignment claims compare trained models under matched systems and, when possible, matched random seeds. The paper does not pool trajectories, seeds, and systems as if they were independent samples. Instead, seeds are summarized within a benchmark system for point estimates, and formal tests use the paired unit appropriate to the claim: the training seed within a controlled multibasin system, or the benchmark system for cross-system Dysts and routing claims. Dense MLP is the reference baseline for the main forecasting and support-diagnostic table. Descriptive tables such as support refresh and support-coordinate intervention summaries report their uncertainty summaries but do not attach confirmatory p-values.

Point estimates. For the main controlled multibasin forecasting columns, each model is trained for 15 random seeds on each of 15 retained systems. For a model a, system s, horizon h, and

completed seeds r, the displayed MSE point estimate is

$$\frac{1}{|\mathcal{S}|}\sum_{s\in\mathcal{S}}\mathrm{IQM}_r\{\mathrm{MSE}_{a,s,r,h}\}\,,$$

where IQM is the empirical interquartile mean: the mean of finite values lying between the 25th and 75th percentiles, with a plain mean fallback for very small samples. The Dysts $dt\times30$ forecasting columns use the same seed-IQM-within-system, arithmetic-mean-across-systems estimator on 10 retained Dysts systems. The scale-normalized Dysts appendix table reports the arithmetic mean across systems of $\mathrm{IQM}_r(\mathrm{MSE}^{\mathrm{cand}}_{s,r,h})/\,\mathrm{IQM}_r(\mathrm{MSE}^{\mathrm{Dense}}_{s,r,h})$.

For the main support-alignment metric $H(B \mid F_{\mathrm{abs}})$, the point estimate is again a per-system seed IQM followed by an arithmetic mean across systems, evaluated on the per-basin deep slice. The family-count column $\overline{|F_{\mathrm{abs}}|}$ is descriptive: it averages per-system seed means because closeness to the represented basin count, not monotone increase or decrease, is the relevant interpretation. For appendix support-conditioned routing, the F_{top8} ratio is $10^{\mathrm{mean}_s\, D_{s,h}}$, where $D_{s,h}$ is the within-system IQM over finite seed-level $\log_{10}(\mathrm{MSE}^{\mathrm{routed}}\,/\,\mathrm{MSE}^{\mathrm{global}})$ values. Values below 1 favor the support-routed rollout.

The support-coordinate intervention table is descriptive. It reports mean $\pm$ sample standard deviation and median $[25\%, 75\%]$ accumulated MSE at $H = 21$. Coordinate dropping uses 100 held-out basin-interior starts. Random support shuffling uses 20 inactive-coordinate reassignments for each of the same 100 starts.

Bands and error bars. Forecasting horizon curves use the same point estimates as the tables. Their shaded bands are fixed-system seed-bootstrap 95% intervals after system-wise log-relative normalization, anchored to the displayed mean. Concretely, for each fixed system the implementation resamples seeds with replacement, recomputes the seed IQM, converts that draw to a $\log_{10}$-relative perturbation around that system's observed seed IQM, averages those perturbations across systems, and maps the 2.5th and 97.5th percentiles back onto the displayed mean. The default implementation uses 10,000 bootstrap replicates. These bands summarize finite-seed uncertainty for an average fixed benchmark system; they are not confidence intervals over a newly sampled population of dynamical systems.

The multibasin forest plot reports per-system paired $\log_{10}$-MSE effects at $H = 1000$. Points are per-system median paired seed differences, candidate minus Dense MLP, and whiskers are 95% bootstrap intervals over the common seeds for that system. Filled markers denote systems that pass the Holm-corrected Wilcoxon test. The $H(B \mid F_{\mathrm{abs}})$ strip plot shows system–seed points, gray first-to-third quartile bars, and black IQM summary bars. The coordinate-dropping intervention curves use bootstrap 95% intervals over the 100 initial states; random-support intervention curves show the interquartile range over shuffles, with median and mean curves shown separately.

Formal tests. For controlled multibasin forecasting against Dense MLP, the tested paired effect is

$$\delta_{s,r,h} = \log_{10}\mathrm{MSE}^{\mathrm{cand}}_{s,r,h} - \log_{10}\mathrm{MSE}^{\mathrm{Dense}}_{s,r,h}\,.$$

Within each system and model–horizon cell, the test is a one-sided paired Wilcoxon signed-rank test on common finite positive seed pairs with alternative $\delta < 0$, because lower MSE is better. System tests with fewer than four valid paired seeds or degenerate signed-rank inputs are omitted from that cell's denominator. The displayed main-table stars and appendix K/N counts use Holm-corrected Wilcoxon p-values at $\alpha = 0.05$.

For $H(B \mid F_{\text{abs}})$ and $|S_{\text{abs}}|$, the paired unit is again the matched training seed within a system, and the one-sided Wilcoxon alternative is candidate $<$ Dense MLP. For wrong-support/base ablation ratios in the appendix, the alternative is candidate $>$ Dense MLP, because a larger ratio means that replacing the active support is more damaging. The $\overline{|F_{\text{abs}}|}$ count is not assigned a p-value.

For the Dysts $dt\times30$ actual-MSE table, each system first contributes one seed-IQM MSE for the candidate and one for Dense MLP. The formal effect is the system-level $\log_{10}$ ratio of those two IQMs. Main-table Dysts superscripts use an exact one-sided sign test across the 10 systems, with alternative that more than half of systems have ratio below 1. Wilcoxon and paired t-test results are retained in sidecar outputs as audit diagnostics but are not used for the compact table stars.

For appendix F_{top8} support-conditioned routing, seeds are nested inside systems, so significance is tested at the system level. The test enumerates exact sign flips of the finite system-level log effects $D_{s,h}$ while retaining their magnitudes, with alternative $D_{s,h} < 0$. The displayed routing stars use Holm-corrected sign-flip p-values; the adjacent "system wins" count is a directional diagnostic, not the confirmatory p-value.

Holm corrections. All formal families use Holm step-down correction. The implementation sorts valid raw p-values, multiplies the ith smallest by the number of remaining hypotheses, enforces monotonicity by a running maximum, and clips at 1. Controlled multibasin forecasting and support-diagnostic tests correct across eligible systems within the corresponding model–metric–horizon cell. Dysts compact-table stars correct the exact sign-test p-values across the non-Dense model–horizon comparisons in the Dysts table. Routing stars correct across the displayed all-state F_{top8}-routing model–horizon cells. No Holm correction is applied to descriptive quantities without a formal p-value.

Assumptions and caveats. The one-sided alternatives are fixed by metric direction: lower MSE and lower $H(B \mid F_{\text{abs}})$ are better; larger wrong-support/base ratios are better. Log tests require finite positive MSEs or ratios. The paired Wilcoxon tests assume the matched seed differences within a system are the relevant exchangeable paired observations for that system; the Dysts and routing sign tests assume benchmark systems are the independent units for their cross-system claims. The fixed-system bootstrap bands condition on the chosen benchmark systems and quantify seed variability, not uncertainty over all possible dynamical systems. Basin labels, basin counts, and basin-depth slices are used only for benchmark evaluation and scoring, not for training the models or constructing support families at deployment.

The support-refresh table is a descriptive mechanism diagnostic: each cell reports the target-family fraction before re-encoding $\rightarrow$ after re-encoding for controlled transfer, and the no-transfer column reports a false-target control. The support-coordinate intervention figure and table are also descriptive: their bands, standard deviations, and interquartile ranges show the scale and dispersion of the perturbation effect, but no formal p-value or Holm correction is attached to those intervention panels.

S.8 Checklist: Compute Resources

Question. For each experiment, did we report the amount of compute and the type of resources used?

Answer. Yes.

Justification. The paper-facing experiments consist of the 15-system controlled multibasin benchmark, the 10-system Dysts $dt\times30$ benchmark, and several post-training evaluation diagnostics. The main training jobs used the cluster SLURM runners with CUDA enabled. The scripts request one GPU for training jobs, but do not pin a GPU model or constraint; the exact GPU type is therefore cluster-assigned and should be treated as unspecified. CPU-only post-training analyses use SLURM CPU partitions or CPU-mode jobs. The estimates below are allocation upper bounds derived from the paper protocol and the current SLURM resource requests, not measured wall-clock usage.

Hardware and resource details. Training jobs load CUDA 12.6.0 and request one cluster GPU, 4 CPU cores, and 16 GB host memory per active training allocation. Controlled multibasin training uses single-run GPU allocations with a 24-hour time limit. Dysts $dt\times30$ training uses packed GPU allocations with up to 12 runs per allocation, one GPU, 4 CPU cores, 16 GB memory, and a 72-hour time limit per packed allocation. Dysts cache construction uses CPU allocations with 4 CPU cores, 16 GB memory, and a 24-hour time limit. Dysts long-horizon evaluation uses CPU allocations with 4 CPU cores, 24 GB memory, and a 6-hour time limit for the paper-facing $dt\times30$ evaluation queue. Controlled support diagnostics and support-routed analyses use CPU allocations with 4 CPU cores, 16–24 GB memory, and 8–12-hour time limits, depending on the diagnostic.

Per-run and total-compute estimates.

Experiment	Paper-facing count	Per-run or per-allocation request	Allocation upper bound
Controlled multibasin training	15 systems × 6 model rows × 15 seeds = 1350 training runs	One GPU, 4 CPU cores, 16 GB memory, 24 h per run; 200,000 optimization steps, batch size 256, rollout length $L = 8$, latent dimension $d_z = 256$	$\leq$ 32,400 GPU-hours and $\leq$ 129,600 allocated CPU-core-hours
Dysts dt×30 training	10 systems × 6 model rows × 15 seeds = 900 training runs	Packed allocations: one GPU, 4 CPU cores, 16 GB memory, 72 h for up to 12 runs; 100,000 optimization steps, batch size 256, rollout length $L = 10$, latent dimension $d_z = 256$	$\leq$ 5400 GPU-hours and $\leq$ 21,600 allocated CPU-core-hours when packs are full
Dysts cache construction and long-horizon evaluation	10 retained systems; evaluation rows match the 900 trained Dysts runs	Cache jobs: 4 CPU cores, 16 GB, 24 h. Evaluation packs: 4 CPU cores, 24 GB, 6 h for up to 12 evaluation rows, plus one validation row and one collector	Approximately $\leq$ 4700 CPU-core-hours for train/validation/test cache construction, packed evaluation, validation, and collection
Controlled support diagnostics for the main table	6 model rows on the retained 15-system benchmark	CPU reducer shards: 4 CPU cores, 16 GB, 12 h per model row; merge jobs: 1 CPU core, 4 GB, 0.5 h	Approximately $\leq$ 290 CPU-core-hours for the six-row per-basin deep-slice support diagnostic pass
Appendix routed forecasting and support refresh diagnostics	Run on saved checkpoints; shard count depends on selected model rows and seed splits	CPU shards: 4 CPU cores, 24 GB, 12 h per model-row/seed-split shard; merges: 1 CPU core, 4 GB, 0.5 h	For 6 rows and three 5-seed splits, $\leq$ 864 CPU-core-hours per diagnostic family, plus merge overhead
Support-coordinate intervention figure	One representative trained checkpoint	One GPU, 1 CPU core, 8 GB memory, 3 h; plotting uses 1 CPU core, 2 GB, 10 min	$\leq$ 3 GPU-hours, plus negligible CPU plotting time

Caveats. The totals above are conservative allocation ceilings. Actual consumed compute can be lower because SLURM jobs may finish before their time limits, completed runs may be skipped, and packed jobs share one allocation across several sequential runs. Conversely, the repository contains older and supplemental launchers for 17-system controlled packets and 12-system Dysts packets; those extra runs are not included in the paper-facing totals above because the manuscript reports the retained 15-system controlled benchmark and the retained 10-system Dysts benchmark. The scripts

do not specify a GPU model, so the hardware type is reported only as a single cluster-assigned CUDA GPU per training allocation.

S.9 Checklist: Code of Ethics

Question. Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics?

Answer (Yes/No/NA). Yes.

Justification. The paper studies sparse Koopman autoencoders for forecasting and interpreting nonlinear dynamical systems. The experiments reported in the manuscript use simulation and benchmark trajectory data: a procedurally generated collection of two-dimensional multibasin dynamical systems, and a ten-system three-dimensional Dysts forecasting benchmark from the pinned project environment. The models observe stored trajectory states. They do not collect, process, infer from, or release human-subject data, personal information, private records, medical data, demographic attributes, user-generated content, or sensitive social data.

The manuscript also separates training-time assumptions from benchmark-only evaluation information. In the intended training and deployment setting, the method is not given basin labels, the number of basins, or trajectory-to-basin assignments. When benchmark basin labels or attractor metadata are available, they are used after training for evaluation slices, diagnostic scoring, controlled-transfer construction, and architecture diagnostics, not as ordinary training supervision for the support-selection mechanism. This keeps the paper's claims focused on label-free representation learning from simulated dynamical trajectories rather than on decisions about people or protected groups.

Anonymization and ethics considerations. The manuscript is prepared under anonymous authorship. Because the experiments use synthetic or public benchmark dynamical-system trajectories rather than participant data, no participant de-identification, consent process, or institutional human-subject review is implicated by the reported experiments. The procedural benchmark construction and the Dysts source should remain clearly disclosed and cited, including the package version used for the paper-facing benchmark. Any released code, logs, tables, or supplementary artifacts should be checked before submission for accidental disclosure of author identity, local filesystem paths, usernames, cluster job metadata, or other non-anonymous provenance.

Caveats. The paper is methodological and simulation-based. Its forecasting and interpretability results should not be presented as validated evidence for safety-critical control, clinical decision support, surveillance, or other deployment contexts where prediction errors could cause direct harm. If future versions add real-world datasets, human-generated measurements, proprietary data, biological or medical data, or safety-critical deployment claims, the ethics analysis should be revisited for consent, privacy, licensing, dual-use risk, and domain-specific review requirements. If code or artifacts are released, the authors should also verify external benchmark licensing and venue disclosure requirements for generated benchmark content and compute use.

S.10 Checklist: Broader impacts

Question. Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer (Yes/No/NA). Yes.

Justification. This work studies representation learning for nonlinear dynamical systems with multiple basins of attraction. The method is intended to learn forecasting representations from trajectory data without being given basin labels, basin counts, or trajectory-to-basin assignments during training or deployment. Its main output is a sparse latent representation whose active-coordinate support can be inspected after training as a model-produced regime variable. The empirical claims are limited to controlled multibasin benchmarks, an external chaotic-flow forecasting stress test, support-coordinate interventions, and post-hoc basin-support alignment diagnostics. The work is therefore not a deployed decision system and does not directly involve human subjects, personal data, or social classification. Broader impacts arise primarily through possible downstream use in scientific modeling, forecasting, and control.

Potential positive impacts.

- The method may improve learned forecasting models for multistable scientific and engineering systems by avoiding a single dense latent representation that collapses incompatible local dynamics.
- The sparse support object gives practitioners an inspectable regime variable, which can make learned dynamical models easier to diagnose than purely dense latent rollouts.
- Because the intended training and deployment setting does not assume known basin counts or labeled trajectories, the approach may be useful in settings where regime annotations are unavailable or expensive to obtain.
- Better local forecasting and regime inspection could support safer model development for downstream tools such as world models, monitoring systems, and local control design, provided those tools include independent validation.

Negative and misuse considerations.

- Forecasting errors in multistable systems can be most consequential near basin boundaries, rare transitions, partial-observation regimes, or high-dimensional physical systems. The paper explicitly does not establish reliable behavior in all of these settings.
- A learned support family may be overinterpreted as a true physical basin or causal regime. In the experiments, ground-truth basin labels are used only after training to evaluate alignment on benchmark systems; the model itself does not prove that a support family is a real-world regime.
- If used inside autonomous control, robotics, or physical infrastructure pipelines without additional safeguards, incorrect forecasts or support switches could lead to unsafe actions.
- More accurate forecasting methods for dynamical systems can be dual use in the ordinary sense that they may improve both beneficial and harmful planning in physical domains.
- The experiments require training many neural models across systems and random seeds, so they carry ordinary computational cost and energy-use impacts, although the paper does not train or deploy a large foundation model.

Mitigations and caveats.

- The paper narrows its claims to label-free representation learning, forecasting, support-coordinate intervention evidence, and post-hoc basin-support alignment on held-out benchmark states.

- Basin labels and known basin counts are treated as evaluation-only information for benchmark diagnostics, not as assumptions available to the training-time method.
- The discussion identifies important limitations: behavior near separatrices, rare transitions, partial observation, and high-dimensional physical systems remains unresolved.
- Downstream deployments should validate forecasts on held-out trajectories from the target system, monitor support changes and uncertainty near regime boundaries, and use independent safety constraints rather than treating the learned support as an oracle.
- For safety-critical control applications, this method should be used only as one component of a validated pipeline with conservative fallback behavior and domain-specific review.

S.11 Checklist: Safeguards

Question. If the authors are releasing assets such as code, data, or models that could be used for harmful purposes, did they follow appropriate safeguards?

Answer (Yes/No/NA). NA.

Justification. Based on the current manuscript and release-facing repository files, the paper does not release high-risk misuse assets. The released material is methodological research infrastructure for sparse Koopman autoencoders: training and evaluation code, low-dimensional dynamical-system benchmark definitions, paper figures and tables, and numerical result artifacts such as forecasting rows. The paper-facing experiments use procedurally generated two-dimensional multibasin systems and a ten-system Dysts $dt \times 30$ forecasting benchmark of three-dimensional autonomous flows. These are simulation and benchmark trajectory settings; they do not involve human subjects, personal information, private records, surveillance data, clinical data, biological sequence data, cyber-security exploits, weapons-relevant instructions, or other content that would itself create a high-risk misuse pathway.

The current release layout also does not treat trained model checkpoints as released assets. Training outputs such as `runs/`, `results/`, checkpoint files, and generated rollout tensors are excluded from the normal versioned release path. The manuscript describes checkpoints as internal training artifacts used for evaluation, not as deployed or public models for a safety-critical domain. The models are generic predictors over stored dynamical-system states and are not presented as autonomous control policies, decision systems, or domain-validated tools for physical deployment.

Caveats. This answer applies to the current paper-facing artifact set. It should be revisited if a later release includes trained checkpoints, large generated trajectory caches, domain-specific control policies, real-world robotics or biomedical datasets, human-generated measurements, private logs, or any artifact intended for safety-critical deployment. Even for the current low-risk release, the final artifact bundle should be checked for ordinary research-release issues such as external benchmark licensing, accidental local filesystem paths, cluster job metadata, usernames, or non-anonymous provenance.

S.12 Checklist: Licenses for existing assets

Question. Are the creators or original owners of assets, including code, data, models, benchmark definitions, third-party software, generated figures or tables, and model artifacts used in the paper, properly credited, with license and terms of use explicitly mentioned and respected?

Answer (Yes/No/NA). No.

Justification. The current manuscript uses existing software and one external benchmark source, and it also contains author-generated paper figures and tables derived from experiment results. The locally supported license and credit information is summarized below, but the check is not fully satisfied because some asset licenses or release terms still need confirmation from local evidence before submission or artifact release. In particular, the installed Dysts package has locally conflicting license indicators, and the final release terms for LLM-assisted benchmark definitions and generated paper assets should be stated explicitly.

Assets used.

- **Repository code and documentation.** The project code, paper source, appendix source, and generated documentation artifacts are in this repository. The paper source of truth is `docs/neurips_sparse_koopman_multibasin.tex`.
- **Paper figures and tables.** The manuscript includes generated PDF figures and TeX tables from `docs/figures/neurips_paper_2026/`, including forecasting curves, support-coordinate intervention plots, support-family entropy strips, training-dynamics diagnostics, support-codebook diagnostics, per-system tables, routing tables, refresh tables, and Dysts forecasting summaries. These are experiment-derived artifacts, not external photographic or artistic assets.
- **Procedural multibasin benchmark.** The main benchmark contains 15 two-dimensional multibasin systems. The manuscript appendix states that these systems were generated procedurally with Claude Opus 4.5 and then represented as analytic dynamical-system definitions and benchmark metadata.
- **Dysts benchmark subset.** The external forecasting stress test uses ten systems from Dysts version 0.96: Chua, Dadras, Dequan Li, Hadley, Lu–Chen–Cheng, Qi–Chen, Sakarya, San–Um–Srisuchinwong, Shimizu–Morioka, and Wang–Sun. The manuscript states that the continuous-time right-hand sides and metadata come from the pinned Dysts package version, and that the stored trajectories are generated and standardized for the project benchmark.
- **Third-party software dependencies.** The project dependency files and imported packages show use of PyTorch, NumPy, SciPy, Matplotlib, tqdm, pytest, ipykernel, JAX, Flax, Optax, `ml_collections`, Dysts, Numba, scikit-learn, and pandas. Pandas is imported by result-analysis and figure/table-building scripts even though it is not listed as a direct dependency in `pyproject.toml`.
- **Assets not used by the current manuscript.** Current source inspection found image files with ChatGPT-style names in the paper figure directory, but the current manuscript does not include them. They are therefore not treated as paper assets here. If they are later used or released as submission artifacts, their license and allowed-use terms need confirmation.

Known licenses and credit from local files.

- **Repository license.** The top-level `LICENSE` file is the MIT License, with copyright (*c*) 2025 Aidan Li. The `README.md` license section points readers to this file. The project metadata in `pyproject.toml` does not itself list a license field.
- **Dysts credit.** The manuscript credits Dysts through the Gilpin 2021 NeurIPS benchmark paper and the Gilpin 2023 Physical Review Research paper. The installed Dysts metadata also asks

users to cite those papers for published work and notes that Dysts stores attractor names, default parameter values, references, and other metadata in parseable JSON database files.

- **Dysts local license evidence.** The installed `dysts` 0.96 package metadata contains `Classifier: License :: OSI Approved :: MIT License`, but the bundled license file recorded by the same installed distribution is an Apache License 2.0 text. This inconsistency is local evidence that the Dysts license needs confirmation rather than a basis for choosing one license statement.
- **Direct and imported software package licenses.** Local package metadata records the following licenses or license classifiers for the main direct or imported dependencies inspected: PyTorch 2.9.1, BSD-3-Clause; NumPy 2.1.2, BSD classifier; SciPy 1.15.3, BSD classifier; Matplotlib 3.10.7, Matplotlib license with PSF license classifier; tqdm 4.66.5, MPL-2.0 and MIT; pytest 8.4.2, MIT; Numba 0.63.1, BSD; scikit-learn 1.7.2, BSD-3-Clause; pandas 2.3.3, BSD 3-Clause; JAX 0.6.2, Apache-2.0; Flax 0.10.7, Apache Software License classifier; Optax 0.2.6, Apache Software License classifier; `ml_collections` 1.1.0, Apache Software License classifier; and ipykernel 7.1.0, BSD-3-Clause.
- **Generated figures and tables.** The figures and tables included by the current manuscript appear to be generated from project code, local experiment outputs, and the benchmark sources described above. No local evidence was found in the current main paper source that any included figure is an external stock image, photograph, third-party illustration, or copied plot from another work.
- **Hosted Dysts time-series data.** The installed Dysts metadata mentions a hosted Hugging Face dataset of precomputed time series, but the current manuscript describes using Dysts right-hand sides and metadata to generate the project benchmark trajectories. No local evidence from the current paper source indicates that the hosted precomputed Dysts dataset is used as a manuscript asset.

Outstanding license-confirmation items.

- Confirm the authoritative Dysts 0.96 license for the package code, system database, right-hand-side definitions, metadata, and any generated trajectory caches derived from them. Local files currently show an MIT classifier and an Apache-2.0 bundled license file.
- Confirm whether the selected Dysts systems require additional system-level attribution beyond the two Gilpin papers already cited. The installed Dysts metadata notes that parameter values are based on standard or published values and acknowledges earlier named-system collections, so the final paper or artifact release should verify whether Dysts citation alone is sufficient for the selected ten systems.
- State the intended release license for paper figures, TeX tables, generated benchmark definitions, result summaries, and trajectory caches. The repository has a top-level MIT license, but the final artifact package should make clear whether that license covers documentation, figures, tables, and derived data artifacts or whether those are distributed under separate terms.
- Confirm the publication and release terms for the Claude-assisted procedural multibasin benchmark definitions. The manuscript discloses Claude Opus 4.5 involvement, but local files do not by themselves establish the applicable generated-content license or any provider-specific attribution requirements.
- Exclude unused generated-image files from any paper artifact bundle, or confirm their source, allowed-use terms, and license before release if they are later included.

- If the code artifact is released, add explicit license metadata to `pyproject.toml` or the release documentation so package-level metadata agrees with the top-level MIT license.
- If transitive dependencies are redistributed inside an artifact rather than installed by users from their package sources, perform a full transitive-dependency license audit. The list above covers the inspected direct and commonly imported packages, not every package in `uv.lock`.

S.13 Checklist: New Assets

Question. Does the paper introduce new assets, such as code, datasets, benchmark systems, trained models, generated data, figures, tables, or other artifacts that are intended for release or reuse?

Answer (Yes/No/NA). Yes.

Justification. The paper introduces and uses several paper-specific assets. The primary asset is the source code for sparse Koopman autoencoders, including sparse LISTA-style encoders, dense and structured Koopman transitions, training entry points, evaluation utilities, benchmark adapters, result collectors, statistical summaries, and figure/table generation scripts. These assets are represented in the current repository by the core package, command-line tools, batch scripts, tests, dependency metadata, and paper source.

The paper also introduces a controlled 15-system two-dimensional multibasin benchmark. The manuscript describes these systems as procedurally generated with Claude Opus 4.5 and documents their system keys, generator families, basin counts, stored-step convention, and analytic vector-field definitions. The benchmark includes basin labels, basin counts, and attractor metadata only as evaluation annotations. In the intended training and deployment setting, the models are not given basin labels, basin counts, or trajectory-to-basin assignments.

The paper additionally uses a 10-system Dysts $dt \times 30$ forecasting benchmark. The Dysts equations and metadata come from the external `dysts` package version resolved in the project environment, documented as `dysts` 0.96 in the paper appendix. These public Dysts systems are not new assets introduced by this paper, but the stored-step protocol, cache construction settings, split conventions, evaluation scripts, and collected paper summaries are paper-specific generated artifacts.

Current repository search also identifies generated paper-facing figures, LaTeX tables, CSV/JSON metric sidecars, manifests, and support-family codebook artifacts under paths such as `docs/figures/neurips_paper_2026` and its `_tables` subdirectory. The training code writes best and latest model checkpoint files named `checkpoint.pt` and `last.pt`. In this checkout, `ls runs` reported no entries, but generated `results/` artifacts include some `.pt` stage-two local-map and checkpoint files from routed diagnostics. These checkpoint-like files are derived experiment artifacts and should be treated separately from the minimal paper release unless the authors explicitly select them for archival release.

Documentation, release, and anonymization status.

- **Code release.** The releasable code asset should include the core package, benchmark definitions and adapters, training and evaluation tools, paper-facing collection and plotting scripts, tests, the README, `pyproject.toml`, and `uv.lock`. The current repository includes an MIT license file, but the checkout's license metadata contains an author name; an anonymous-review release must

either use venue-approved anonymous licensing metadata or defer identity-revealing attribution until the public de-anonymized release.

- **Benchmark release.** The 15 procedural multibasin systems should be released as explicit code and documentation, not as an opaque generated dataset. The release should preserve the disclosure that Claude Opus 4.5 was used for procedural benchmark generation. Generated training, validation, and test trajectories can be regenerated from the released vector fields and stored-step protocol. Evaluation-only basin metadata should be clearly marked as unavailable to training-time support selection.
- **Dysts protocol release.** The Dysts benchmark should be released as adapter code, system lists, pinned dependency metadata, stored-step settings, cache split conventions, and evaluation commands. Because the underlying Dysts systems are external public assets, the release should cite Dysts and respect its license rather than redistributing it as a new dataset.
- **Generated figures and tables.** The paper-facing figure files, LaTeX tables, and compact CSV/JSON summaries used by the manuscript should be released or archived with enough metadata to trace each display to the corresponding benchmark, model row, seed aggregation rule, and statistical test. Exploratory figures, failed pilot outputs, and scratch manifests are not required assets unless they support a reported claim.
- **Checkpoints and large generated files.** Exhaustive training runs, private scratch directories, large Dysts cache tensors, and all exploratory checkpoints do not need to be released if the code and protocols regenerate them. If any model checkpoints or second-stage local-map files are released, each file should be paired with its model configuration, system, seed, training budget, checkpoint-selection rule, evaluation command, and checksum.
- **Anonymization.** The anonymous artifact should remove or neutralize author names, local usernames, absolute private paths, private scratch locations, SLURM job logs, queue records, and other cluster-specific provenance that is not necessary for reproduction. No human-subject data, personal data, user logs, images of people, medical records, or proprietary datasets are introduced by the paper assets.

Caveats. This checklist item records that new assets exist; it does not by itself verify that a final public archive, DOI, or anonymous review URL has been prepared. Before submission, the authors should make an explicit release decision for model checkpoints and generated cache tensors, because the paper can be reproduced from code and protocols but reviewers may still value selected paper-facing checkpoints for auditability. The distinction between benchmark metadata used for evaluation and information unavailable during training should remain visible in the release documentation. If future versions add real-world datasets, human-generated measurements, proprietary data, or safety-critical deployment artifacts, this answer should be revisited.

S.14 Checklist: Crowdsourcing and Research with Human Subjects

Question. For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation?

Answer (Yes/No/NA). NA.

Justification. The manuscript reports a methodological and empirical study of sparse Koopman autoencoders for nonlinear dynamical systems. The reported data are numerical trajectory observations from controlled dynamical-system benchmarks: a procedurally generated collection of two-dimensional multibasin systems and a ten-system chaotic-flow benchmark from Dysts. The models train and evaluate on stored state trajectories, reconstruction and forecasting losses, rollout mean squared error, support-coordinate interventions, and post-hoc support-basin alignment diagnostics.

No part of the reported study recruits people, assigns tasks to crowdworkers or annotators, runs a user study or survey, collects participant responses, or uses human-generated labels. The benchmark basin labels discussed in the paper come from dynamical-system generator metadata and known attractor geometry, and are used only for benchmark evaluation or explicitly described diagnostic purposes. They are not labels supplied by human subjects or crowdworkers.

A current source search of the manuscript and repository code paths used for models, tools, tests, experiments, and scripts found no evidence of human-subject protocols, participant recruitment, informed consent, IRB review, crowdsourcing platforms, surveys, user studies, annotators, or personal data. The work therefore does not trigger the crowdsourcing or human-subject reporting requirements for the experiments described in the current paper.

Caveats. This answer is limited to the current manuscript and current repository contents. If a later version adds real-world measurements from people, human-generated annotations, behavioral data, medical or biological human data, private records, user studies, surveys, or crowdworker tasks, this checklist item should be changed and the paper should report the relevant ethics review, consent process, participant protections, compensation, task instructions, data handling, privacy safeguards, and release restrictions. External benchmark licensing and artifact-release checks remain separate reproducibility and open science issues; they do not by themselves make the reported experiments human-subject research.

S.15 Checklist: Institutional review board approval

Question. Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board approvals or an equivalent approval/review were obtained?

Answer (Yes/No/NA). NA.

Justification. The reported work does not involve human subjects. The manuscript studies sparse Koopman autoencoders on numerical dynamical-system trajectory data. The main benchmark consists of procedurally generated two-dimensional multibasin systems with known attractor geometry, and the additional forecasting benchmark uses public chaotic-flow systems from the Dysts package. The observations are simulated state vectors, generated trajectories, basin metadata used only for benchmark evaluation, model checkpoints, and aggregate forecasting or support-diagnostic metrics.

No part of the described experimental protocol recruits, observes, surveys, intervenes on, or collects data from people. The experiments do not use participant records, medical or clinical data, user logs, demographic attributes, private text, images, audio, biometric measurements, or other personally identifiable or sensitive human-subject data. A search of the current manuscript and source/script/test files for human-subjects terms such as IRB, institutional review, participant, patient, survey, consent, privacy, and personally identifiable data did not identify any human-subjects component. Institutional review board approval is therefore not applicable to the current paper.

Caveats. This answer applies only to the current manuscript and current experimental code. Benchmark basin labels, attractor coordinates, and basin counts are labels for simulated dynamical systems, not labels for people or protected groups. The external Dysts dependency should still be disclosed, cited, and checked for licensing requirements, but its use does not by itself create a human-subjects review requirement.

If a later revision adds a user study, human-generated data, medical or clinical measurements, user logs, private records, demographic attributes, biometric data, surveys, interviews, safety-critical deployment with observed people, or any dataset whose subjects are people, this checklist answer should be changed and the authors should obtain and report the appropriate human- subjects review, consent, privacy, and data-governance approvals before submission.

S.16 Checklist: Declaration of LLM Usage

Question. Does the paper describe the usage of LLMs if they are an important, original, or non-standard component of the core methods in this research, while distinguishing any writing, editing, or formatting assistance from the scientific methodology?

Answer. Yes.

Justification. The current manuscript describes the proposed method as a sparse Koopman autoencoder: an encoder, linear decoder, latent Koopman transition, sparse training objective, and post-hoc support-family diagnostics. The paper does not describe an LLM as part of the learned predictor, training loss, inference procedure, rollout evaluation, support-family construction, or statistical testing protocol.

The manuscript does disclose LLM involvement in benchmark construction. In the multibasin benchmark appendix, the 15 two-dimensional multibasin benchmark systems are described as having been generated procedurally with Claude Opus 4.5. Current source inspection also finds Claude-specific benchmark catalog adapters, manifests, plotting utilities, validation utilities, and queueing scripts. These artifacts support benchmark definition and experiment infrastructure; they do not make an LLM a runtime component of the SKAE models or of the reported forecasting and support-alignment evaluations.

Distinction between core methodology and assistance.

- **Core methodology: No LLM component.** The core method is the sparse Koopman autoencoder and its associated support-based evaluation. Model training and evaluation use numerical trajectory data, PyTorch model components, deterministic benchmark code, and post-hoc metrics. No LLM is used to encode states, choose supports, advance rollouts, assign support families, compute losses, select basin labels, or produce the reported experimental measurements.
- **Benchmark construction: LLM-assisted.** Claude Opus 4.5 was used to procedurally generate the multibasin benchmark systems reported in the benchmark inventory. These systems are subsequently represented as explicit dynamical-system code and benchmark metadata. Basin labels and basin counts are used only as benchmark metadata for evaluation and diagnostic slicing, except where the manuscript explicitly notes their use to set fixed architecture group counts for structured-transition ablations.
- **Writing and editing assistance: LLM-assisted.** LLM assistance was used to draft and edit this checklist declaration. Such assistance is language-level documentation support and is

separate from the scientific method, model implementation, experiment execution, and numerical results.

Caveats.

- This declaration is based on the current manuscript and current code and documentation search, with the manuscript treated as the source of truth.
- The search confirms named Claude-related benchmark and infrastructure artifacts, but it is not a broader provenance audit of unpublished drafting records or external notebooks.
- The LLM-assisted benchmark-generation disclosure should be retained in any final submission version that uses these 15 multibasin systems. If additional LLM assistance is used later for manuscript text, code generation, experiment design, or artifact preparation, this declaration should be updated before submission.

References

Ali Abbasi, Parsa Nooralinejad, Vladimir Braverman, Hamed Pirsiavash, and Soheil Kolouri. Sparsity and Heterogeneous Dropout for Continual Learning in the Null Space of Neural Activations. In *Proceedings of The 1st Conference on Lifelong Learning Agents*, pages 617–628. PMLR, November 2022. URL https://proceedings.mlr.press/v199/abbasi22a.html.

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C Courville, and Marc Bellemare. Deep reinforcement learning at the edge of the statistical precipice. *Advances in Neural Information Processing Systems*, 34, 2021.

Omri Azencot, N. Benjamin Erichson, Vanessa Lin, and Michael Mahoney. Forecasting Sequential Data Using Consistent Koopman Autoencoders. In *Proceedings of the 37th International Conference on Machine Learning*, pages 475–485. PMLR, November 2020.

Amir Beck and Marc Teboulle. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM Journal on Imaging Sciences*, 2(1):183–202, 2009. doi: 10.1137/080716542.

Steven L. Brunton, Bingni W. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Koopman Invariant Subspaces and Finite Linear Representations of Nonlinear Dynamical Systems for Control. *PLOS ONE*, 11(2), February 2016. ISSN 1932-6203. doi: 10.1371/journal.pone.0150171. URL https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0150171.

Steven L. Brunton, Marko Budišić, Eurika Kaiser, and J. Nathan Kutz. Modern Koopman Theory for Dynamical Systems. *SIAM Review*, 64(2):229–340, May 2022. ISSN 0036-1445. doi: 10.1137/21M1401243. URL https://doi.org/10.1137/21M1401243.

Scott Shaobing Chen, David L. Donoho, and Michael A. Saunders. Atomic Decomposition by Basis Pursuit. *SIAM Journal on Scientific Computing*, 20(1):33–61, 1998. doi: 10.1137/S1064827596304010. URL https://doi.org/10.1137/S1064827596304010.

Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models, 2023. URL https://arxiv.org/abs/2309.08600.

David L. Donoho and Michael Elad. Optimally sparse representation in general (nonorthogonal) dictionaries via ℓ_1 minimization. *Proceedings of the National Academy of Sciences*, 100(5): 2197–2202, March 2003. doi: 10.1073/pnas.0437847100. URL https://doi.org/10.1073/pnas.0437847100.

Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. Toy models of superposition, 2022. URL https://arxiv.org/abs/2209.10652.

Mahan Fathi, Clement Gehring, Jonathan Pilault, David Kanaa, Pierre-Luc Bacon, and Ross Goroshin. Course correcting koopman representations. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=A18gWgc5mi.

Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders, 2024. URL https://arxiv.org/abs/2406.04093.

William Gilpin. Chaos as an interpretable benchmark for forecasting and data-driven modelling. In *Advances in Neural Information Processing Systems (NeurIPS) 2021*, 2021. URL http://arxiv.org/abs/2110.05266.

William Gilpin. Model scale versus domain knowledge in statistical forecasting of chaotic systems. *Physical Review Research*, 5(4):043252, December 2023. doi: 10.1103/PhysRevResearch.5.043252. URL https://link.aps.org/doi/10.1103/PhysRevResearch.5.043252.

Karol Gregor and Yann LeCun. Learning Fast Approximations of Sparse Coding. In *Proceedings of the 27th International Conference on Machine Learning*, ICML'10, pages 399–406, Haifa, Israel, 2010. Omnipress. ISBN 978-1-60558-907-7. doi: 10.5555/3104322.3104374. URL https://dl.acm.org/doi/abs/10.5555/3104322.3104374.

D. M. Grobman. Homeomorphism of systems of differential equations. *Doklady Akademii Nauk SSSR*, 128:880–881, 1959.

Lars Grüne and Jürgen Pannek. *Nonlinear Model Predictive Control*. Communications and Control Engineering. Springer International Publishing, Cham, 2017. ISBN 978-3-319-46023-9 978-3-319-46024-6. doi: 10.1007/978-3-319-46024-6. URL http://link.springer.com/10.1007/978-3-319-46024-6.

Philip Hartman. A lemma in the theory of structural stability of differential equations. *Proceedings of the American Mathematical Society*, 11(4):610–620, 1960. doi: 10.1090/S0002-9939-1960-0121542-7.

R.E. Kalman. On the general theory of control systems. *1st International IFAC Congress on Automatic and Remote Control, Moscow, USSR, 1960*, 1(1):491–502, August 1960. ISSN 1474-6670. doi: 10.1016/S1474-6670(17)70094-8. URL https://www.sciencedirect.com/science/article/pii/S1474667017700948.

Adam Karvonen, Can Rager, Johnny Lin, Curt Tigges, Joseph Bloom, David Chanin, Yeu-Tong Lau, Eoin Farrell, Callum McDougall, Kola Ayonrinde, Demian Till, Matthew Wearden, Arthur Conmy, Samuel Marks, and Neel Nanda. SAEBench: A comprehensive benchmark for sparse autoencoders in language model interpretability. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025. URL https://arxiv.org/abs/2503.09532.

B. O. Koopman. Hamiltonian Systems and Transformation in Hilbert Space. *Proceedings of the National Academy of Sciences*, 17(5):315–318, May 1931. doi: 10.1073/pnas.17.5.315. URL https://www.pnas.org/doi/10.1073/pnas.17.5.315.

B. O. Koopman and J. v. Neumann. Dynamical Systems of Continuous Spectra. *Proceedings of the National Academy of Sciences*, 18(3):255–263, March 1932. doi: 10.1073/pnas.18.3.255. URL https://www.pnas.org/doi/abs/10.1073/pnas.18.3.255.

Milan Korda and Igor Mezić. Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control. *Automatica*, 93:149–160, July 2018. ISSN 00051098. doi: 10.1016/j.automatica.2018.03.046. URL http://arxiv.org/abs/1611.03537.

Matthew D. Kvalheim and Shai Revzen. Existence and uniqueness of global Koopman eigenfunctions for stable fixed points and periodic orbits. *Physica D: Nonlinear Phenomena*, 425:132959, November 2021. ISSN 0167-2789. doi: 10.1016/j.physd.2021.132959. URL https://www.sciencedirect.com/science/article/pii/S0167278921001160.

Yueheng Lan and Igor Mezić. Linearization in the large of nonlinear systems and Koopman operator spectrum. *Physica D: Nonlinear Phenomena*, 242(1):42–53, January 2013. ISSN 0167-2789. doi: 10.1016/j.physd.2012.08.017.

Jessy Lin, Luke Zettlemoyer, Gargi Ghosh, Wen-Tau Yih, Aram Markosyan, Vincent-Pierre Berges, and Barlas Oguz. Continual learning via sparse memory finetuning, 2025. URL https://arxiv.org/abs/2510.15103.

Zexiang Liu, Necmiye Ozay, and Eduardo D. Sontag. Properties of immersions for systems with multiple limit sets with implications to learning Koopman embeddings. *Automatica*, 176: 112226, June 2025. ISSN 0005-1098. doi: 10.1016/j.automatica.2025.112226. URL https://www.sciencedirect.com/science/article/pii/S0005109825001189.

Bethany Lusch, J. Nathan Kutz, and Steven L. Brunton. Deep learning for universal linear embeddings of nonlinear dynamics. *Nature Communications*, 9(1):4950, November 2018. ISSN 2041-1723. doi: 10.1038/s41467-018-07210-0.

Aleksandar Makelov, George Lange, and Neel Nanda. Towards principled evaluations of sparse autoencoders for interpretability and control, 2024. URL https://arxiv.org/abs/2405.08366.

Arun Mallya and Svetlana Lazebnik. PackNet: Adding multiple tasks to a single network by iterative pruning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018. URL https://arxiv.org/abs/1711.05769.

Giorgos Mamakoukas, Maria Castano, Xiaobo Tan, and Todd Murphey. Local Koopman Operators for Data-Driven Control of Robotic Systems. In *Robotics: Science and Systems XV*. Robotics: Science and Systems Foundation, June 2019. ISBN 978-0-9923747-5-4. doi: 10.15607/RSS.2019.XV.054. URL http://www.roboticsproceedings.org/rss15/p54.pdf.

D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert. Constrained model predictive control: Stability and optimality. *Automatica*, 36(6):789–814, June 2000. ISSN 0005-1098. doi: 10.1016/S0005-1098(99)00214-9. URL https://www.sciencedirect.com/science/article/pii/S0005109899002149.

Igor Mezić. Spectral properties of dynamical systems, model reduction and decompositions. *Nonlinear Dynamics*, 41(1–3):309–325, 2005. doi: 10.1007/s11071-005-2824-x.

Ruben Ohana, Michael McCabe, Lucas Meyer, Rudy Morel, Fruzsina J. Agocs, et al. The well: A large-scale collection of diverse physics simulations for machine learning. In *Advances in Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2412.00568.

Bruno A. Olshausen and David J. Field. Emergence of simple-cell receptive field properties by learning a sparse code for natural images. *Nature*, 381(6583):607–609, June 1996. ISSN 1476-4687. doi: 10.1038/381607a0. URL https://doi.org/10.1038/381607a0.

Samuel E. Otto and Clarence W. Rowley. Linearly recurrent autoencoder networks for learning dynamics. *SIAM Journal on Applied Dynamical Systems*, 18(1):558–593, 2019. doi: 10.1137/18M1177846.

Shaowu Pan and Karthik Duraisamy. On the lifting and reconstruction of nonlinear systems with multiple invariant sets. *Nonlinear Dynamics*, 112(12):10157–10165, June 2024. ISSN 1573-269X. doi: 10.1007/s11071-024-09581-0. URL https://doi.org/10.1007/s11071-024-09581-0.

Clarence W. Rowley, Igor Mezić, Shervin Bagheri, Philipp Schlatter, and Dan S. Henningson. Spectral analysis of nonlinear flows. *Journal of Fluid Mechanics*, 641:115–127, 2009. doi: 10.1017/S0022112009992059.

Peter J. Schmid. Dynamic mode decomposition of numerical and experimental data. *Journal of Fluid Mechanics*, 656:5–28, 2010. ISSN 0022-1120. doi: 10.1017/S0022112010001217.

Joan Serrà, Dídac Surís, Marius Miron, and Alexandros Karatzoglou. Overcoming catastrophic forgetting with hard attention to the task, 2018. URL https://arxiv.org/abs/1801.01423.

Haojie Shi and Max Q.-H. Meng. Deep Koopman Operator With Control for Nonlinear Systems. *IEEE Robotics and Automation Letters*, 7(3):7700–7707, July 2022. ISSN 2377-3766. doi: 10.1109/LRA.2022.3184036.

Makoto Takamoto, Timothy Praditia, Raphael Leiteritz, Dan MacKinlay, Francesco Alesiani, Dirk Pflüger, and Mathias Niepert. PDEBench: An extensive benchmark for scientific machine learning. In *Advances in Neural Information Processing Systems*, 2022. URL https://arxiv.org/abs/2210.07182.

Naoya Takeishi, Yoshinobu Kawahara, and Takehisa Yairi. Learning koopman invariant subspaces for dynamic mode decomposition. In *Advances in Neural Information Processing Systems 30*, volume 30, 2017. URL https://papers.nips.cc/paper/6713-learning-koopman-invariant-subspaces-for-dynamic-mode-decomposition.

Stone Tao, Fanbo Xiang, Arth Shukla, Yuzhe Qin, Xander Hinrichsen, Xiaodi Yuan, Chen Bao, Xinsong Lin, Yulin Liu, Tse-kai Chan, et al. ManiSkill3: GPU parallelized robotics simulation and rendering for generalizable embodied AI, 2024. URL https://arxiv.org/abs/2410.00425.

Jonathan H. Tu, Clarence W. Rowley, Dirk M. Luchtenburg, Steven L. Brunton, and J. Nathan Kutz. On dynamic mode decomposition: Theory and applications. *Journal of Computational Dynamics*, 1(2):391–421, December 2014. ISSN 2158-2491. doi: 10.3934/jcd.2014.1.391. URL https://www.aimsciences.org/en/article/doi/10.3934/jcd.2014.1.391.

Matthew O. Williams, Ioannis G. Kevrekidis, and Clarence W. Rowley. A Data–Driven Approximation of the Koopman Operator: Extending Dynamic Mode Decomposition. *Journal of Nonlinear Science*, 25(6):1307–1346, December 2015. ISSN 1432-1467. doi: 10.1007/s00332-015-9258-5. URL https://doi.org/10.1007/s00332-015-9258-5.

Matthew O. Williams, Clarence W. Rowley, and Ioannis G. Kevrekidis. A kernel-based method for data-driven koopman spectral analysis. *Journal of Computational Dynamics*, 2(2):247–265, May 2016. ISSN 2158-2491. doi: 10.3934/jcd.2015005. URL https://www.aimsciences.org/article/doi/10.3934/jcd.2015005.

Mitchell Wortsman, Vivek Ramanujan, Rosanne Liu, Aniruddha Kembhavi, Mohammad Rastegari, Jason Yosinski, and Ali Farhadi. Supermasks in superposition. In *Advances in Neural Information Processing Systems*, 2020. URL https://arxiv.org/abs/2006.14769.